Prerequisite Practical

AIM: Study of logic gates (AND, OR, NOT, NAND, NOR, EX-OR) using ICs and NAND and NOR as Universal Gate.

Components:

Sr No.	Component	Specification	Quantity
1	AND Gate	IC 7408	1
2	OR Gate	IC 7432	1
3	NOT Gate	IC 7404	1
4	NAND Gate	IC 7400	1
5	NOR Gate	IC 7402	1
6	EX-OR Gate	IC 7486	1

THEORY:

Circuit that takes the logical decision and the process are called logic gates. Each gate has one or more input and only one output.

OR, AND & NOT are basic gates. NAND, NOR, X-OR are known as universal gates. Basic gates can be obtained from all this gates.

AND Gate:

The AND gate performs a logical multiplication commonly known as AND function. The output is high only when both the inputs are either one high or one low. When both the inputs are high the output is low level.

OR Gate:

The OR gate performs a logical addition commonly known as OR function. The output is high when any one of the inputs is high and the output is low level when both the inputs are low.

NOT Gate:

The NOT gate is called an inverter. The output is high when the input is low. The output is low when the input is high.

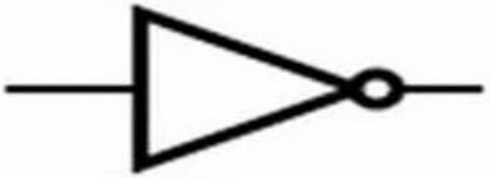
NAND Gate:

The NAND gate is a contraction of AND-NOT. The output is high when both inputs are low and any one of the input is low. The output is low level when the inputs are high.

NOR Gate:
The NOR gate is contraction of OR-NOT. The output is high when both inputs are low. The output is low when one or both inputs are high.

EX-OR Gate:
The output is high when any one of the input is high. The output is also low when both the inputs are low and both inputs are high.

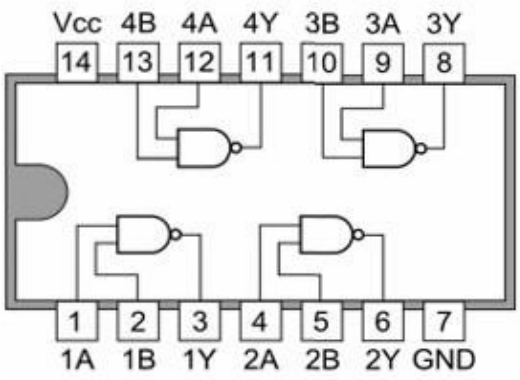
Digital Logic Gate Symbols

Gate	Symbol	Notation	Truth Table
AND		$A \cdot B$	INPUT OUTPUT A B A AND B 0 0 0 0 1 0 1 0 0 1 1 1
OR		$A + B$	INPUT OUTPUT A B A OR B 0 0 0 0 1 1 1 0 1 1 1 1
NOT		$\overline{A}$	INPUT OUTPUT A NOT A 0 1 1 0
NAND		$\overline{A \cdot B}$	INPUT OUTPUT A B A NAND B 0 0 1 0 1 1 1 0 1 1 1 0
NOR		$\overline{A + B}$	INPUT OUTPUT A B A NOR B 0 0 1 0 1 0 1 0 0 1 1 0
XOR		$A \oplus B$	INPUT OUTPUT A B A XOR B 0 0 0 0 1 1 1 0 1 1 1 0

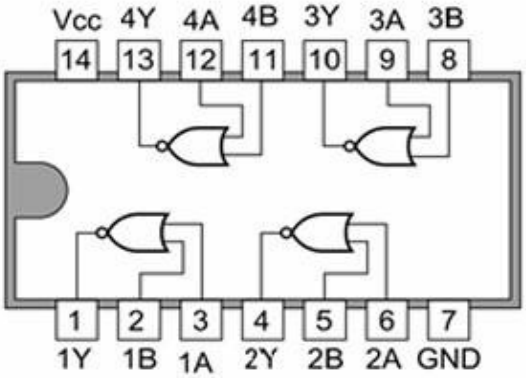
LOGIC GATES USING IC:

IC's PIN DIAGRAMS:

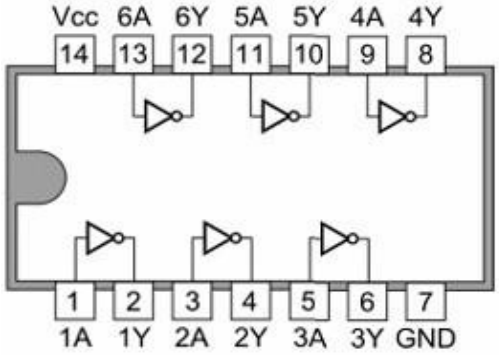
7400 Quad 2-input NAND Gates



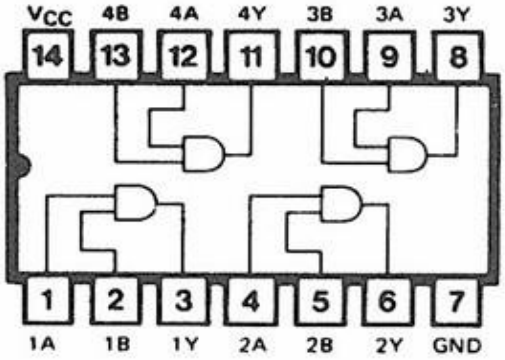
7402 Quad 2-input NOR Gates



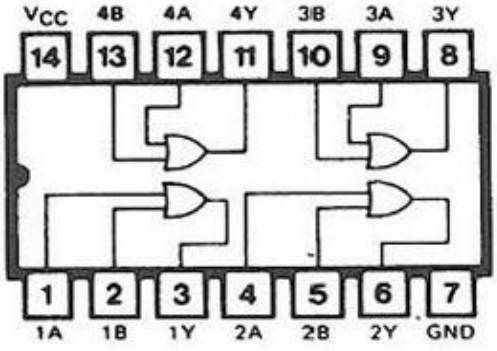
7404 Hex Inverters



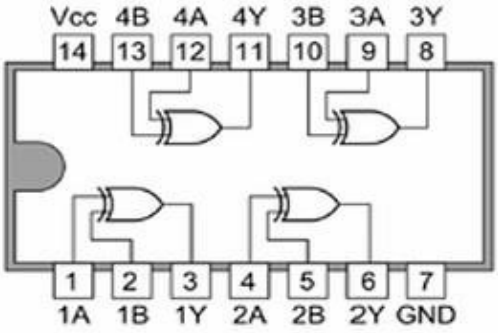
7408 AND Gates



7432 OR Gates

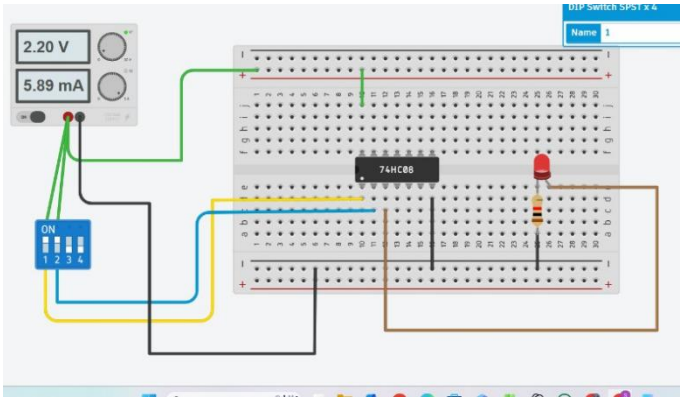


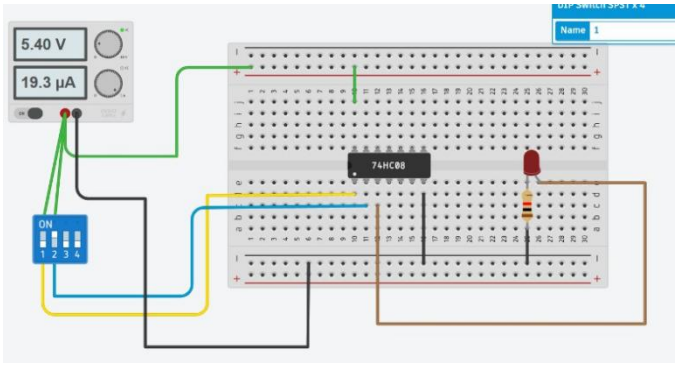
7486 Quad 2-input ExOR Gates

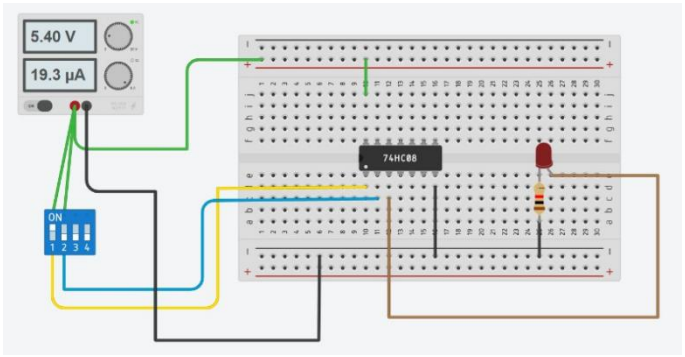


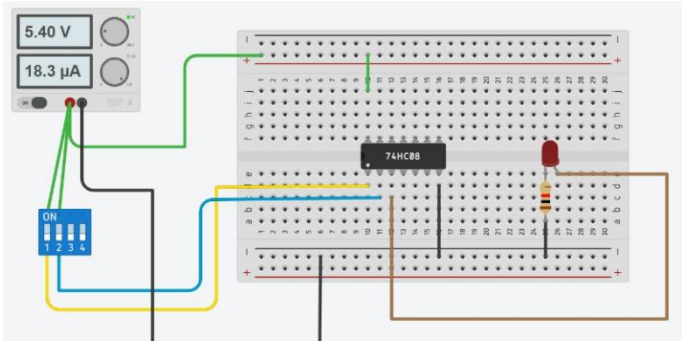
SCREENSHOTS:

1. And gate:

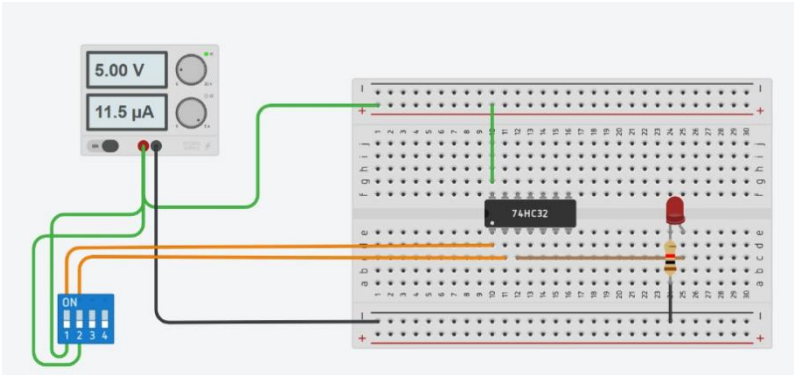


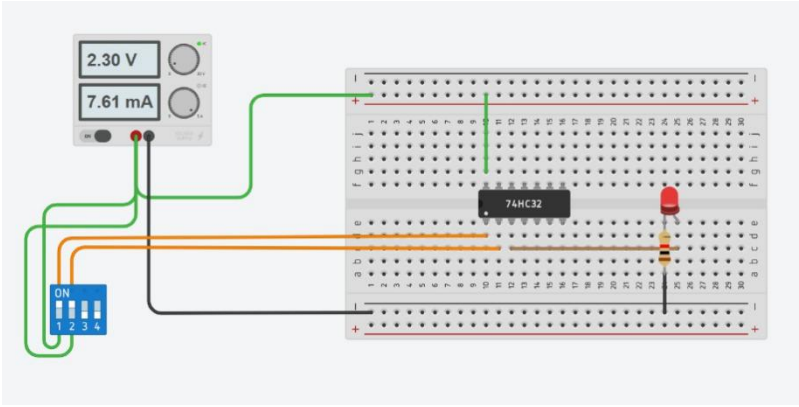


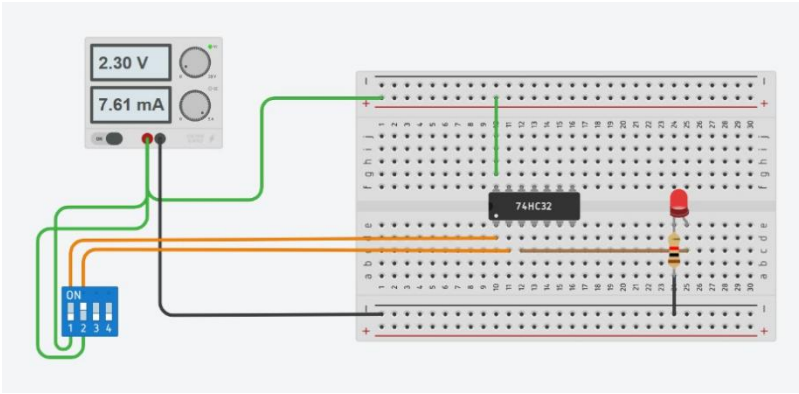


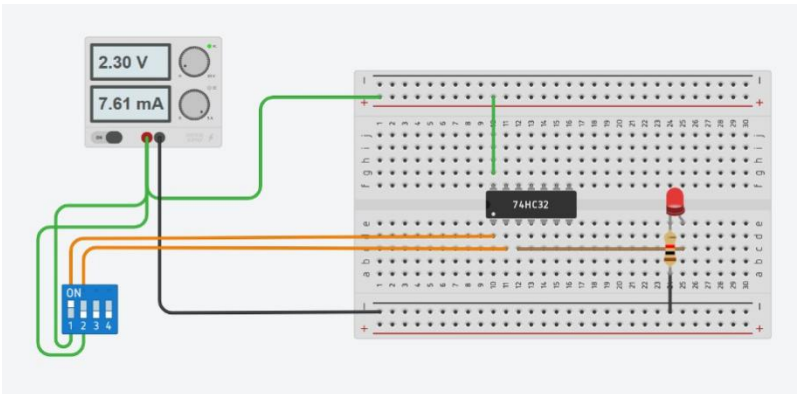


2. Or gate:

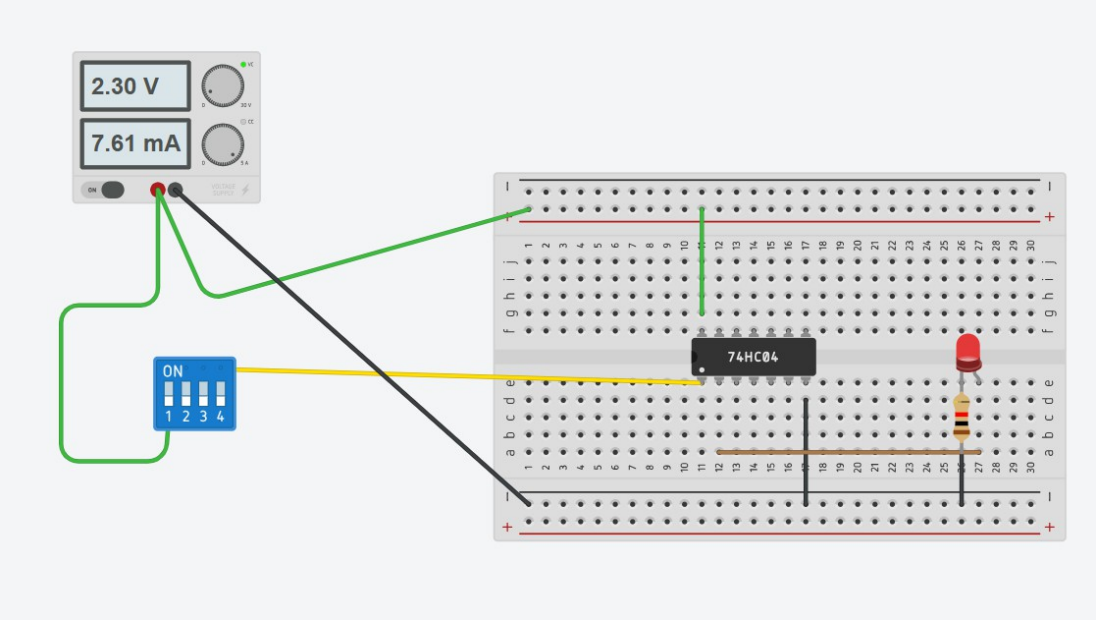

5.00 V
11.5 µA
74HC32
ON
1 2 3 4

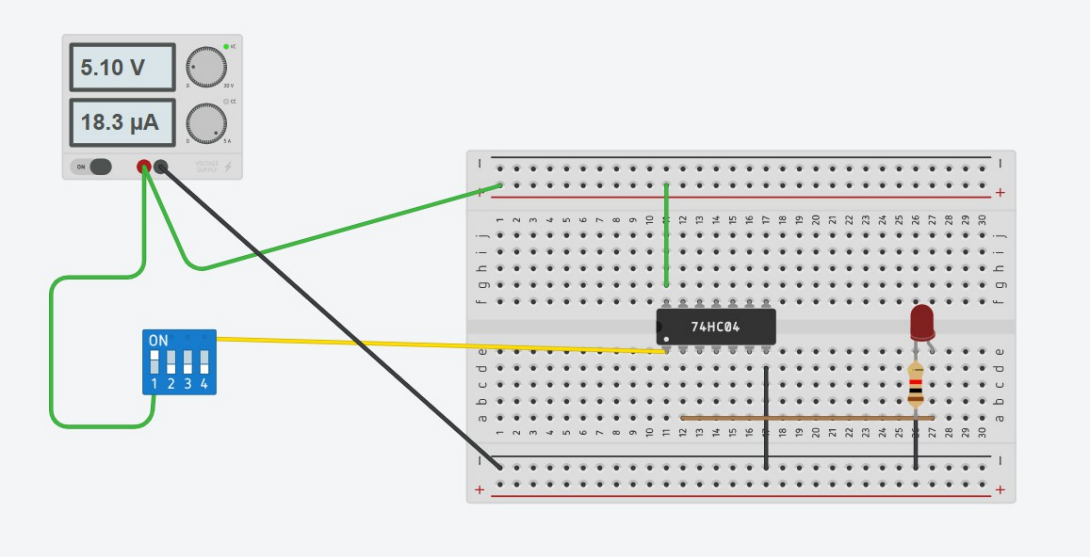

2.30 V
7.61 mA
74HC32
ON
1 2 3 4


2.30 V
7.61 mA
74HC32
ON
1 2 3 4

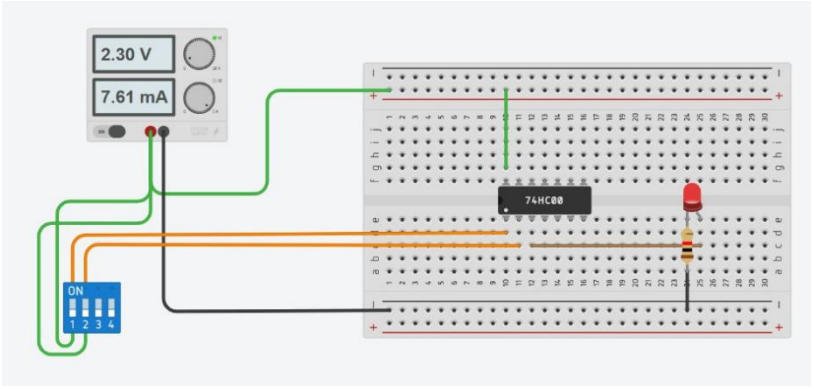

2.30 V
7.61 mA
74HC32
ON
1 2 3 4

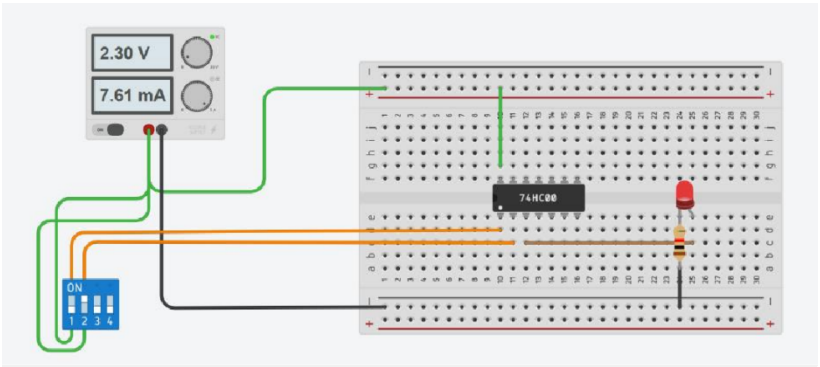
3. Not gate:

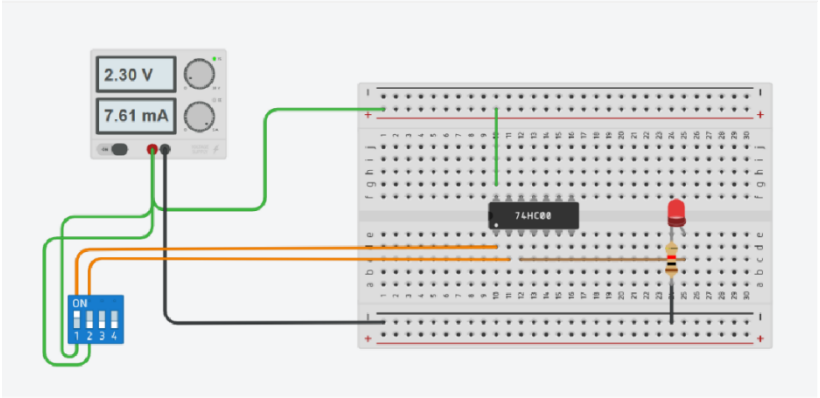


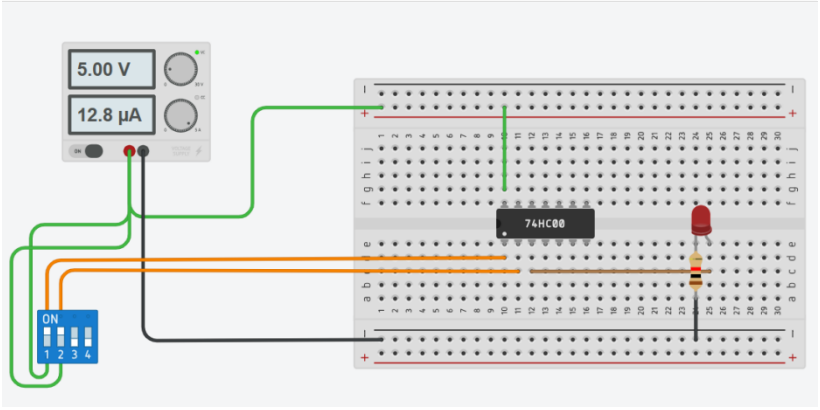


4. Nand gate:

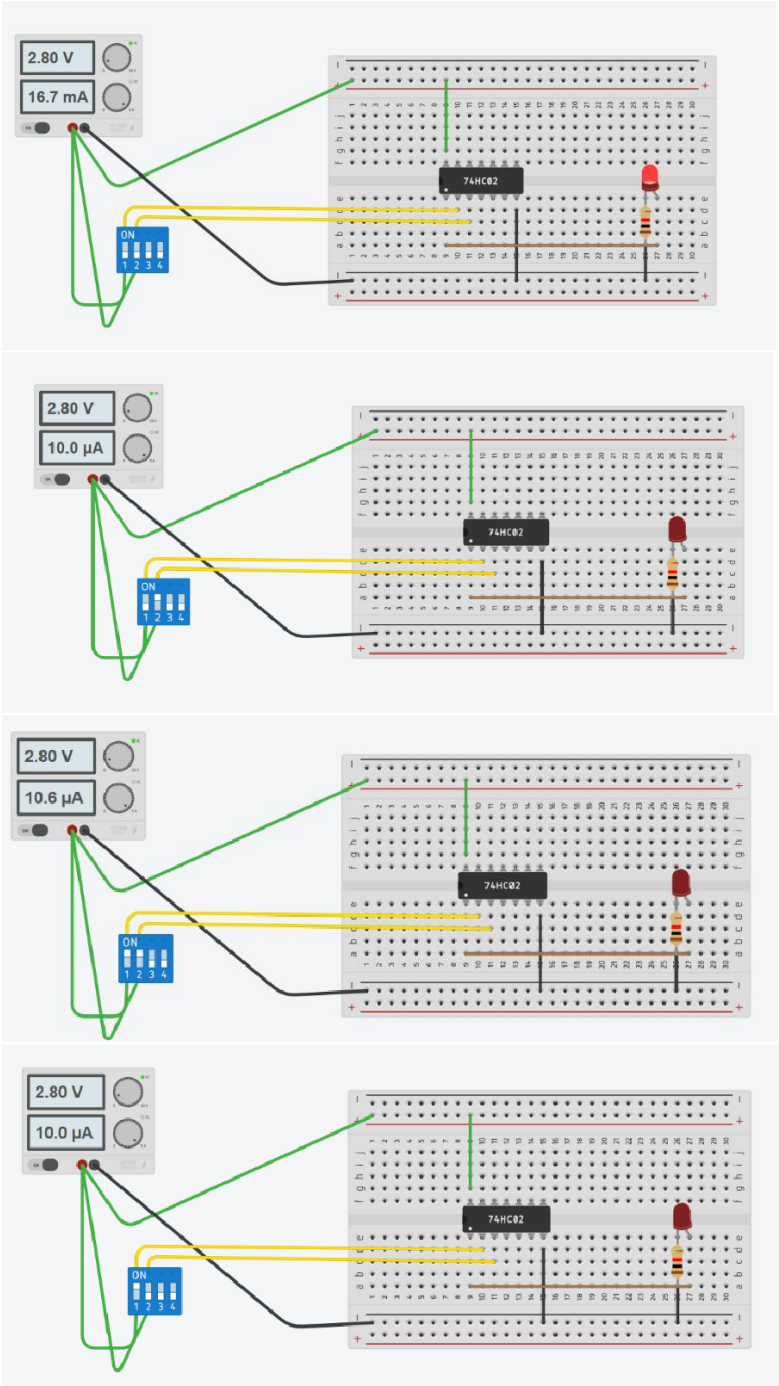

2.30 V
7.61 mA
74HC00
ON
1 2 3 4


2.30 V
7.61 mA
74HC00
ON
1 2 3 4

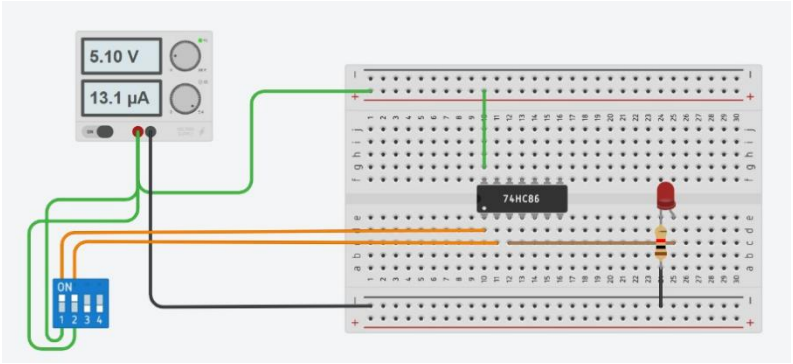

2.30 V
7.61 mA
74HC00
ON
1 2 3 4

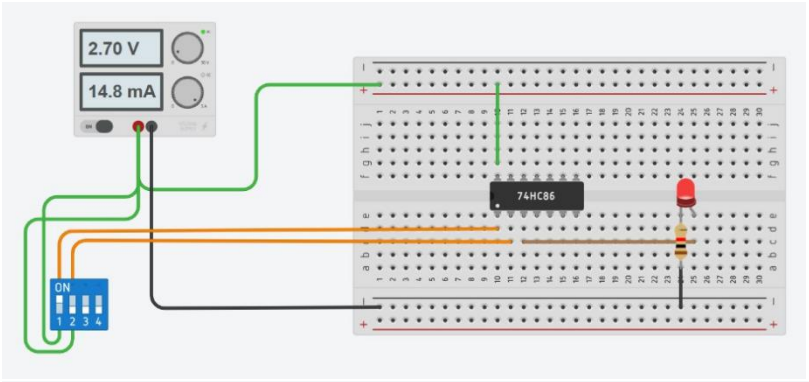

5.00 V
12.8 µA
74HC00
ON
1 2 3 4

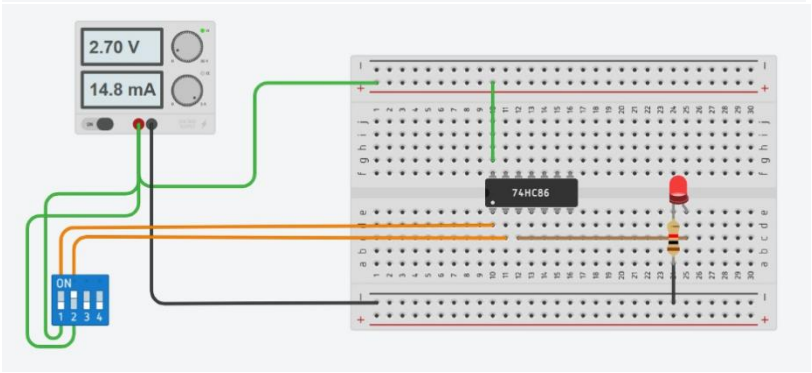
5. Nor gate:

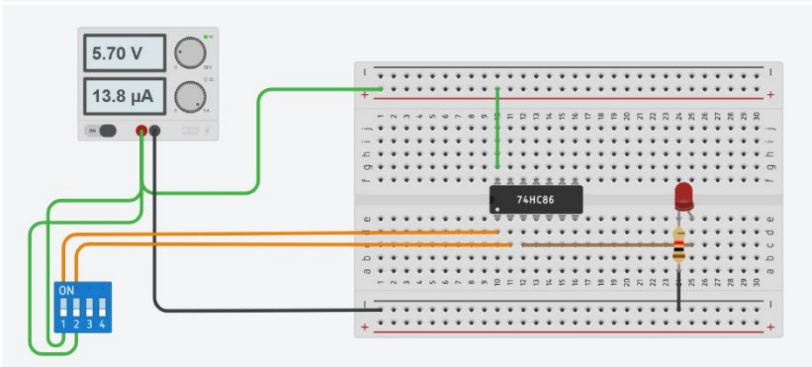

2.80 V
16.7 mA
74HC02
ON
1 2 3 4
2.80 V
10.0 µA
74HC02
ON
1 2 3 4
2.80 V
10.6 µA
74HC02
ON
1 2 3 4
2.80 V
10.0 µA
74HC02
ON
1 2 3 4

6. Ex-Or gate:

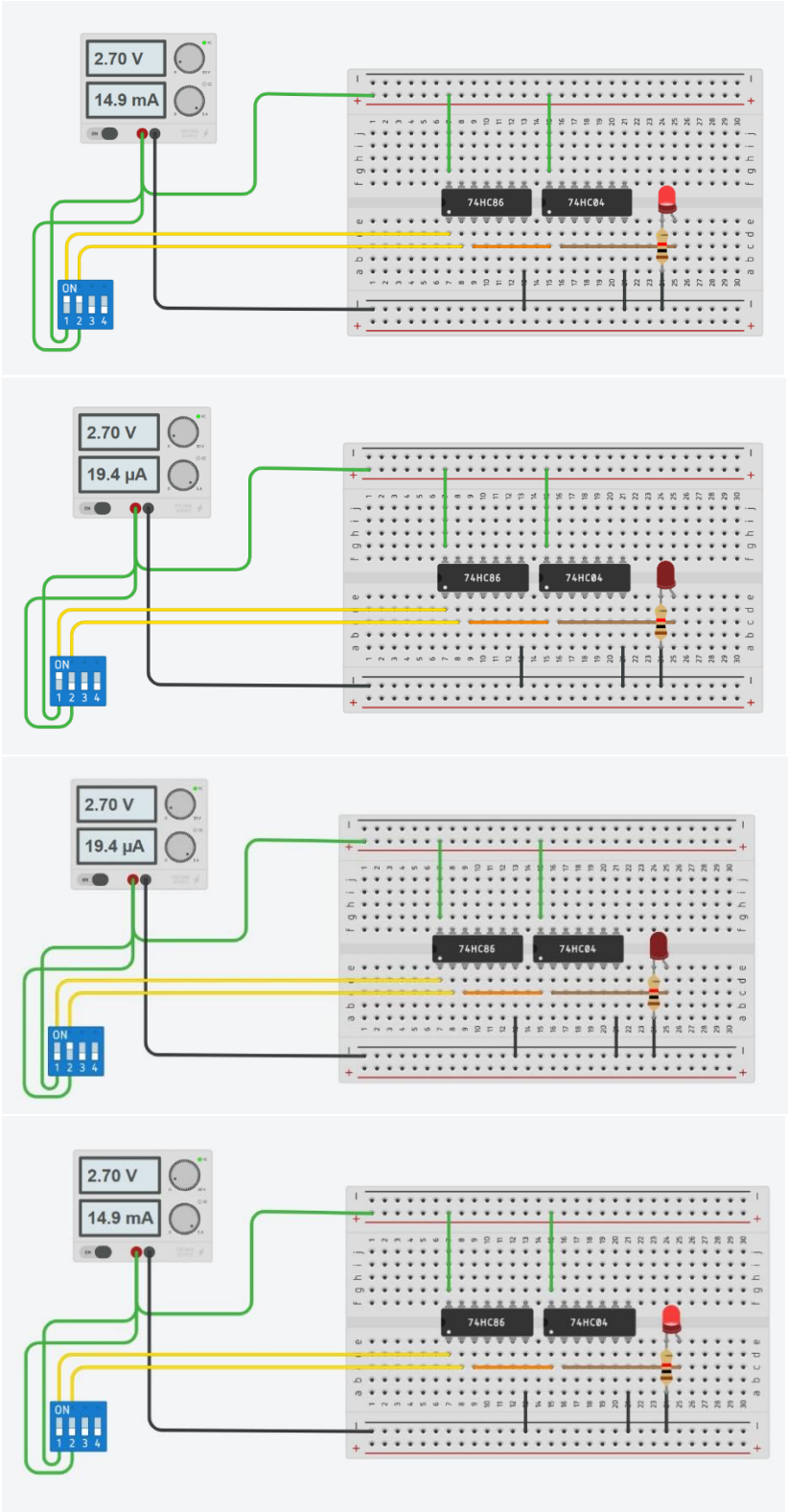

5.10 V
13.1 µA
74HC86
ON
1 2 3 4


2.70 V
14.8 mA
74HC86
ON
1 2 3 4

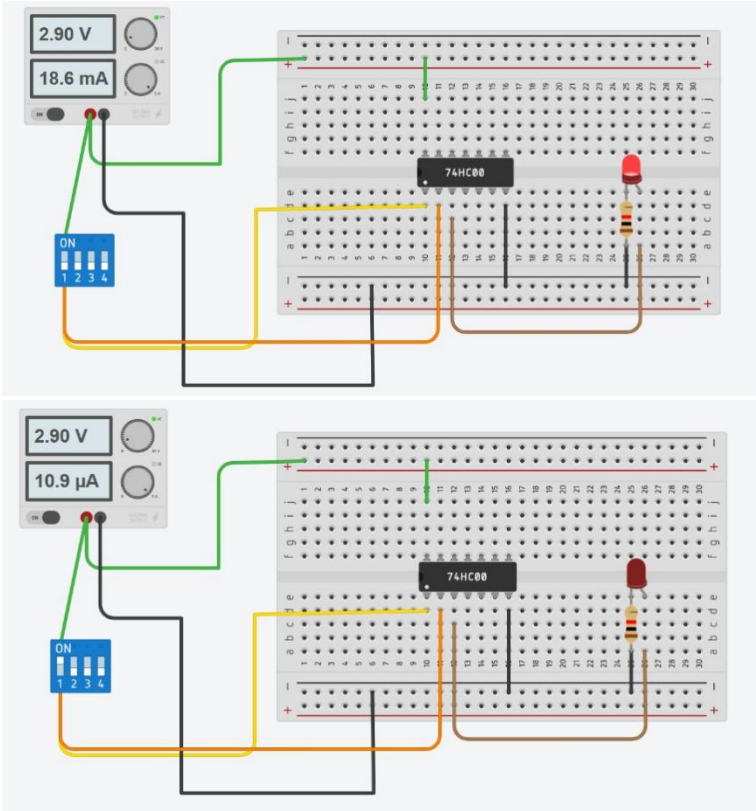

2.70 V
14.8 mA
74HC86
ON
1 2 3 4


5.70 V
13.8 µA
74HC86
ON
1 2 3 4

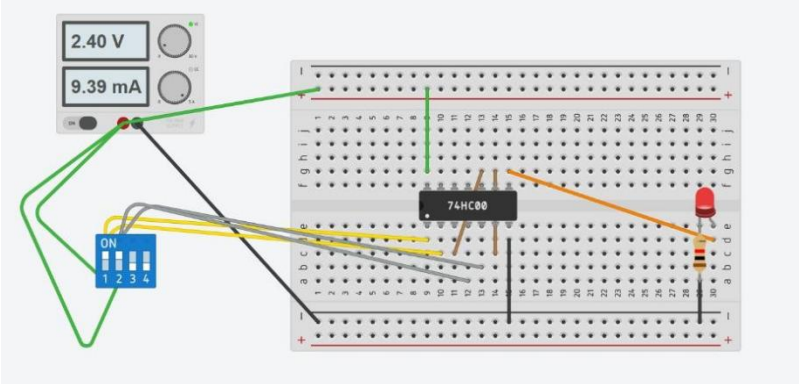
7. Ex-Nor gate:

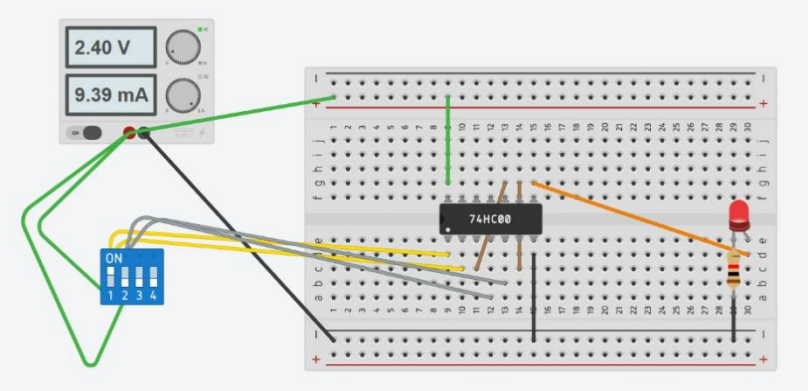

2.70 V
14.9 mA
74HC86
74HC04
ON
1 2 3 4
2.70 V
19.4 µA
74HC86
74HC04
ON
1 2 3 4
2.70 V
19.4 µA
74HC86
74HC04
ON
1 2 3 4
2.70 V
14.9 mA
74HC86
74HC04
ON
1 2 3 4

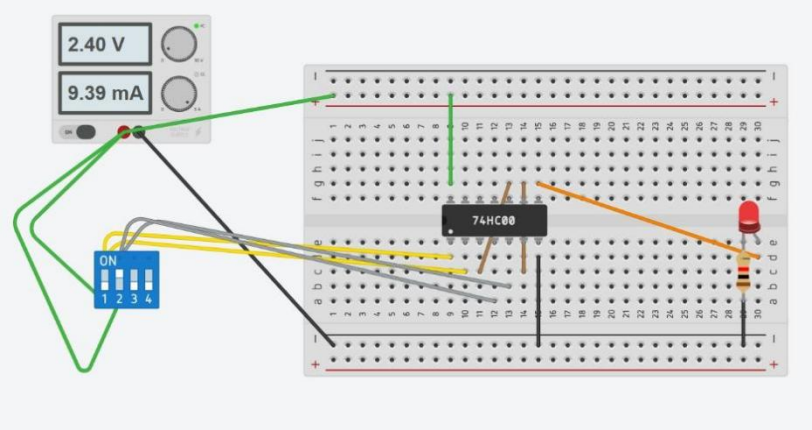
8. Not gate using Nand gate:

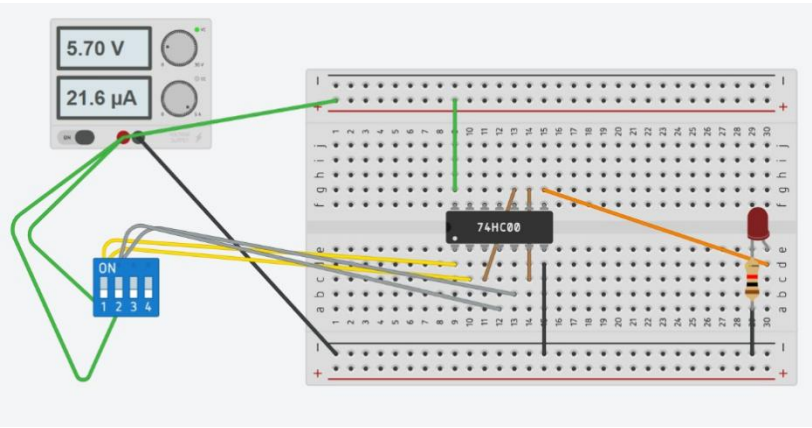


9. Or gate using Nand gate:

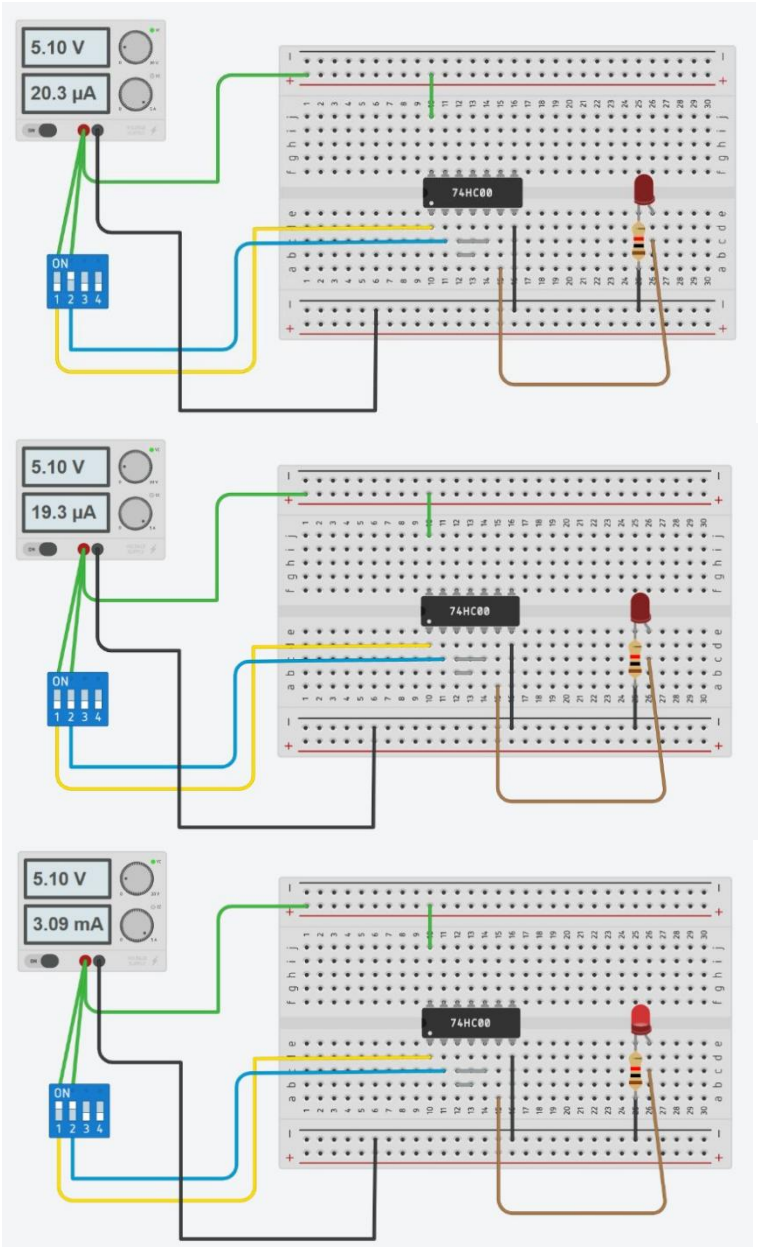


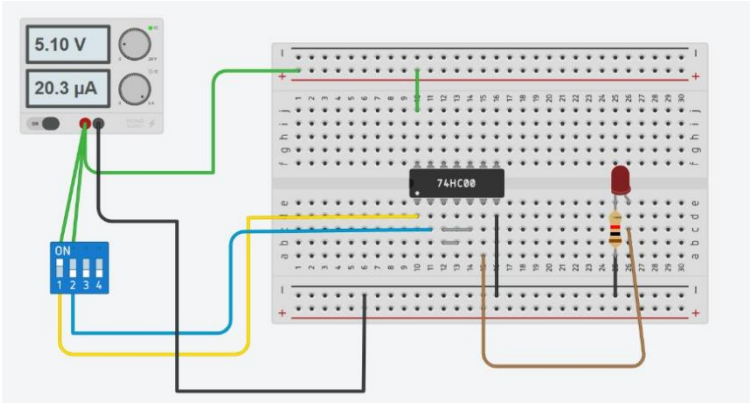

2.40 V
9.39 mA
74HC00
ON
1 2 3 4


2.40 V
9.39 mA
74HC00
ON
1 2 3 4

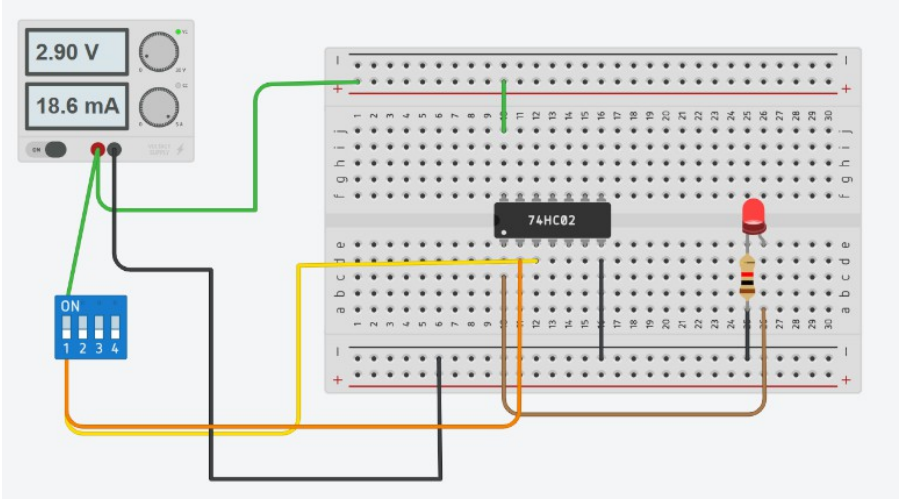

5.70 V
21.6 µA
74HC00
ON
1 2 3 4

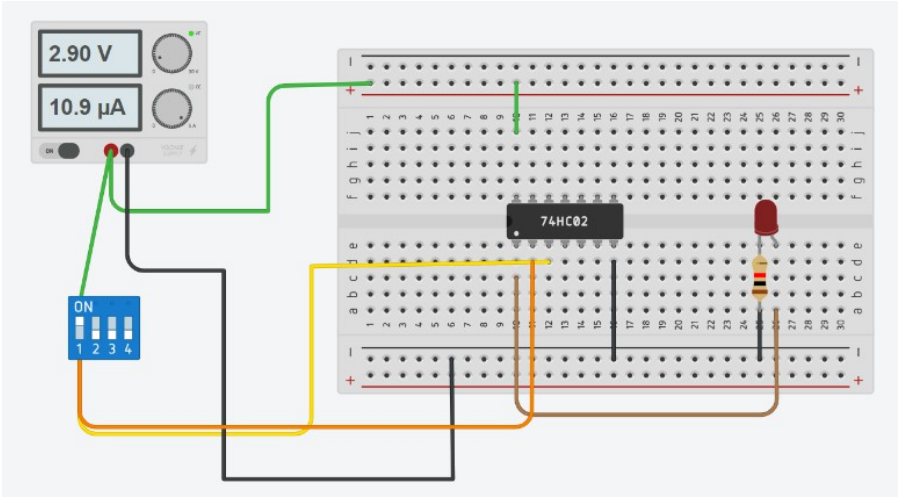
10. And gate using Nand gate:



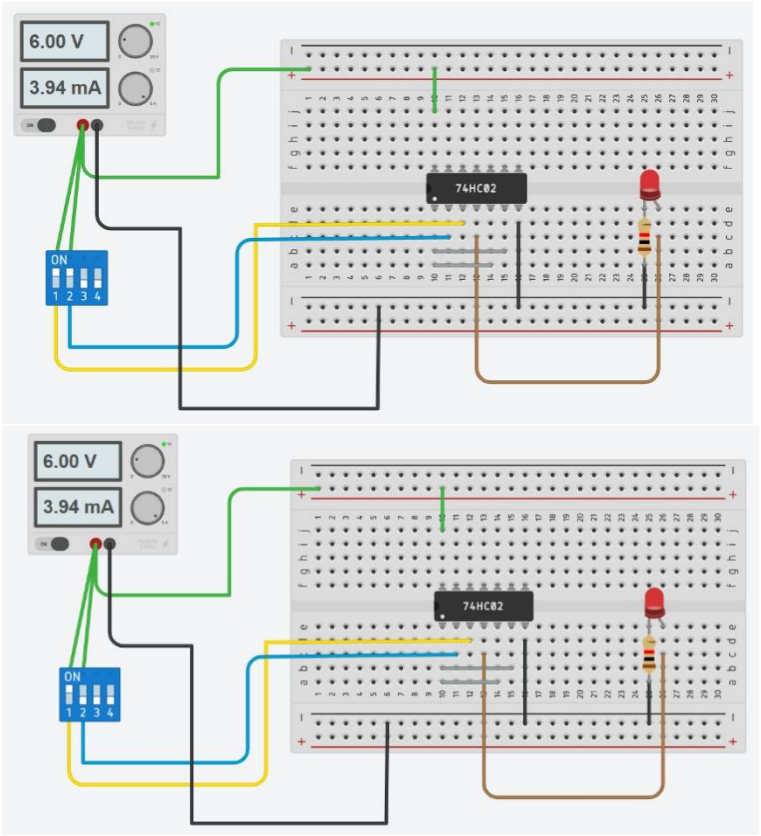


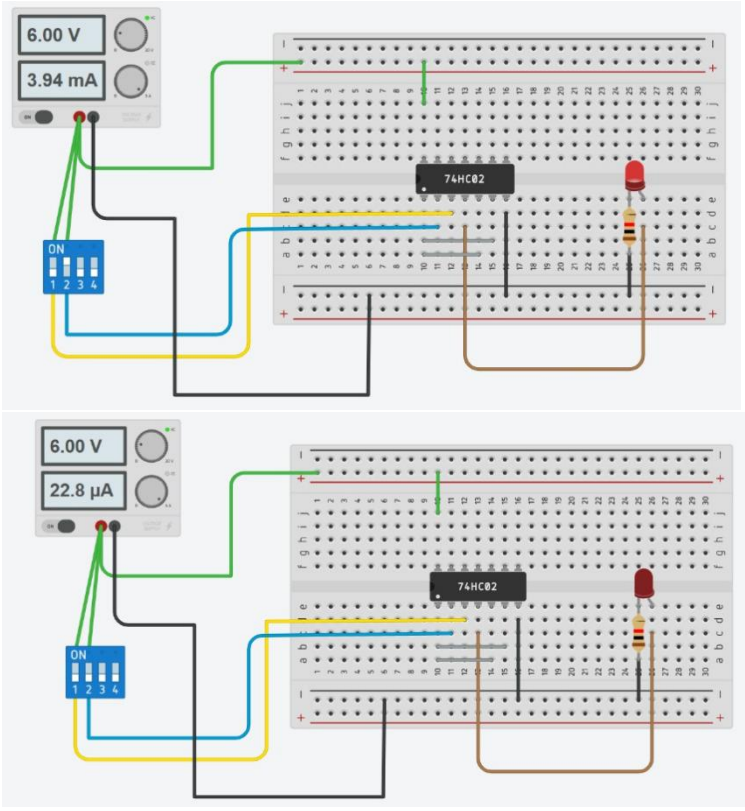
11. Not gate using Nor gate:



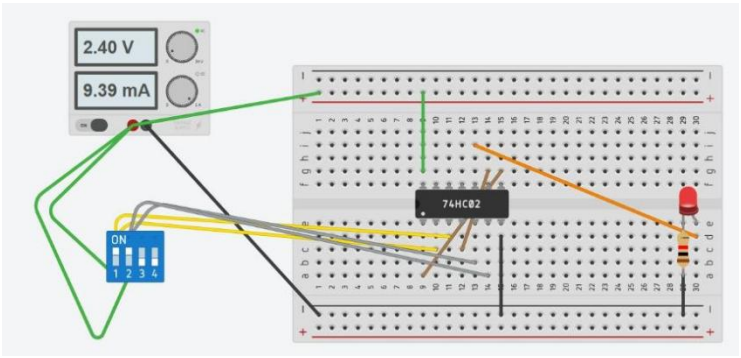


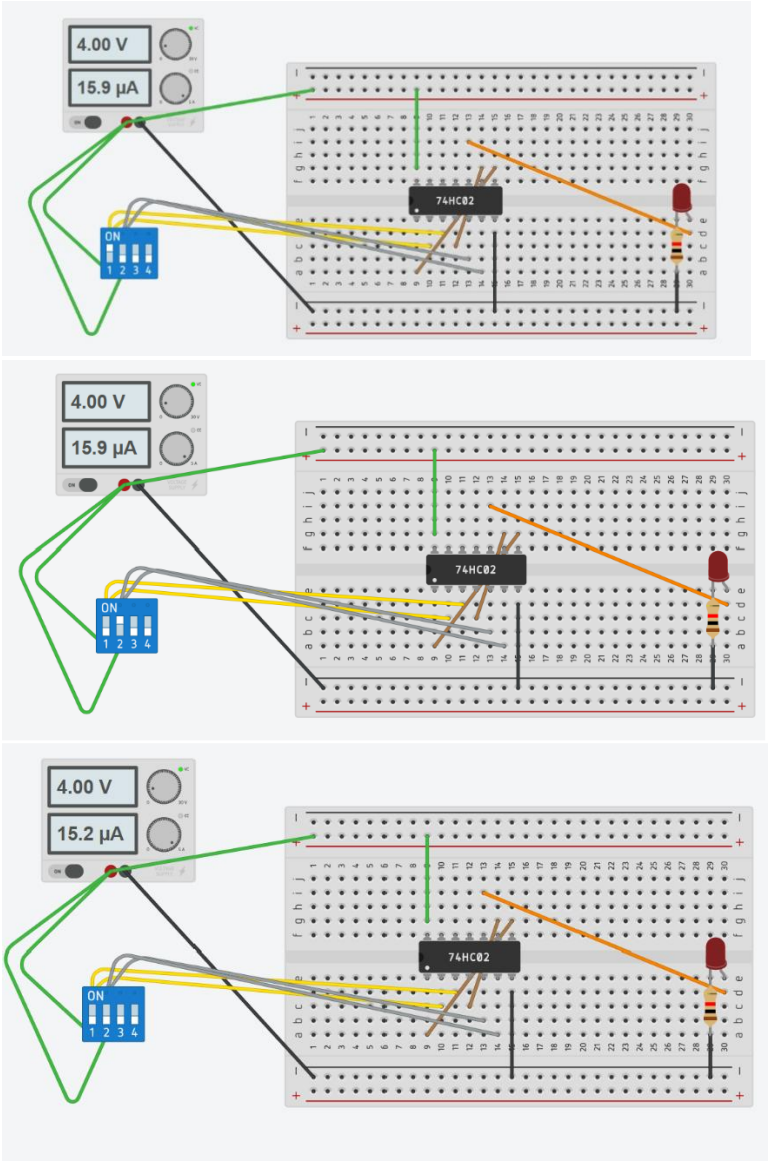
12. Or gate using Nor gate:





13. And gate using Nor gate:





CONCLUSION:

Logic gates are the fundamental building blocks of digital electronic circuits.
They operate on binary inputs (0 and 1) and produce a logical output based on their function.
Different types of logic gates perform different operations such as AND, OR, and NOT.
The AND gate gives output 1 only when all inputs are 1.
The OR gate gives output 1 when at least one input is 1.
The NOT gate inverts the input signal.
NAND and NOR gates are known as universal gates because any logic circuit can be designed using them.

XOR and XNOR gates are used in comparison and arithmetic circuits.
Logic gates are widely used in computers, calculators, and digital communication systems. They help in performing decision-making operations in digital systems.
Understanding logic gates is essential for designing complex digital circuits.
Thus, logic gates play a vital role in modern electronics and technology.

Practical-1

Problem Definition:

Design a 4-input arithmetic-control logic function H(A,B,C,D) = 1 if the 4-bit input (A B C D) matches any of six specific patterns(select by yourself; otherwise H = 0).

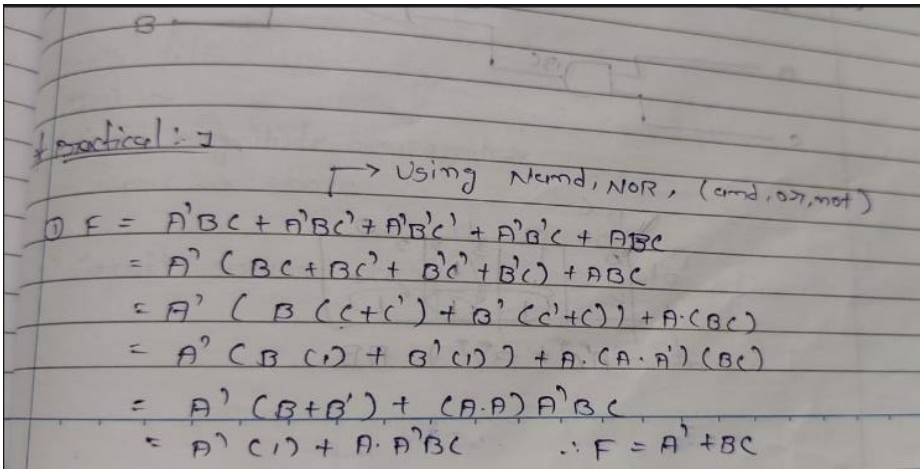
You must:

1. Choose patterns.
2. Derive the Sum-of-Products form (explicit minterms).
3. Attempt simplification with a Karnaugh map and with Boolean algebra; explain differences.
4. Decide which implementation style (NAND-only, NOR- only, or mixed) is most suitable and explain why.
5. Implement the final design in TinkerCad and demonstrate operation with test vectors.
6. Convert the current SOP to a minimal Product-of-Sums (POS) and compare NOR-only implementation cost with NAND-only.

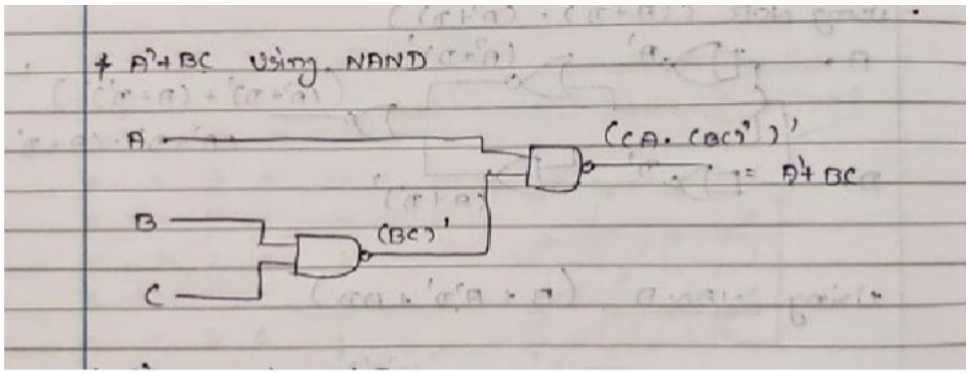
Tools/Technology To Be Used:

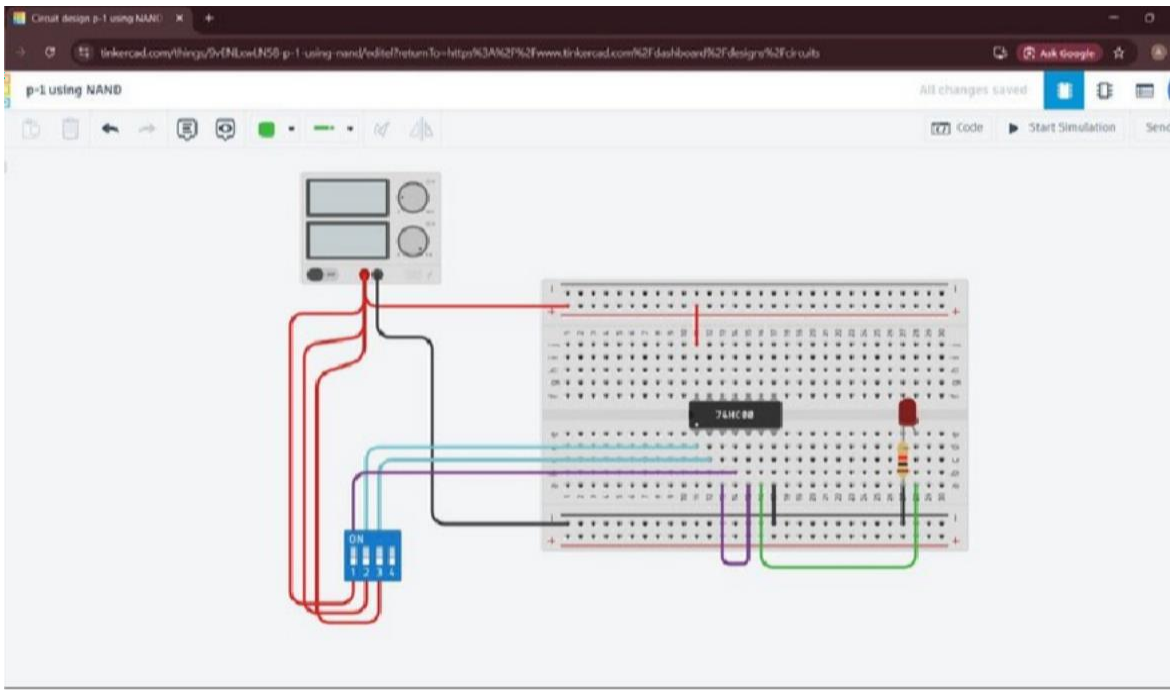
- Logic design: pen and paper
- Gate simulation & wiring: TinkerCad Circuits
- Optional: Multi-input gate ICs or build using cascaded 2-input gates.

1) simplification with Boolean algebra:

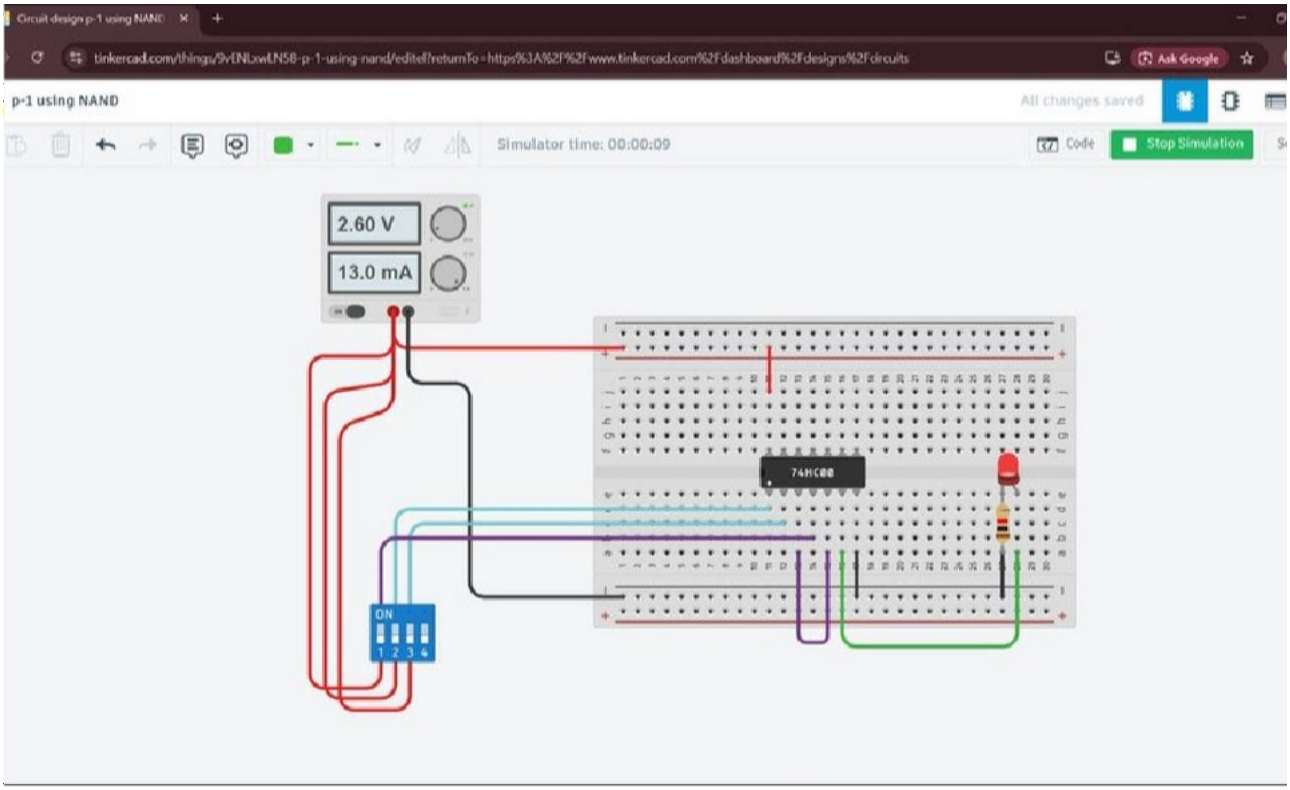


1) Using Nand gate:

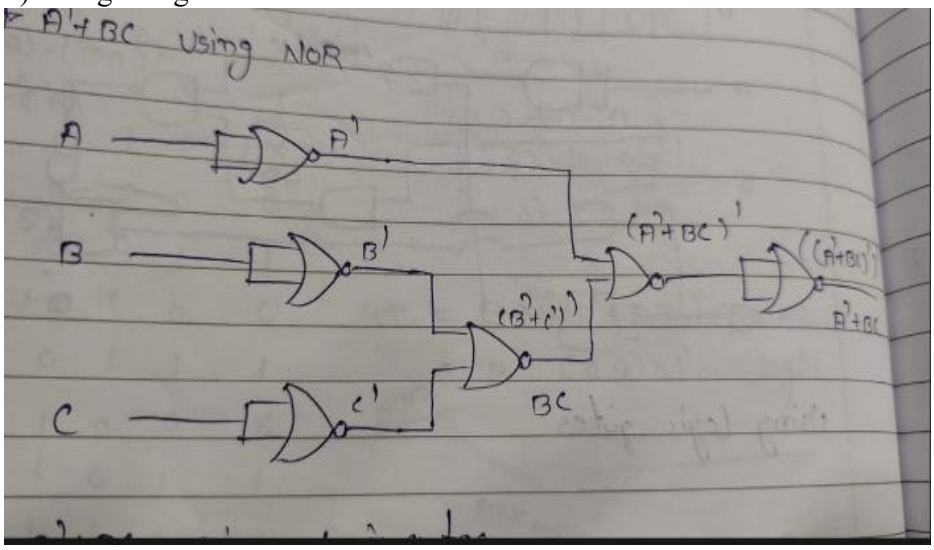


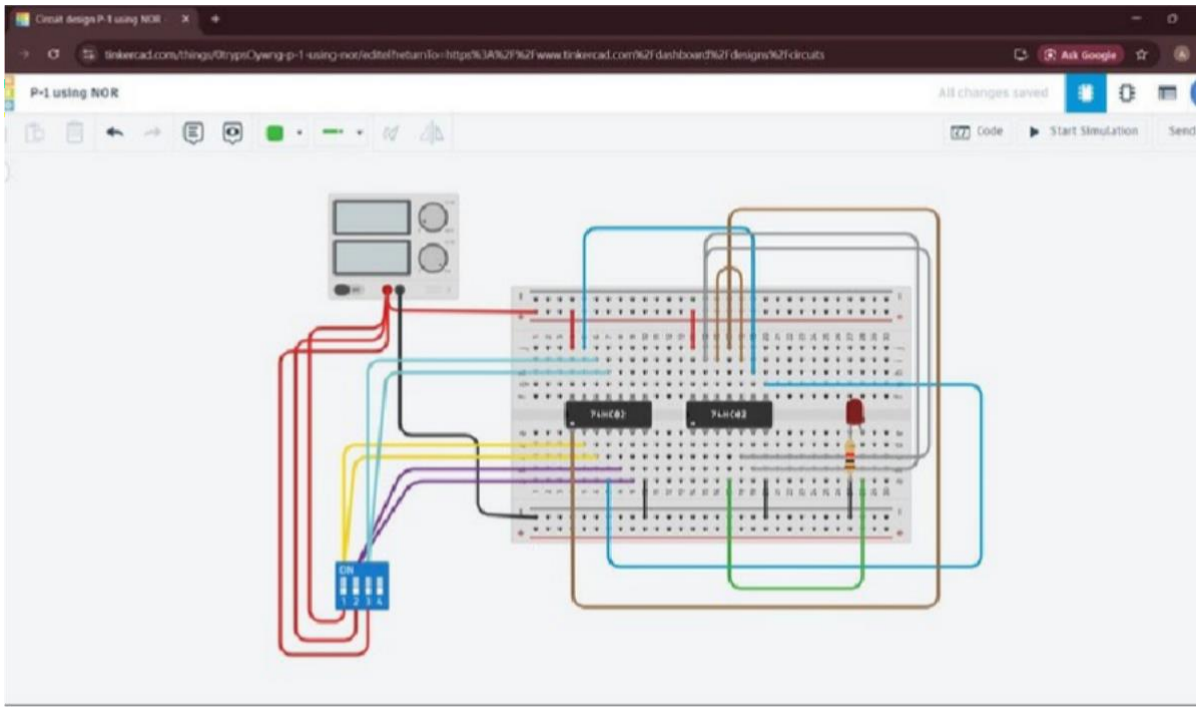


Output:

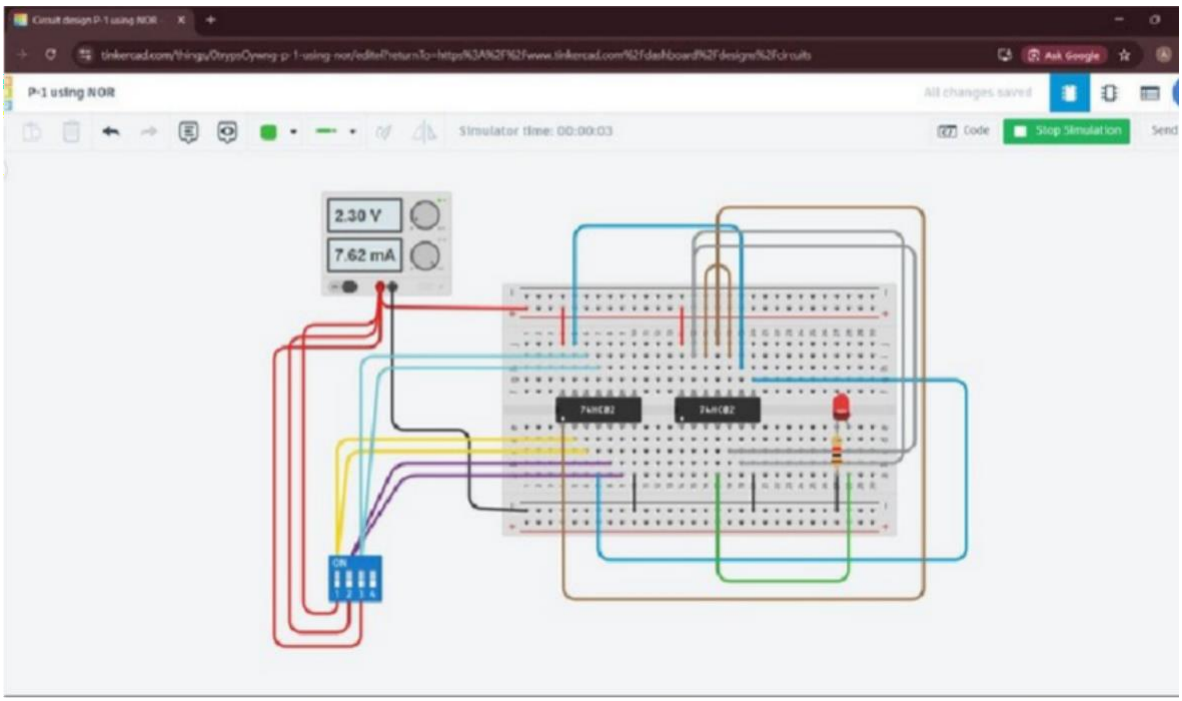


2) Using Nor gate:

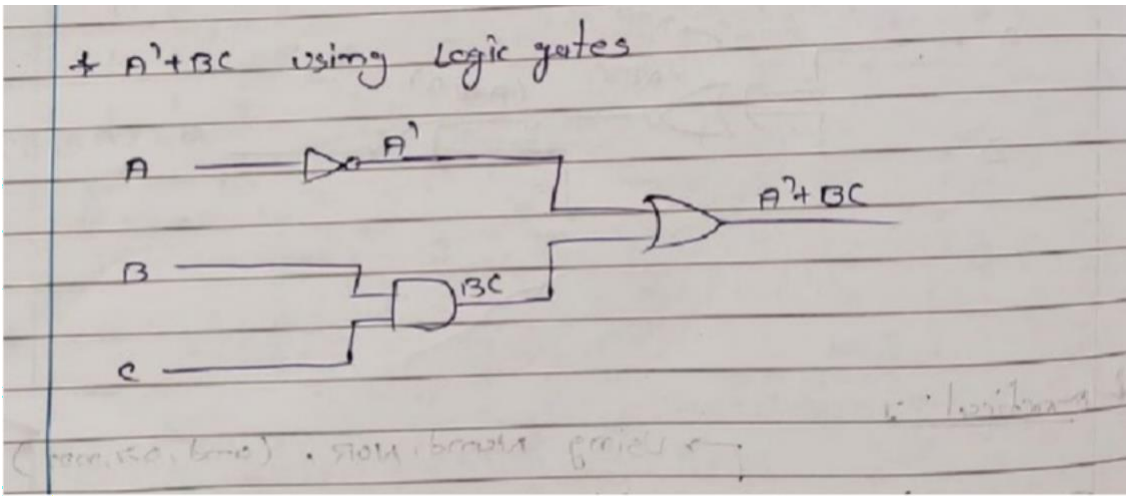


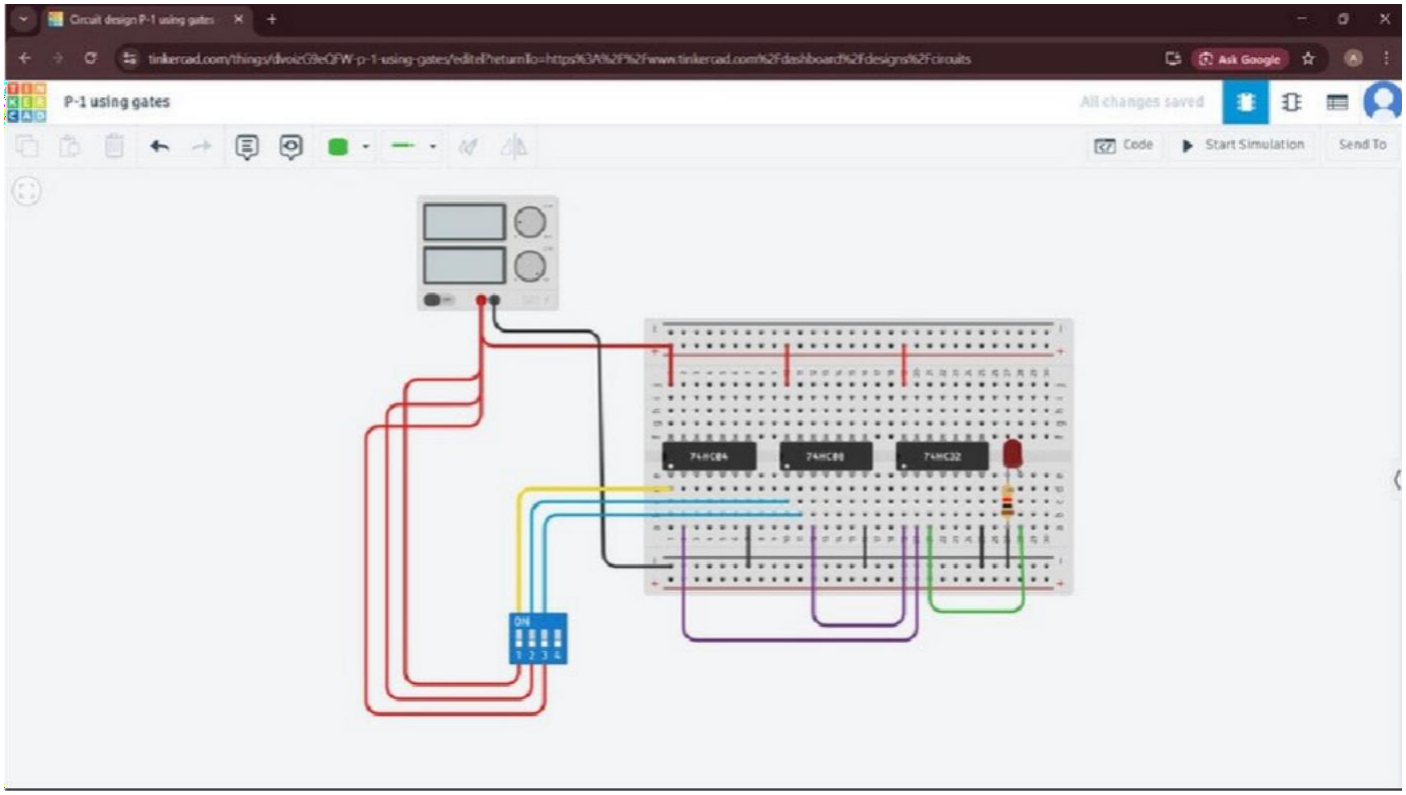

P=1 using NOR
All changes saved
Code
Start Simulation
Send

Output:

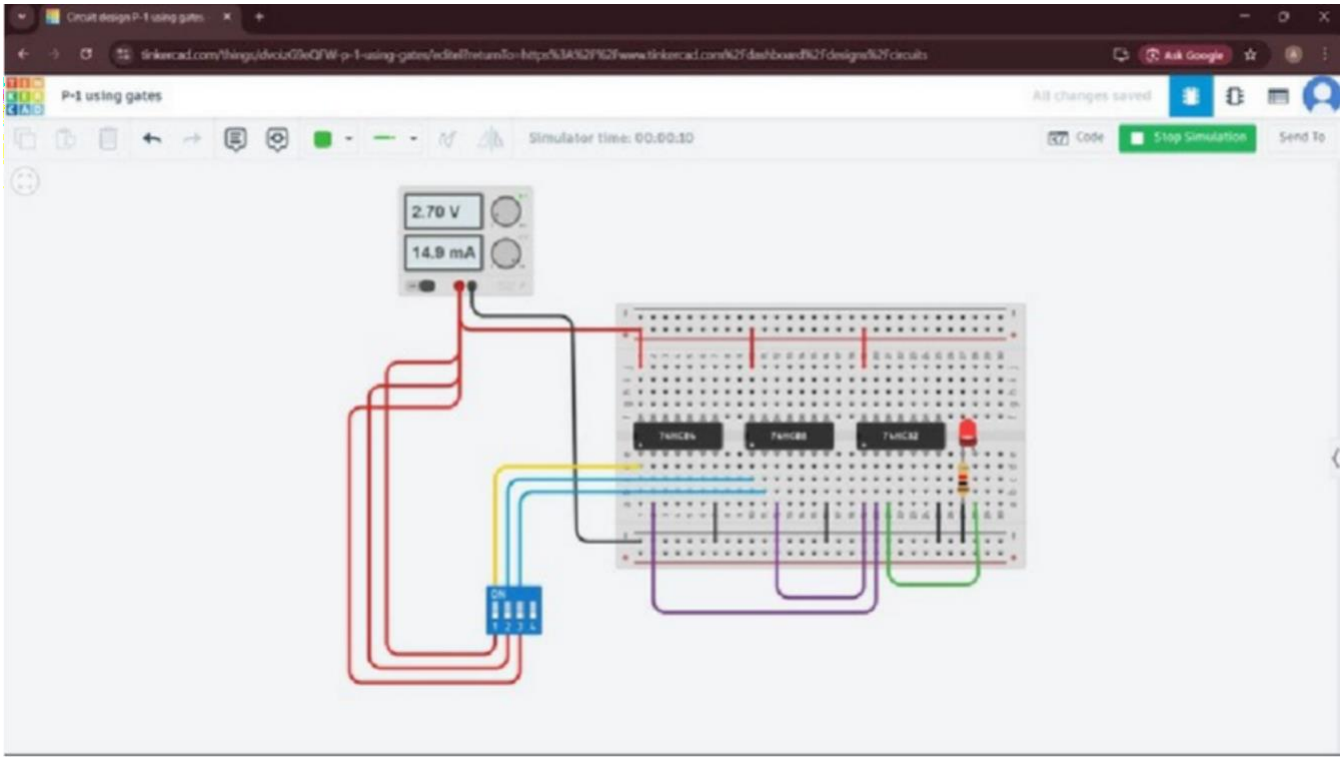

P=1 using NOR
All changes saved
Simulator time: 00:00:03
Code
Stop Simulation
Send
2.30 V
7.62 mA

3) Using And ,Or, Not gate:

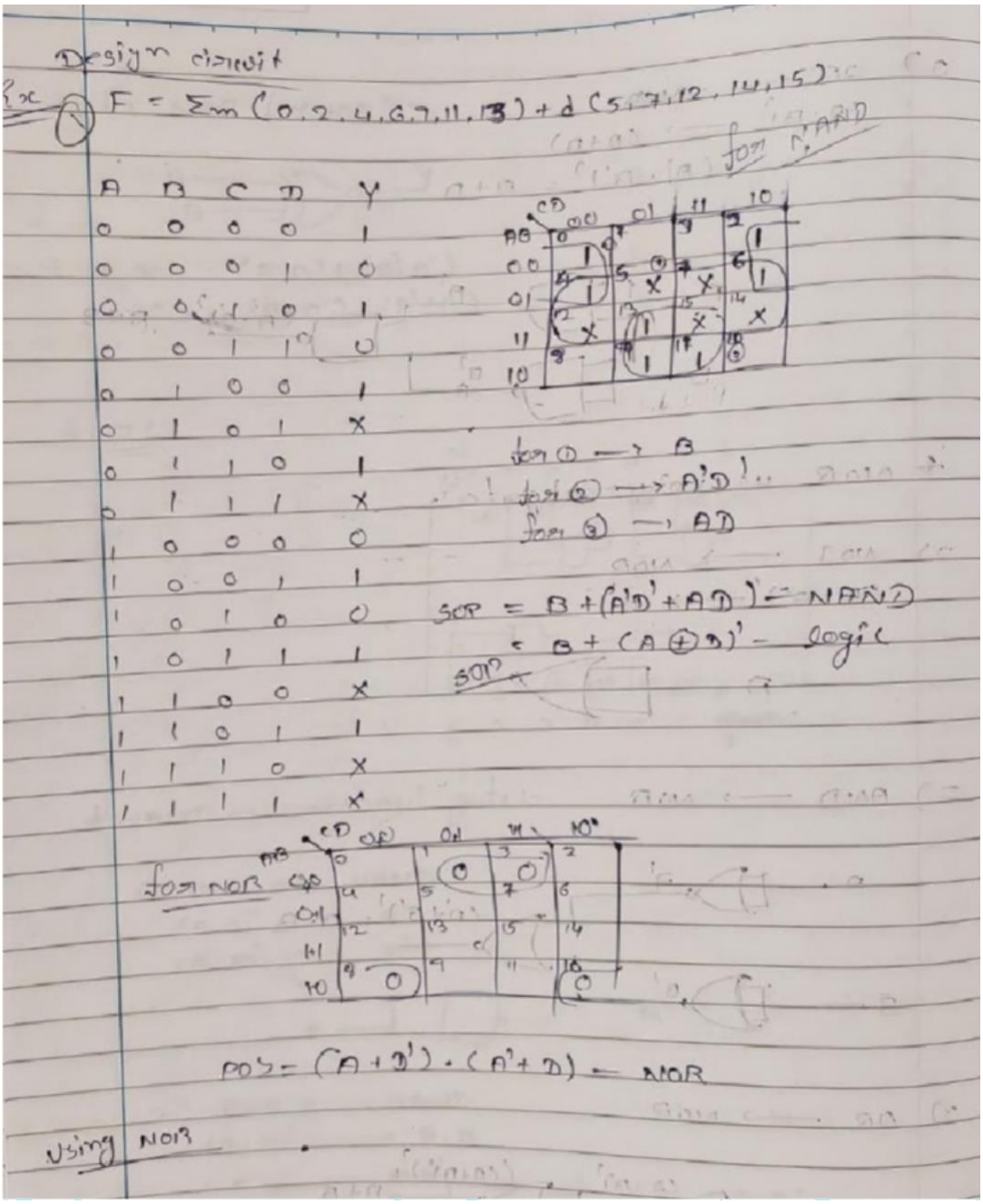




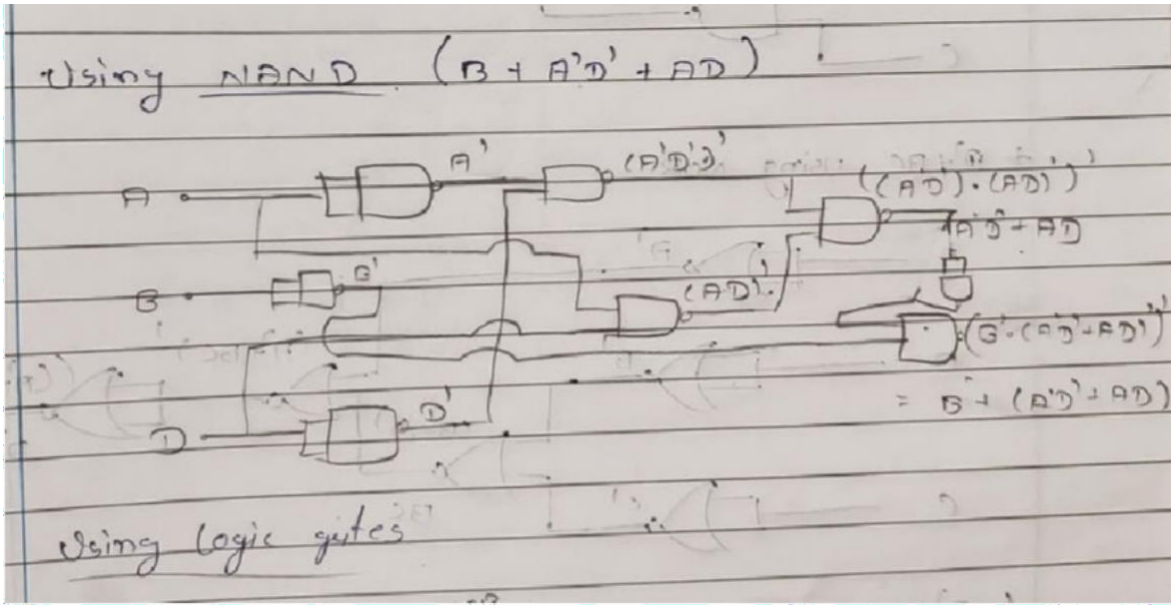
Output:


P-1 using gates
Simulator time: 00:00:10
Code
Stop Simulation
Send To
2.70 V
14.9 mA

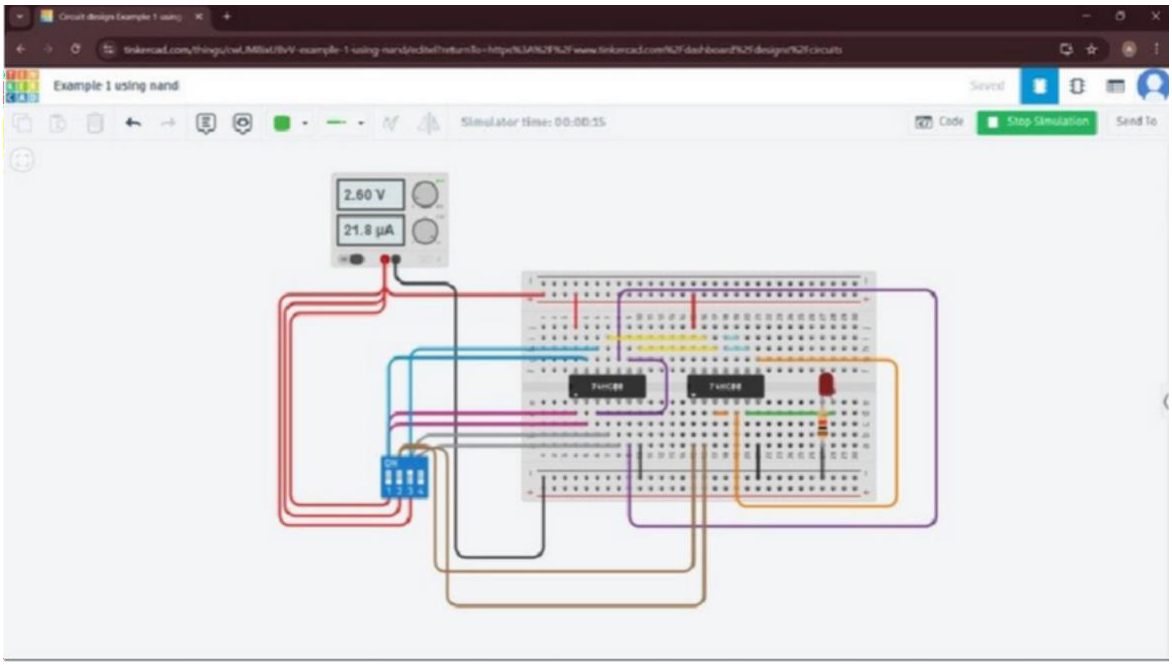
2) simplification with a Karnaugh map:

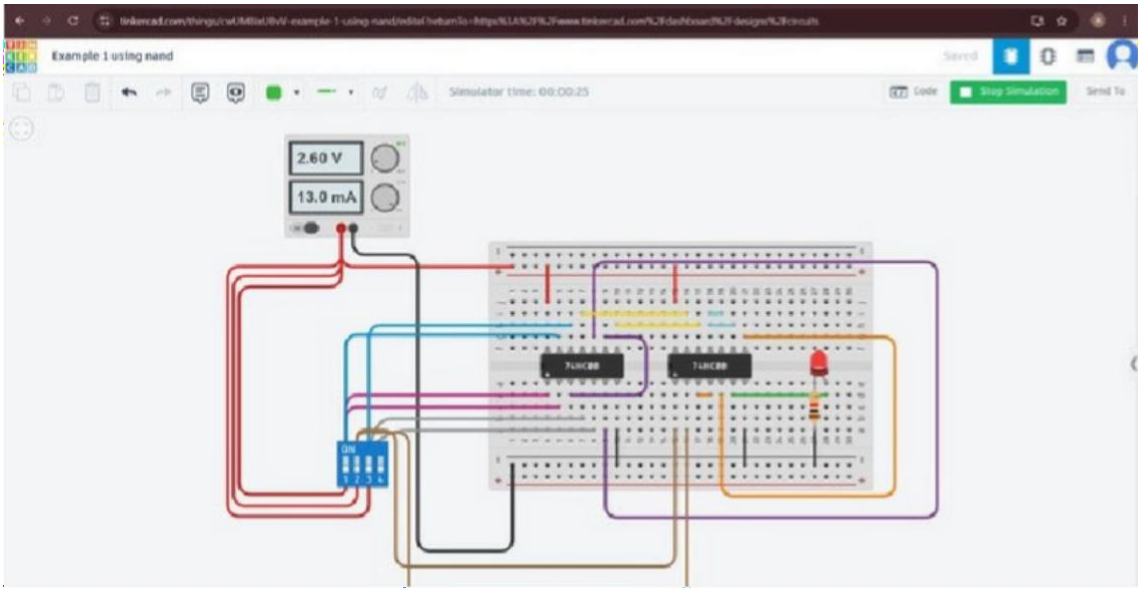


1) Using Nand gate:

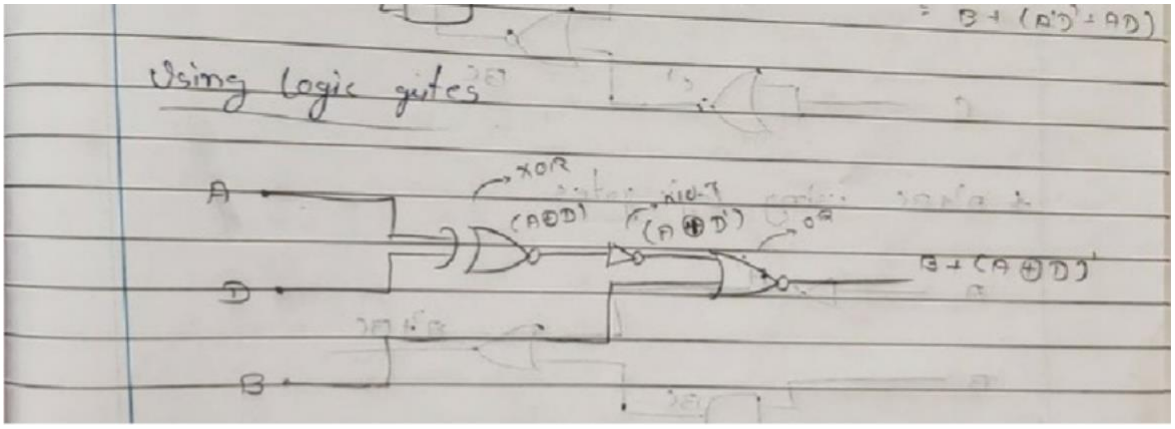


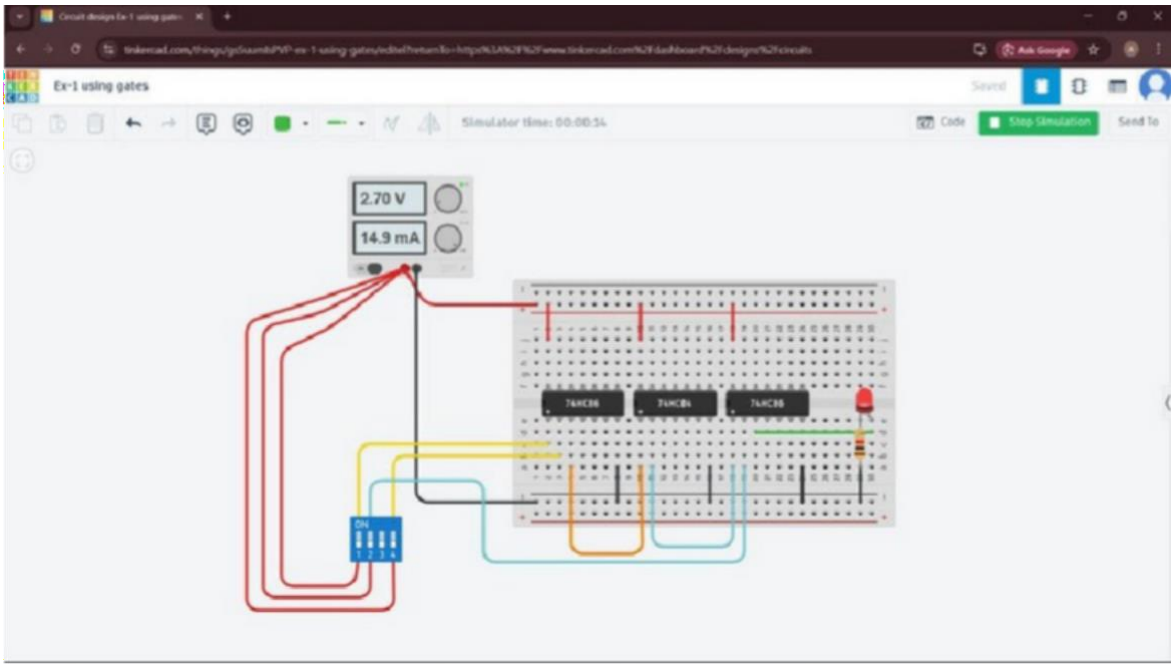
Output:





2) Using Logic gates:




Ex-1 using gates
2.70 V
14.9 mA

Conclusion: This practical successfully demonstrates the process of transforming a specific set of input patterns into an optimized hardware logic function using Karnaugh Map (K-map) and Boolean algebraic techniques. By comparing SOP (Sum-of-Products) and POS (Product-of-Sums) forms, the design reveals how the choice of implementation (NAND-only vs. NOR-only) directly impacts the total gate count and circuit complexity. The transition from theoretical minterms to a physical TinkerCad simulation confirms that Boolean minimization preserves the functional integrity of the original truth table. Ultimately, the project validates that strategic logic reduction not only simplifies the schematic but also reduces the physical hardware cost required to detect specific 4-bit arithmetic control sequences.

Practical-2

Problem Definition :

Design and implement a 4-bit digital calculator that performs basic arithmetic operations on two 4-bit operands A (A3..A0) and B CO2, CO3/PO1 (B3..B0). The calculator should support the following operations selected by a 2-bit opcode Op1 Op0:

● 00 — Addition: S = A + B (4-bit result + carry out)

● 01 — Subtraction: S = A − B using two's complement (detect overflow/borrow)
● 10 — Multiplication: P = A × B (produce an 8-bit product)
● 11 — Optional / Extended: pass-through / increment A / unsigned divide (choose one for extension)

1. Implement at gate-level (use standard ICs such as full adders, XOR, AND/OR/MUX) and test in TinkerCad Circuits (or equivalent simulator).
2. Provide truth table(s), block diagram, logic derivations, test vectors and final circuit schematic.

Tools/Technology To Be Used :

● TinkerCad Circuits — build gate-level design using logic primitives (AND, OR, XOR, NOT) and ICs (74xx family) or breadboard components in the simulator.

● Optionally use Logisim (or similar) for easier logic visualisation.

● Use a multimeter/logic probe or LEDs to show outputs or 7 segment display modules for human-readable results.

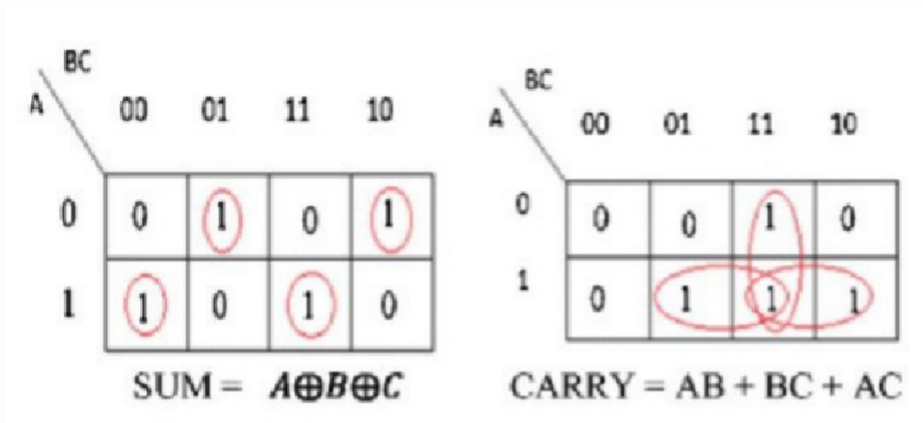
1) Full Adder:

Description: A Full Adder is a combinational logic circuit that performs the arithmetic addition of three binary bits. Unlike a half adder, it can handle a carry bit from a previous stage, making it the fundamental building block for multi-bit binary addition.

Truth Table:

A	B	C_{in}	Sum (S)	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-Map:



Equation:

1. Sum Equation

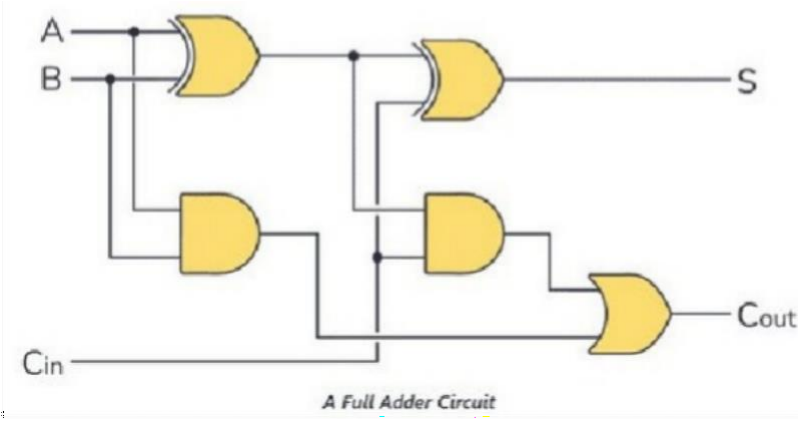
The Sum output is obtained using XOR operation:

$S=A \oplus B \oplus Cin$

2. Carry Equation

The Carry output is $Cout = AB + BCin + ACin$

Circuit:



A Full Adder Circuit

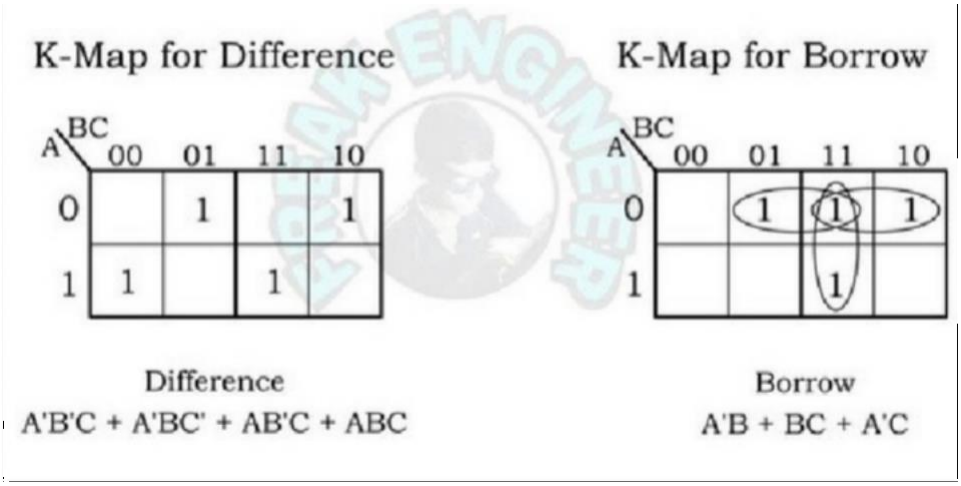
2) Full Subtractor:

Description: A Full Subtractor is a combinational logic circuit that performs the subtraction of three binary bits: the minuend, the subtrahend, and a borrow-in from a previous lower-order bit position. Unlike a Half Subtractor, which can only handle two bits, a full subtractor is essential for multi-bit subtraction as it accounts for the borrow generated during the subtraction of less significant bits.

Truth table:

A	B	Bin	Difference (D)	Borrow-out (Bout)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-Map:

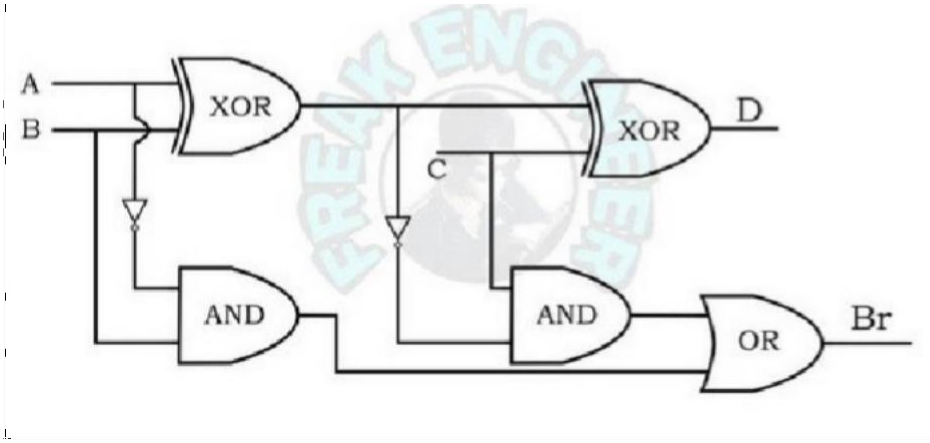


Equation:

Difference : $D=A\oplus B\oplus Bin$

Borrow: $Bo=u't\ AB+ABin+'BBin$

Circuit:



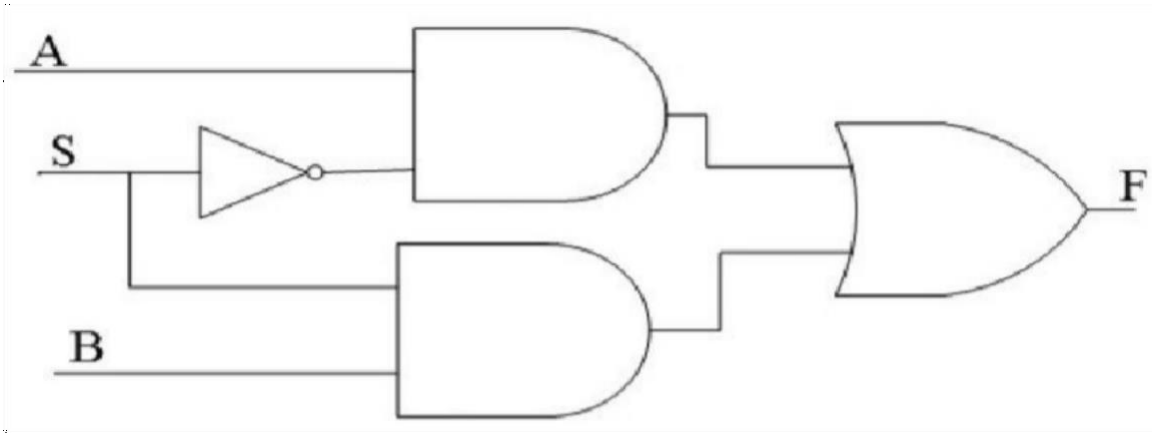
3) Multiplexer:

Description:A multiplexer (commonly abbreviated as MUX) is a combinational logic circuit that selects one of several digital or analog input signals and forwards the selected input into a single output line. Often called a data selector, it acts like a multi-position electronic switch that determines which input reaches the output based on a set of control inputs known as select lines.

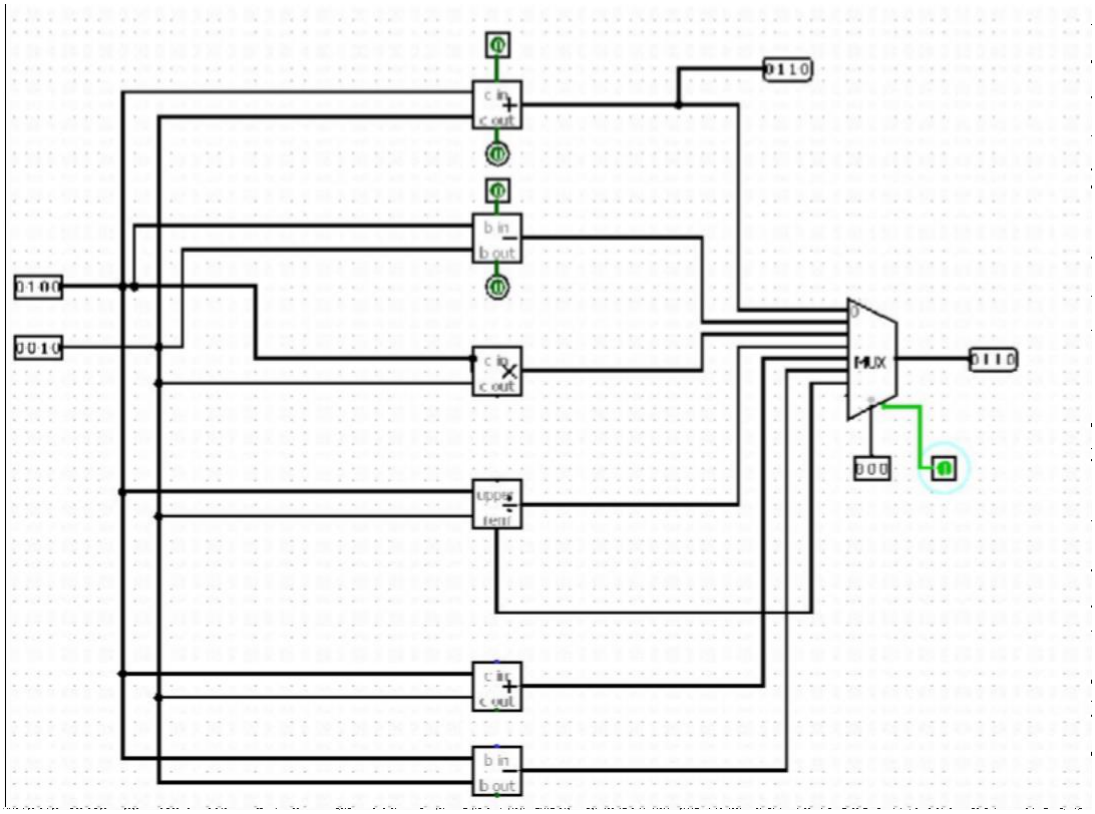
Equation:

$Y=I0S'+I1S$

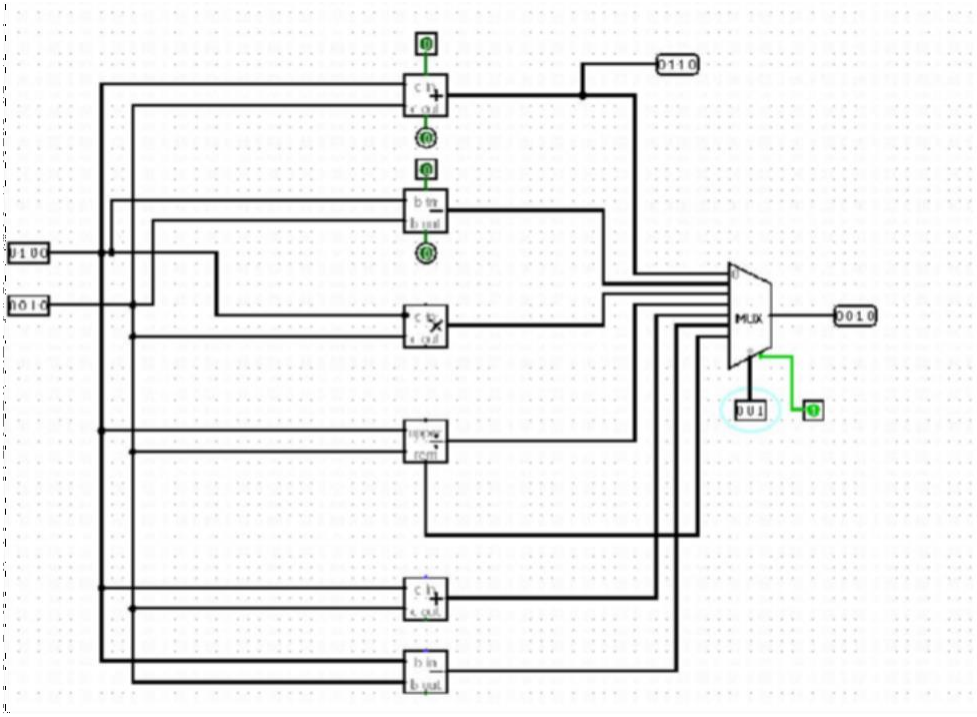
Circuit:



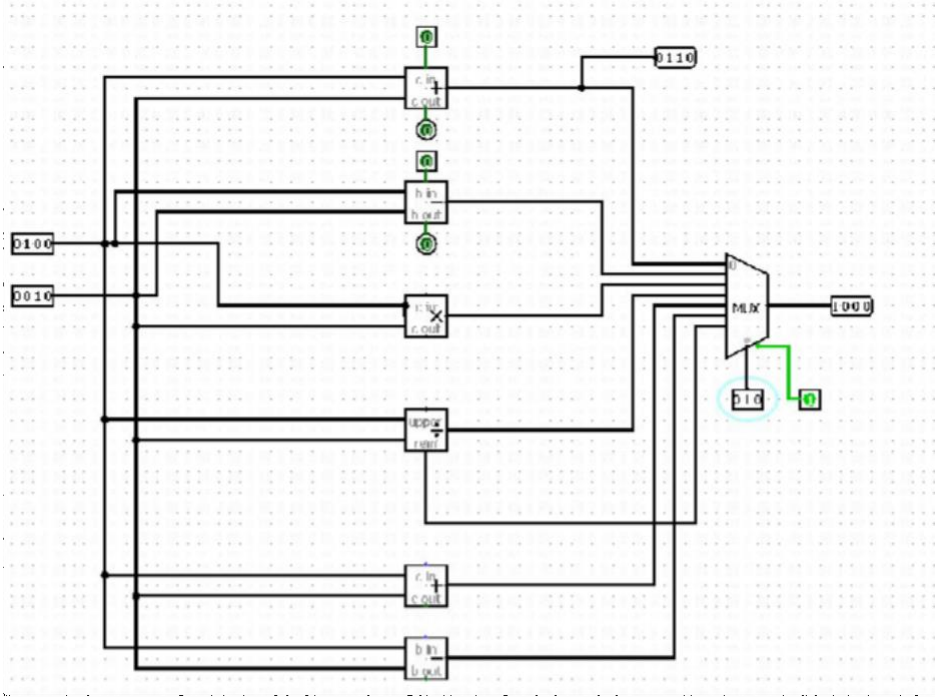
Addition:



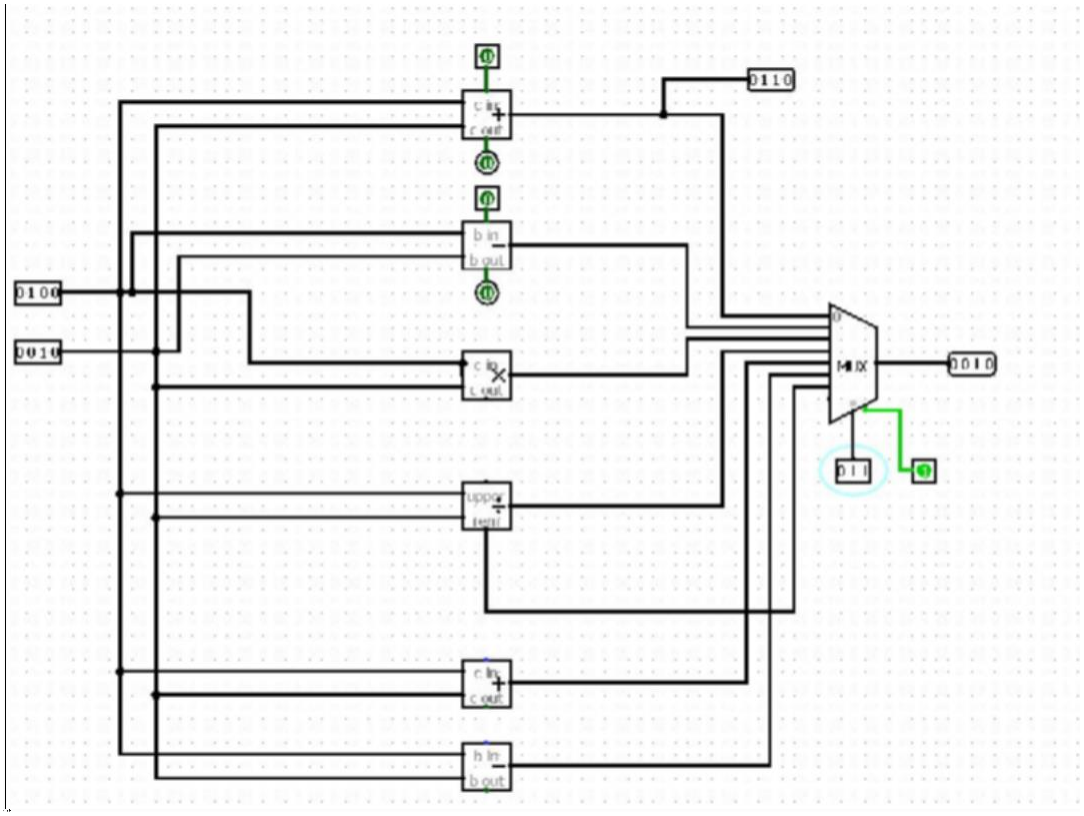
Subtraction:



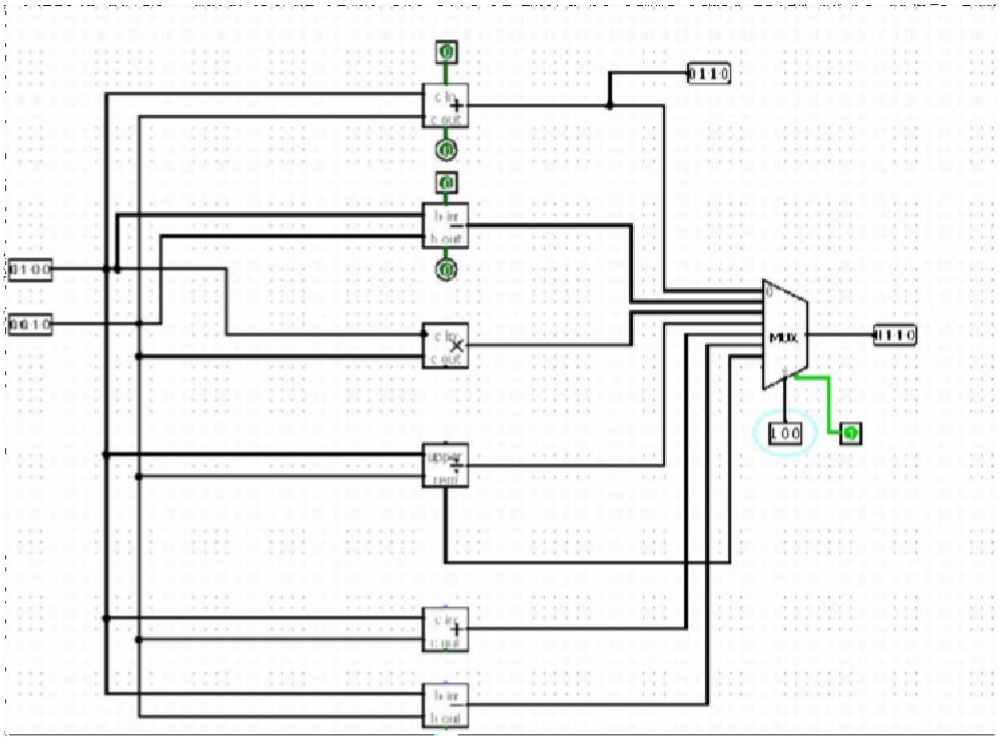
Multiply:



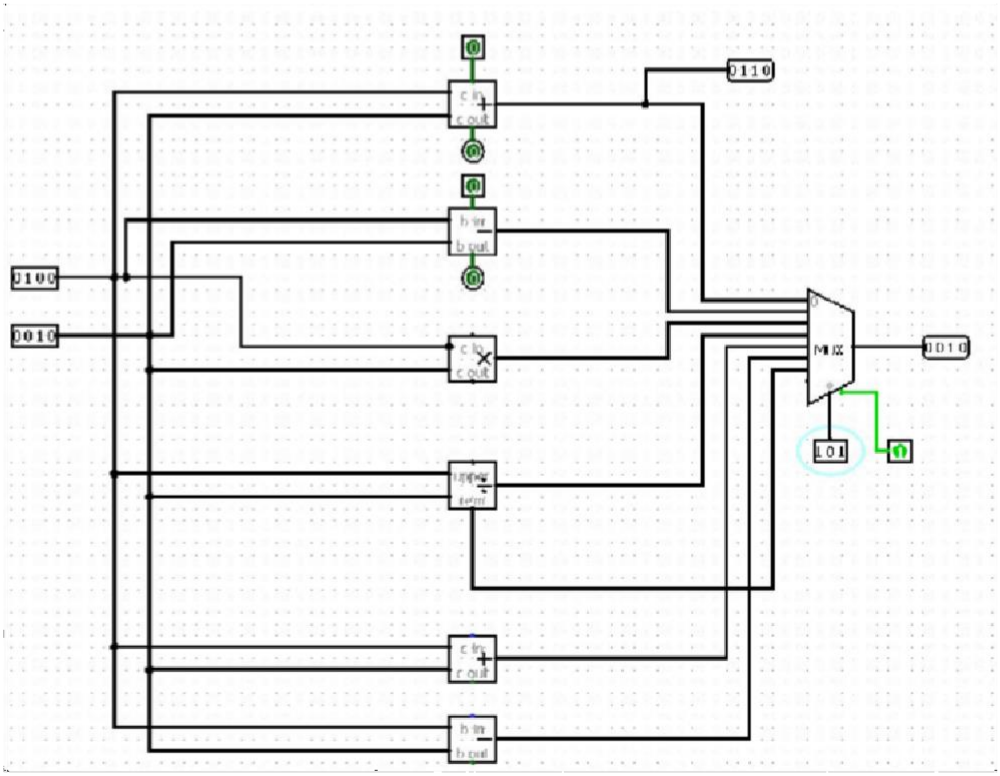
Division:



Increment:



Decrement:



Conclusion: This practical session on the 4-bit digital calculator illustrates the core principles of ALU (Arithmetic Logic Unit) design. By combining standard components like Full Adders, XOR gates for subtraction logic, and AND-gate arrays for multiplication, the circuit successfully processes multi-bit data based on specific control signals. The simulation confirmed that 2-bit opcodes can effectively multiplex diverse functions—ranging from basic addition to 8-bit multiplication—into a single output bus. Ultimately, the project validates the efficiency of binary arithmetic and the scalability of gate-level logic in building functional computational hardware.

Practical-3

Problem Definition:

Adigital thermostatreceives two 4-bit temperature readings:

- T_room (temperature inside the room)
- T_safe (maximum safe temperature threshold)

Design a digital system that:
1. Compares T_room and T_safe.
2. Activates an overheat alarm (A = 1) if T_room > T_safe.
3. Activates a ventilation system (V = 1) if T_room ≥ T_safe – 2 degrees.
4. Triggers a system shutdown (S = 1) if T_room ≥ T_safe + 3 degrees.
5. Ensures that if multiple outputs activate, priority is: Shutdown > Alarm > Ventilation

Tools/Technology To Be Used:

- Circuit Design:Logisim,Multisim,TinkerCad (if logic components available), or Verilog
- Logic Simplification: K-map templates, Boolean Algebra rules

Aim:

To design and implement a digital thermostat system that compares two 4-bit temperature inputs (T_room and T_safe) and generates appropriate control signals—Overheat Alarm (A), Ventilation (V), and System Shutdown (S)—based on defined temperature conditions, using digital logic design techniques and simulation tools such as Logisim or Verilog.

2-Bit Magnitude Comparator

Description:

A **2-bit magnitude comparator** is a combinational logic circuit used to compare two 2-bit binary numbers and determine their relative magnitude. It takes two inputs, **A (A1 A0)** and **B (B1 B0)**, and produces three outputs indicating whether one number is greater than, equal to, or less than the other.

The comparator generates:

G (A > B) when the value of A is greater than B

E (A = B) when both numbers are equal

L (A < B) when A is less than B

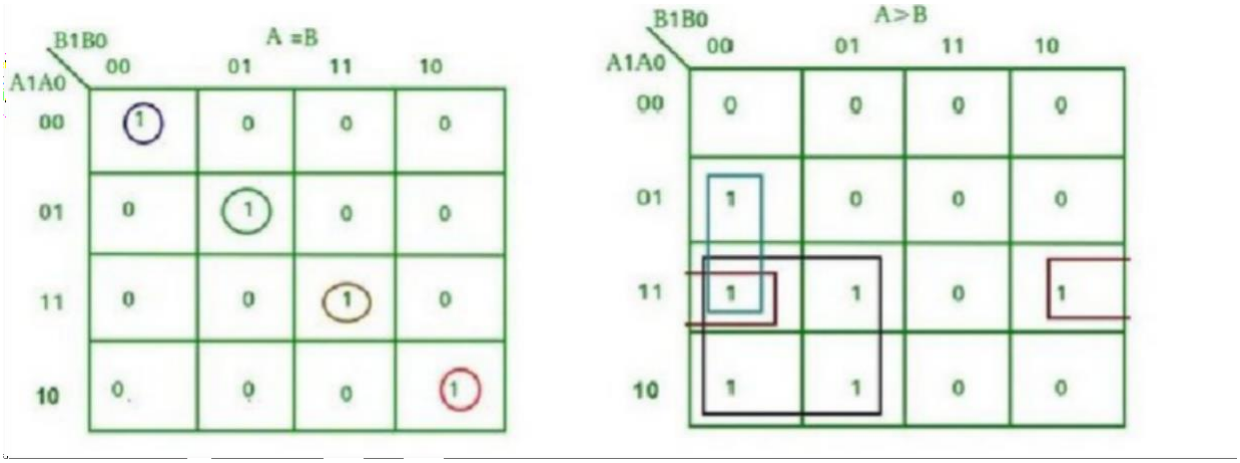
The comparison is performed by first checking the ***most significant bits (MSB)***, A1 and B1. If they are different, the result is decided immediately based on these bits. If the MSBs are equal, then the ***least significant bits (LSB)***, A0 and B0, are compared to determine the final result.
This approach makes the circuit efficient, as it avoids unnecessary comparisons once a decision is made at the higher bit level. The comparator is implemented using basic logic gates such as AND, OR, NOT, and XNOR.

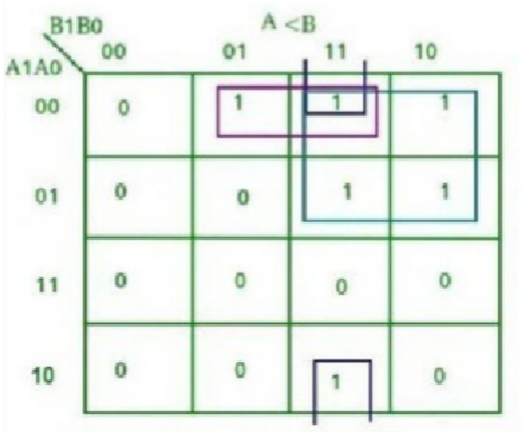
2-bit magnitude comparators are commonly used in digital systems such as arithmetic logic units (ALUs), decision-making circuits, and data comparison operations in processors.

Truth table:

A1	A0	B1	B0	G	E	L
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

K-Map:





Equation:

GreaterThanEquation : $A_1B_1' + A_0B_1'B_0' + A_1A_0B_0'$

EqualityEquation: $A_1'A_0'B_1'B_0' + A_1'A_0B_1'B_0 + A_1A_0B_1B_0 + A_1A_0'B_1B_0'$

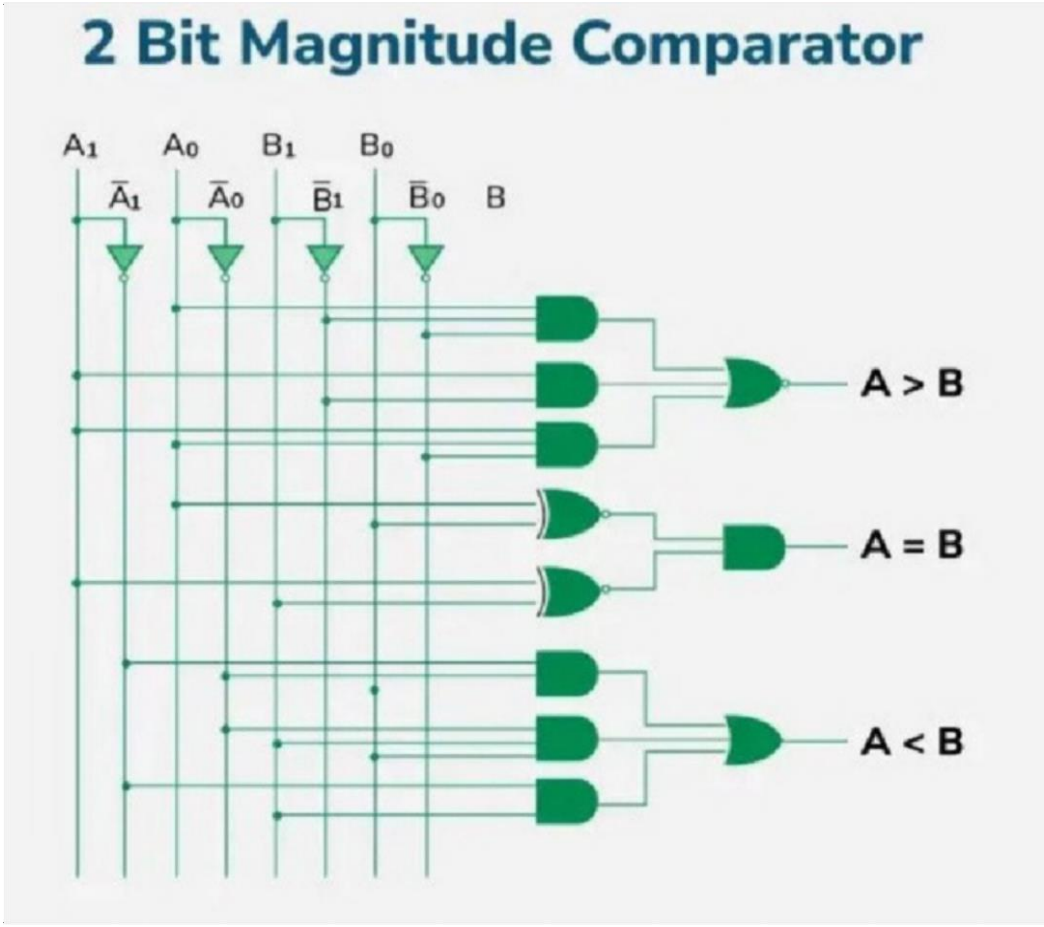
$:A_1'B_1' (A_0'B_0' + A_0B_0) + A_1B_1 (A_0B_0 + A_0'B_0')$

$:(A_0B_0 + A_0'B_0') (A_1B_1 + A_1'B_1')$

$:(A_0$ Ex-Nor $B_0) (A_1$ Ex-Nor $B_1)$

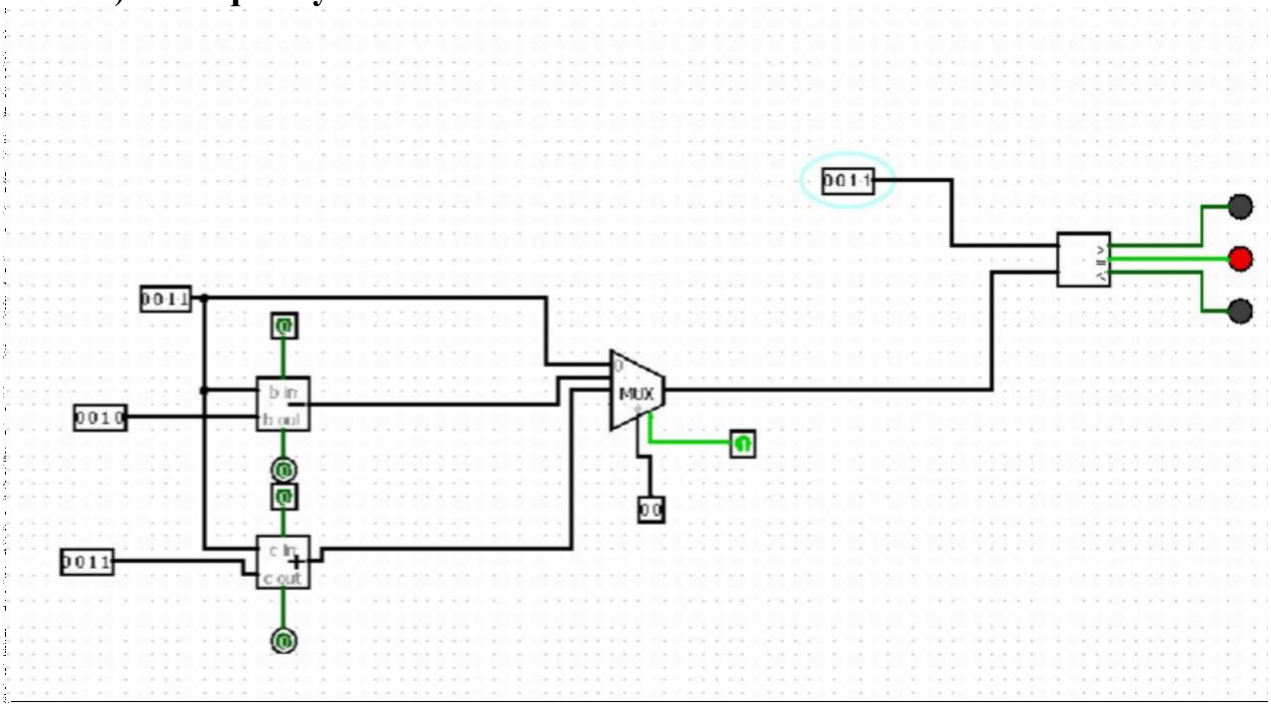
LessThanEquation: $A_1'B_1 + A_0'B_1B_0 + A_1'A0'B0$

Cricuit:

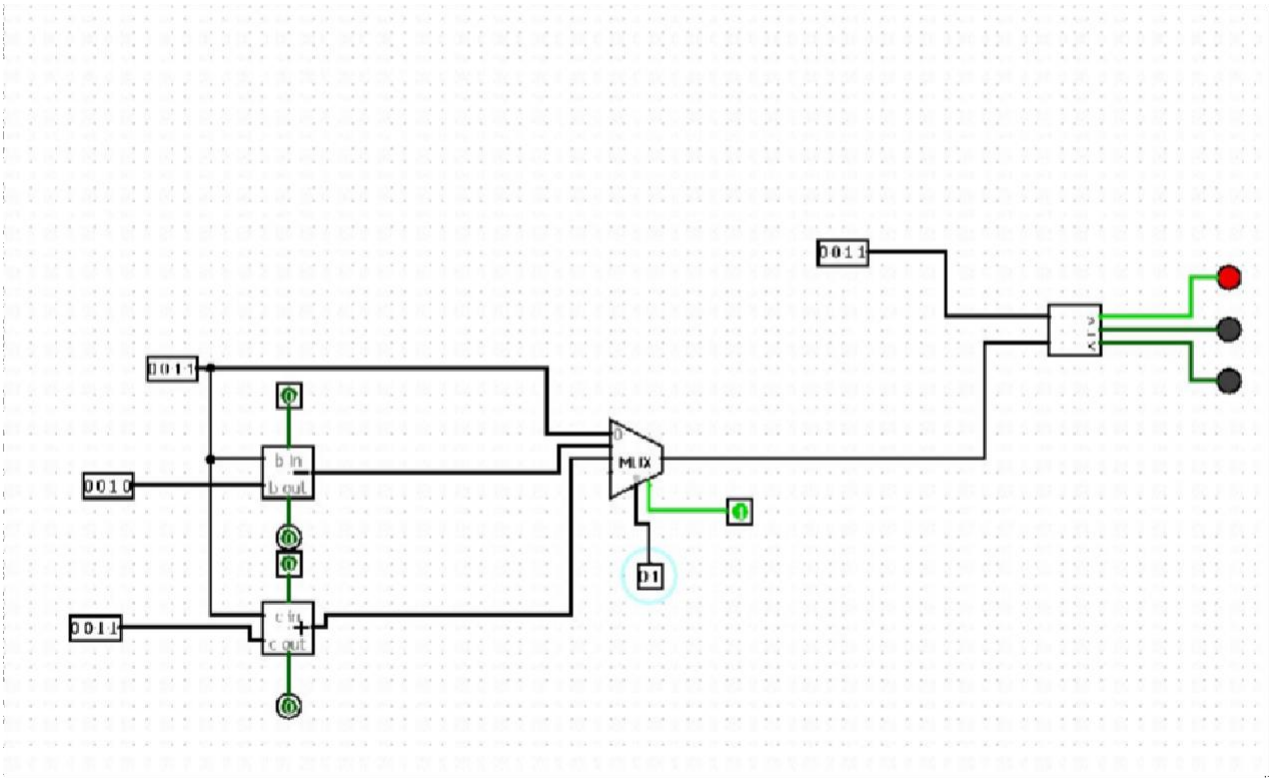


Screen shot:

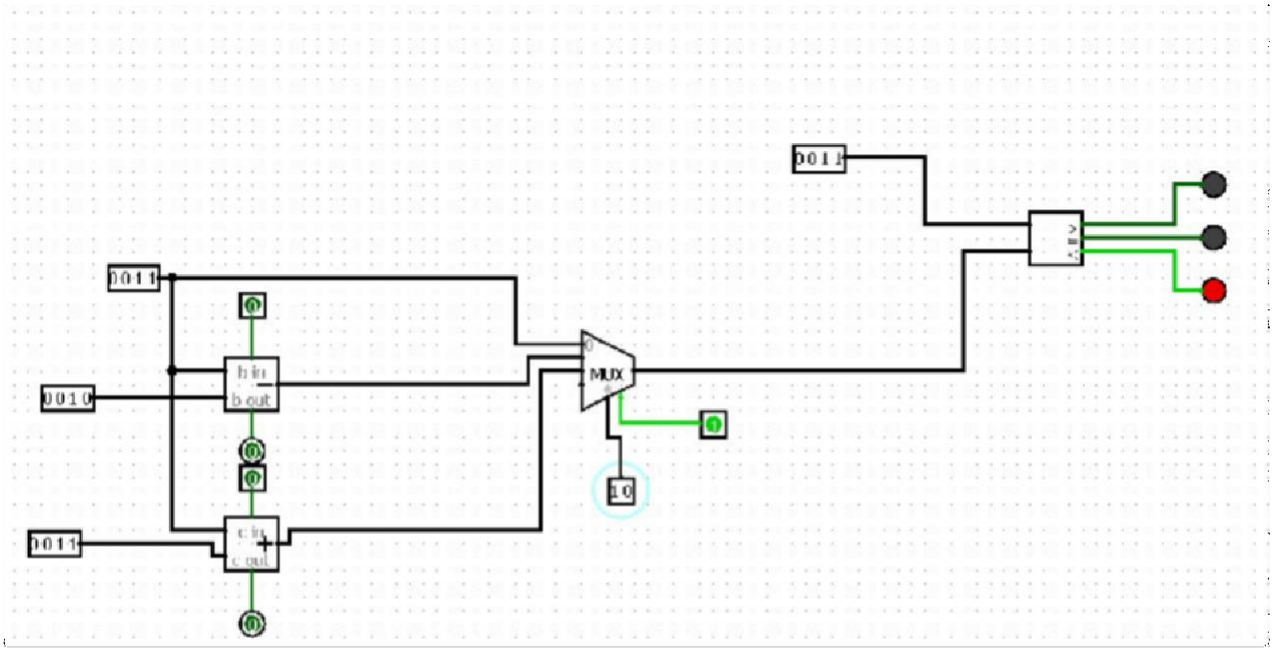
1) For equality:



2) For Greater than:



3) For Less than:



Conclusion: This digital thermostat practical demonstrates the integration of magnitude comparators and combinatorial logic to create a responsive safety system. By implementing specific arithmetic offsets (T_{safe} - 2 and T_{safe} + 3), the design proves how binary adders and subtractors can establish dynamic thresholds for environmental control. The application of priority logic ensures that critical safety states (Shutdown) override lower-level responses (Ventilation), mimicking real-world industrial fail-safes. Ultimately, the project validates the use of gate-level digital electronics in transforming raw sensor data into a structured, hierarchical decision-making system.

Practical-4

Problem Definition:

Designandsimulate a secure digital communication system where two data sources (D1 and D2) are transmitted over a single channel using a 2:1 Multiplexer and received using a 1:2 Demultiplexer. Before transmission, the data should be encrypted using a bitwise XOR encryption key, and at the receiver side, the same key should be used to decrypt the data.

The objective is to verify that encryption and decryption operations cancel out correctly, ensuring data integrity across transmission.

Requirements / Specifications:

- Use a 2:1 Multiplexer to select between two data sources (D1 and D2).
- Apply bitwise XOR encryption with a predefined key (K).
- At the receiver end, use a 1:2 Demultiplexer to separate the transmitted data.
- Apply the same XOR key for decryption to retrieve the original data.
- Validate through truth tables and simulation results that encryption and decryption are complementaryoperations.

Supplementary Problems:

- Modify the design for 4 datasources using a 4:1 MUX and 1:4 DEMUX.
- Add an error detection mechanism using parity bits.
- Replace XOR encryption with bit rotation or substitution encryption for enhanced security.

Tools/Technology To Be Used:

- Simulation Software: Logisim Evolution / Proteus / Multisim / Tinkercad Circuits
- Logic Components: 2:1 Multiplexer, 1:2 Demultiplexer, XOR gates
- Verification Method: Truth tables and waveform simulation

Aim:

To design1 and simulate a **secure digital communication system** using a **2:1 Multiplexer(MUX)** and **1:2 Demultiplexer(DEMUX)**, where data from two sources is transmitted over a single channel after **bitwise XOR encryption**, and then correctly recovered at the receiver using **XOR decryption with the same key**, thereby verifying that encryption and decryption operations ensure data integrity

4:1 Multiplexer and 1:4 Demultiplexer

Description:

A ***4:1 Multiplexer (MUX)*** and ***1:4 Demultiplexer (DEMUX)*** work as complementary devices in digital systems:

MUX(4:1) : Selects **oneoffour inputs (D0, D1, D2, D3)** using **select lines (S1, S0)** and sends it to a**singleoutput(Y)**.

DEMUX(1:4): Takes **oneinput (D)** and distributes it to **one of four outputs (Y0–Y3)** using the same select lines.

Boolean Expressions:

4:1 MUX Output

1:4 DEMUX Outputs $Y = S^{-}1\ S^{-}0\ D0 + S^{-}1\ S0\ D1 + S1\ S^{-}0\ D2 + S1\ S0\ D3$

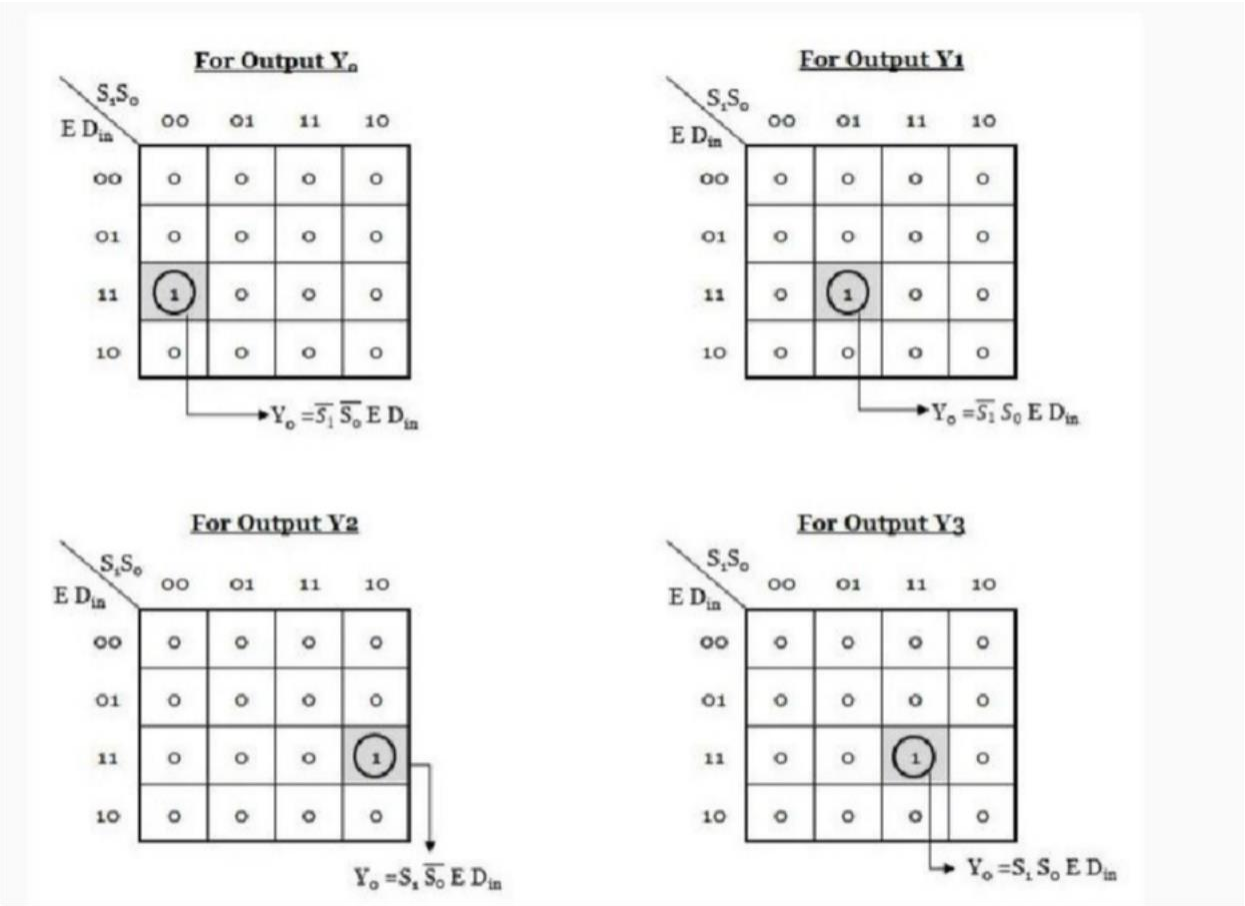
$$Y0 = D \cdot S^{-}1 \cdot S^{-}0$$
$$Y1 = D \cdot S^{-}1 \cdot S0$$
$$Y2 = D \cdot S1 \cdot S^{-}0$$
$$Y3 = D \cdot S1 \cdot S0$$

K-Map:

4:1 Multiplexer:

CD \ AB	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

1:4 Demultiplexer:



Truth table:

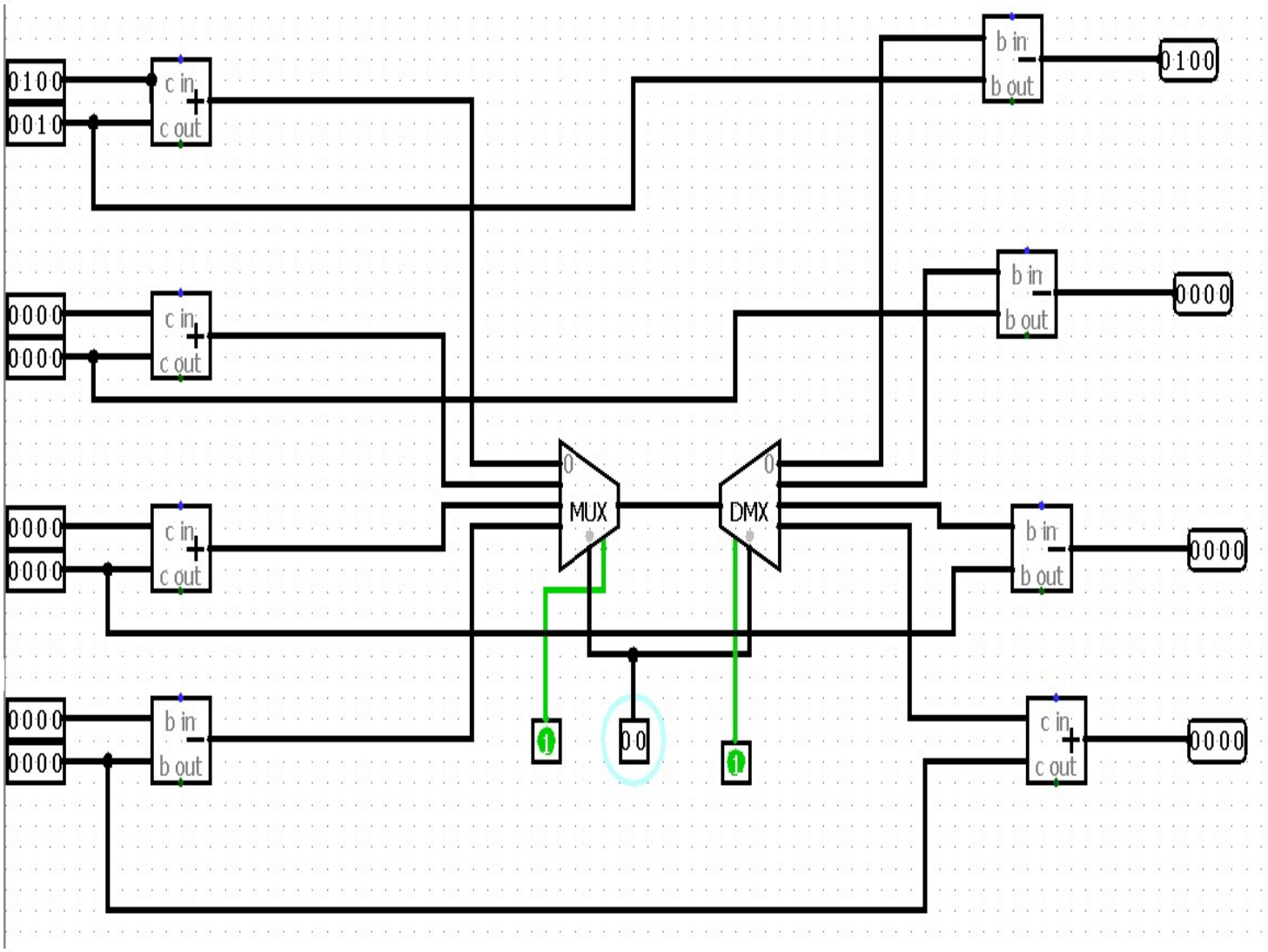
4:1 Multiplexer:

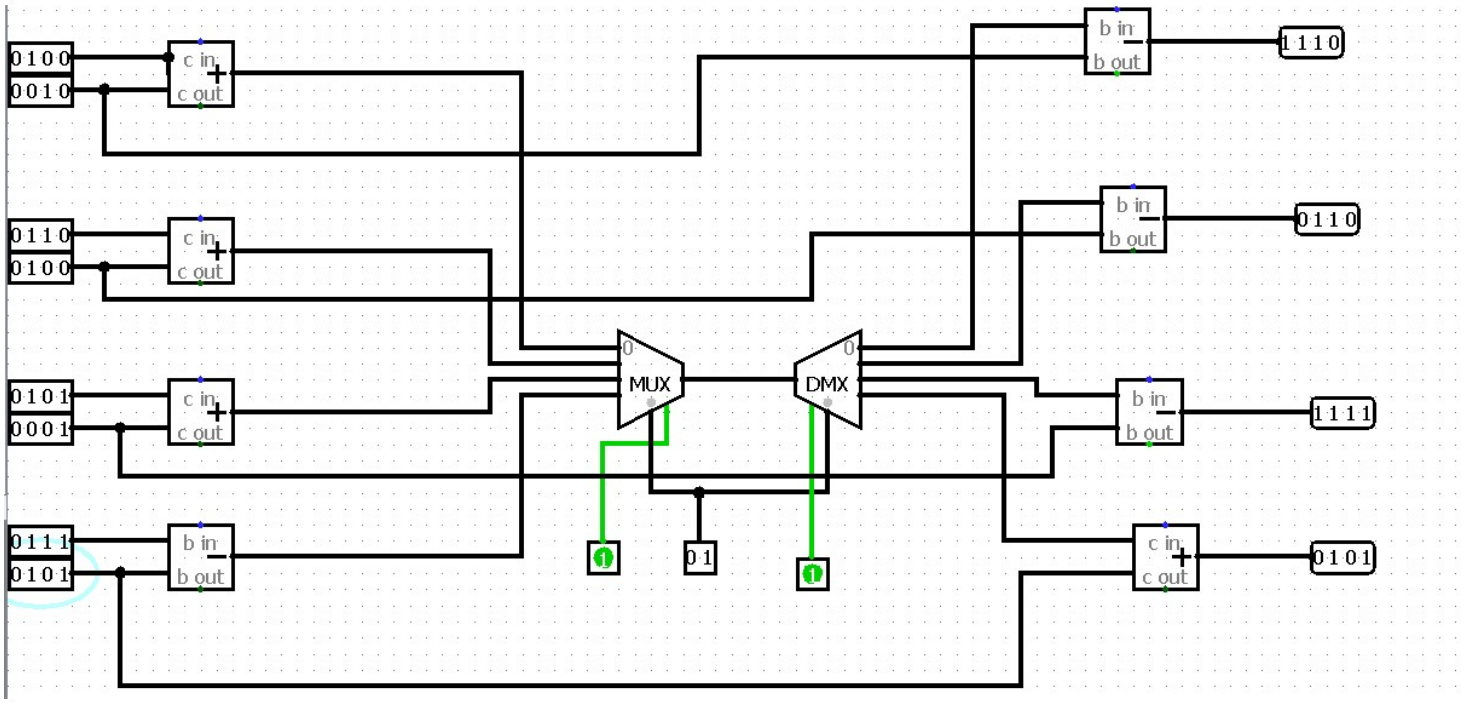
Selection Lines		Output
S0	**S1**	**Y**
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

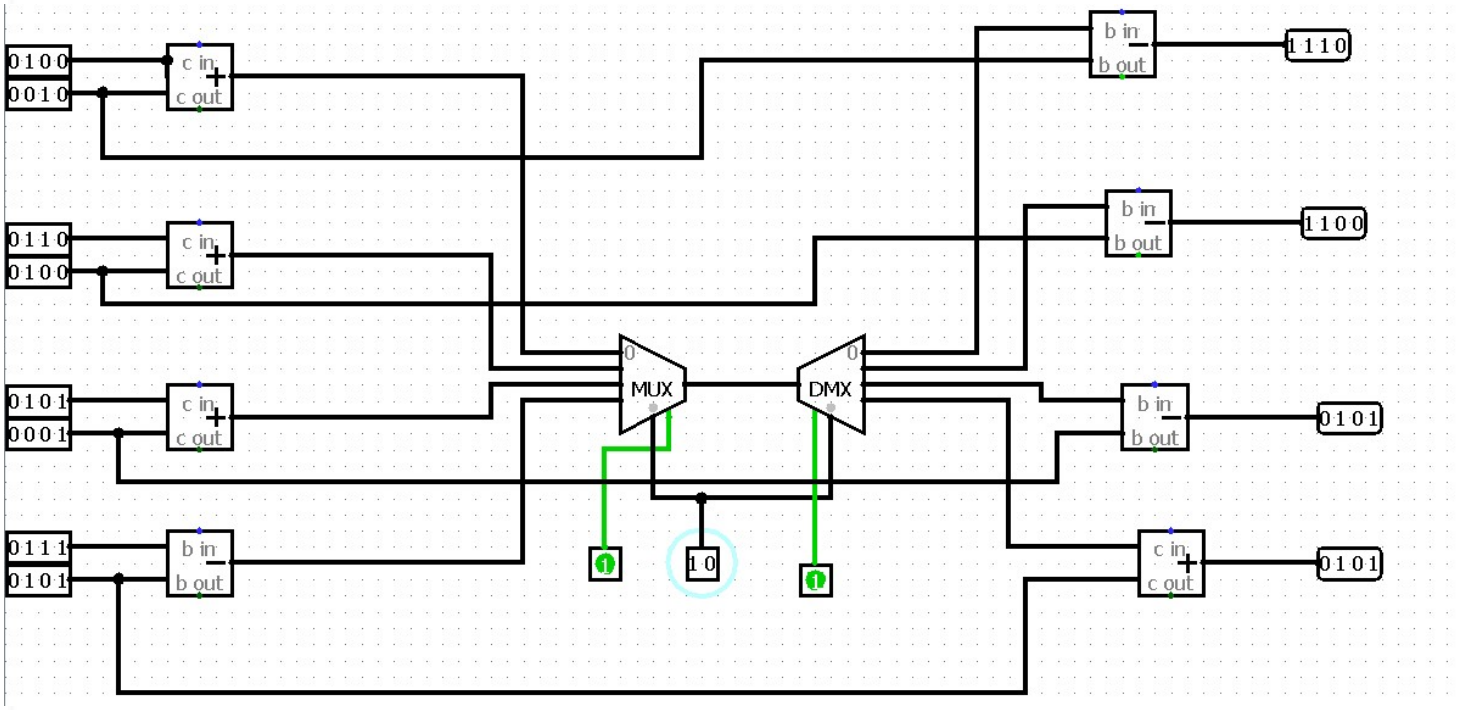
1:4 Demultiplexer:

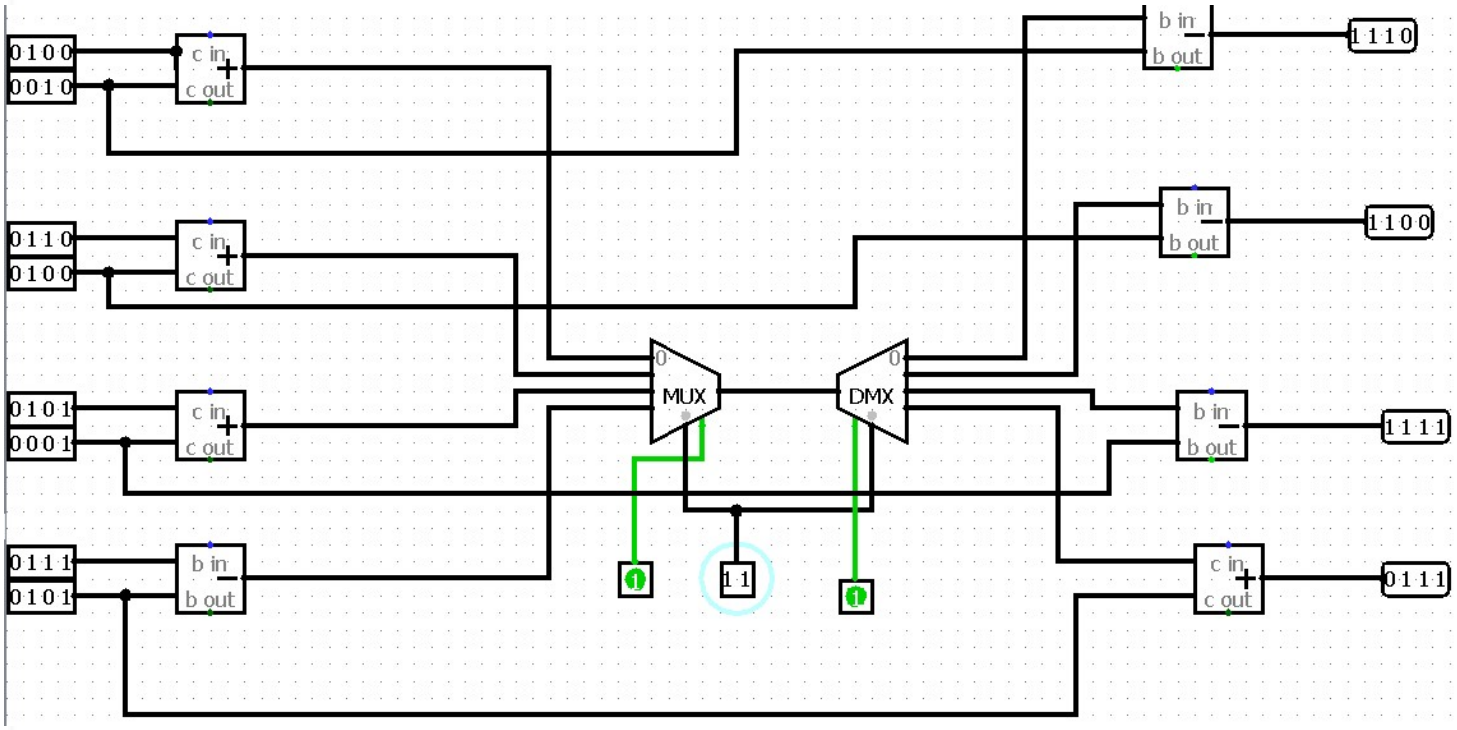
S1	S0	D	Y0	Y1	Y2	Y3
0	0	D	D	0	0	0
0	1	D	0	D	0	0
1	0	D	0	0	D	0
1	1	D	0	0	0	D

Screen shot:

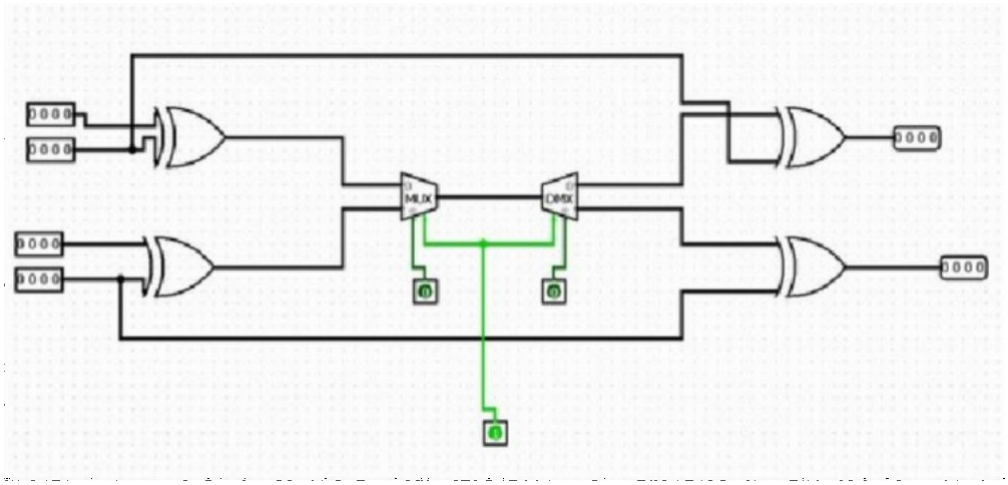

0100
0010
c in
c out
b in
b out
0100
0000
0000
0000
0000
MUX
DMX
0000
0000
0000
0000
00
0000

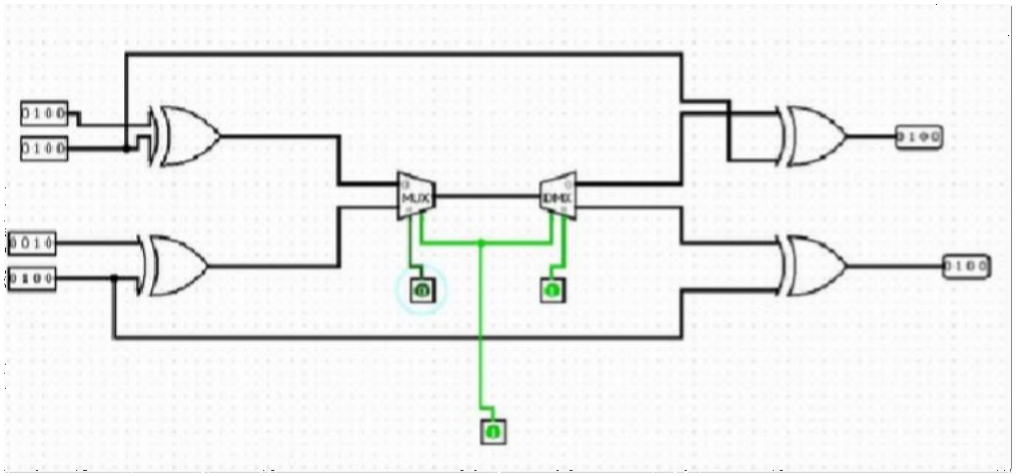

0100
0010
c in
c out
0110
0100
0101
0001
0111
0101
b in
b out
MUX
DMX
01
1110
0110
1111
0101

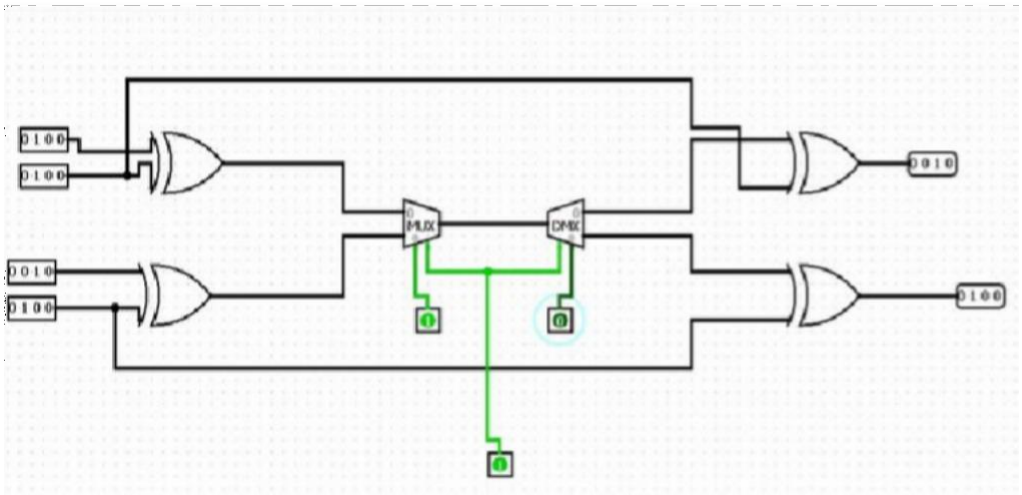

0100
0010
c in
c out
0110
0100
0101
0001
0111
0101
b in
b out
MUX
DMX
10
1110
1100
0101
0101

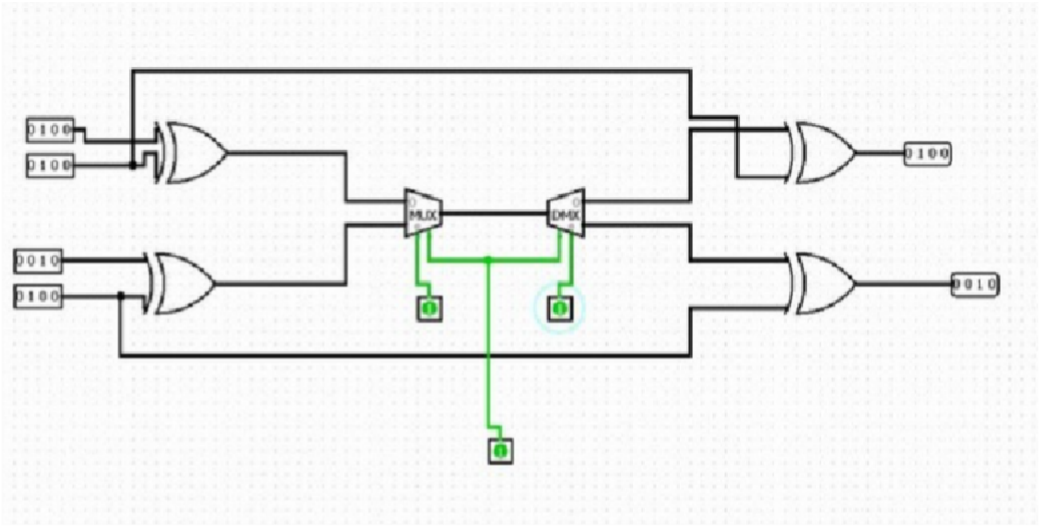

0 1 0 0
0 0 1 0
c in
c out
0 1 1 0
0 1 0 0
0 1 0 1
0 0 0 1
0 1 1 1
0 1 0 1
b in
b out
MUX
DMX
1 1
1 1 1 0
1 1 0 0
1 1 1 1
0 1 1 1

2:1 Multiplexer and 1:2 Demultiplexer









Conclusion: This practical session successfully demonstrates the application of multiplexing and symmetric encryption in digital communication systems. By integrating a 2:1 MUX and 1:2 DEMUX, the design proves that multiple data streams can be efficiently shared over a single physical channel without cross-talk. The implementation of the XOR gate as a stream cipher confirms the fundamental property that bitwise encryption and decryption are inverse operations when using a shared key. Ultimately, the simulation verifies that despite the encryption overhead, data integrity is maintained, highlighting how gate-level logic can be used to secure data against unauthorized access during transmission.

Practical-5

Problem Definition:

- Design and simulate a 4-bit dual-mode counter using JK Flip- Flops that can count up or down based on the MODE control input.

The counter must include:

- An asynchronous CLEAR input to reset the counter to 0000.
- A synchronous LOAD feature to load any 4-bit preset value on command.

The design should include excitation tables, Boolean expressions, and optimized logic equations for all J and K inputs, verified through truth tables and simulation.

Requirements / Specifications:

- Use JK Flip-Flops only.
- Implement MODE control for Up/Down counting.
- Include asynchronous CLEAR to reset the counter.
- Include synchronous LOAD to preset the counter with a 4 bit input value.
- Derive excitation table and Boolean expressions for J and K inputs.
- Optimize logic using K-map simplification.
- Verify the circuit operation using simulation software (Logisim / Proteus / Multisim).

Tools/Technology To Be Used:

- Simulation Software: Logisim Evolution / Proteus / Multisim / Tinkercad Circuits
- Components: JK Flip-Flops, Logic Gates, Clock Pulse Generator
- Verification: Truth table and waveform simulation

Aim:

To design and simulate a 4-bit dual-mode counter using JK Flip-Flops that can count in both up and down directions based on a MODE control input, with features of asynchronous CLEAR for reset and synchronous LOAD for presetting values, and to verify its operation using truth tables and simulation tools.

J-K flip flop:

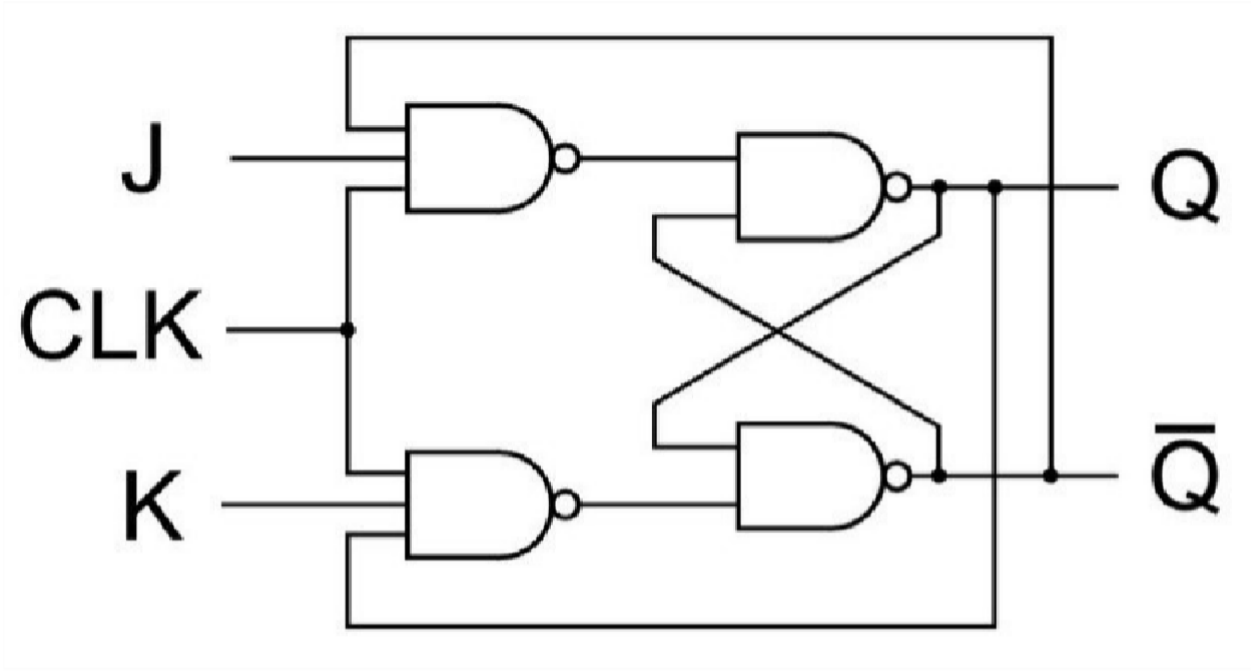
Truth Table:

J	K	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Excitation Table:

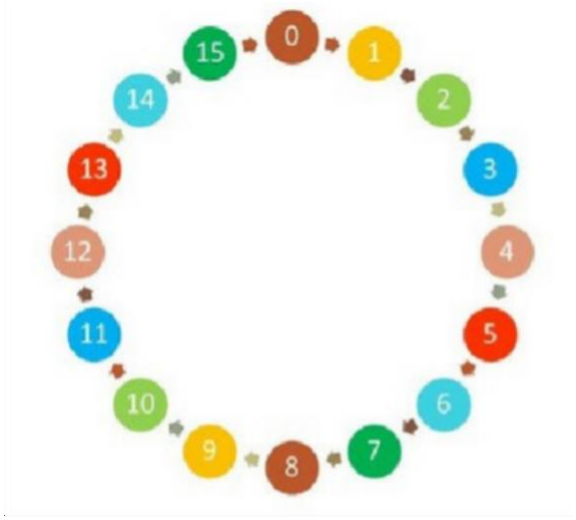
Q	Qn+1	J	J
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Circuit diagram:



4-bit binary up counter:

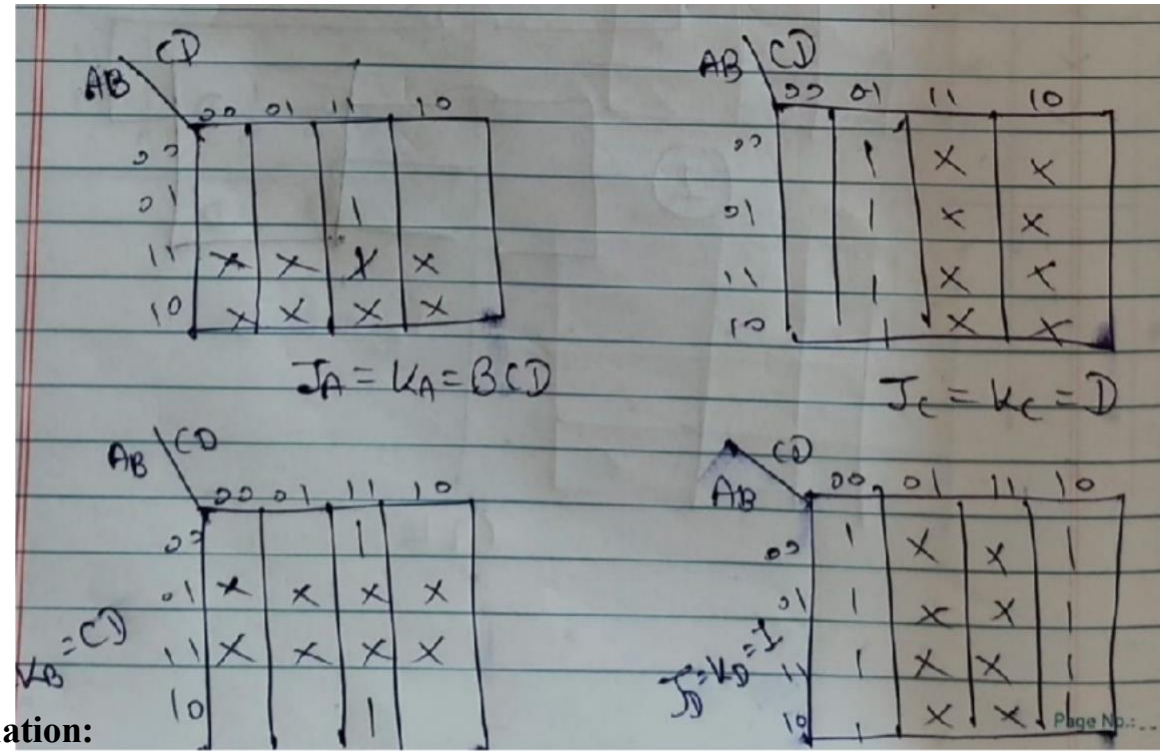
State diagram:



State table:

A_0	B	C	D	A_1	B_1	C_1	D_1	J_A K_A	J_B K_B	J_C K_C	J_D K_D
0	0	0	0	0	0	0	1	0 x	0 x	0 x	1 x
0	0	0	1	0	0	1	0	0 x	0 x	1 x	x 1
0	0	1	0	0	0	1	1	0 x	0 x	x 0	1 x
0	0	1	1	0	1	0	0	0 x	1 x	x 1	x 1
0	1	0	0	0	1	0	1	0 x	x 0	0 x	1 x
0	1	0	1	0	1	1	0	0 x	x 0	1 x	x 1
0	1	1	0	0	1	1	1	0 x	x 0	x 0	1 x
0	1	1	1	1	0	0	0	1 x	x 1	x 1	x 1
1	0	0	0	1	0	0	1	x 0	0 x	0 x	1 x
1	0	0	1	1	0	1	0	x 0	0 x	1 x	x 1
1	0	1	0	1	0	1	1	x 0	0 x	x 0	1 x
1	0	1	1	1	1	0	0	x 0	1 x	x 1	x 1
1	1	0	0	1	1	0	1	x 0	x 0	0 x	1 x
1	1	0	1	1	1	1	0	x 0	x 0	1 x	x 1
1	1	1	0	1	1	1	1	x 0	x 0	x 0	1 x
1	1	1	1	0	0	0	0	x 1	x 1	x 1	x 1

K -map:



Equation:

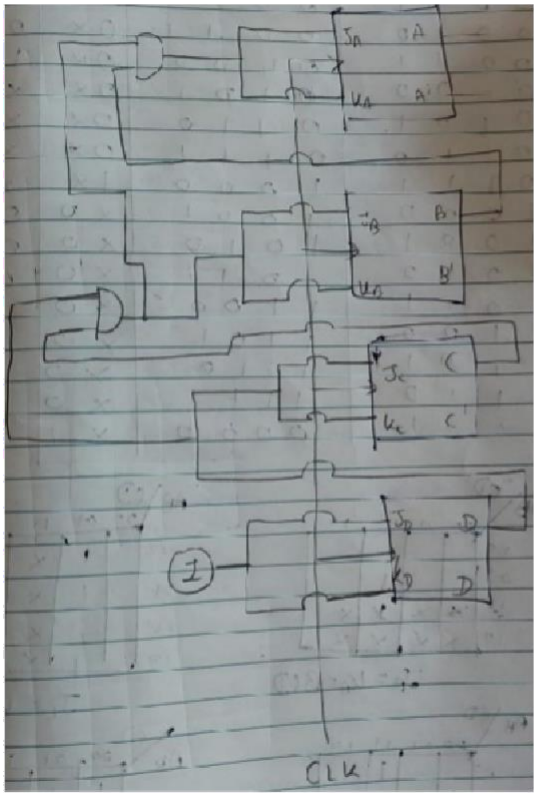
For Flip-Flop A: $J^A=K^A=B\cdot C\cdot D$

For Flip-Flop B: $JB=K^B=C\cdot D$

For Flip-Flop C:
Jc=Kc=D

For Flip-Flop D: Jd=Kd=1

Circuit:



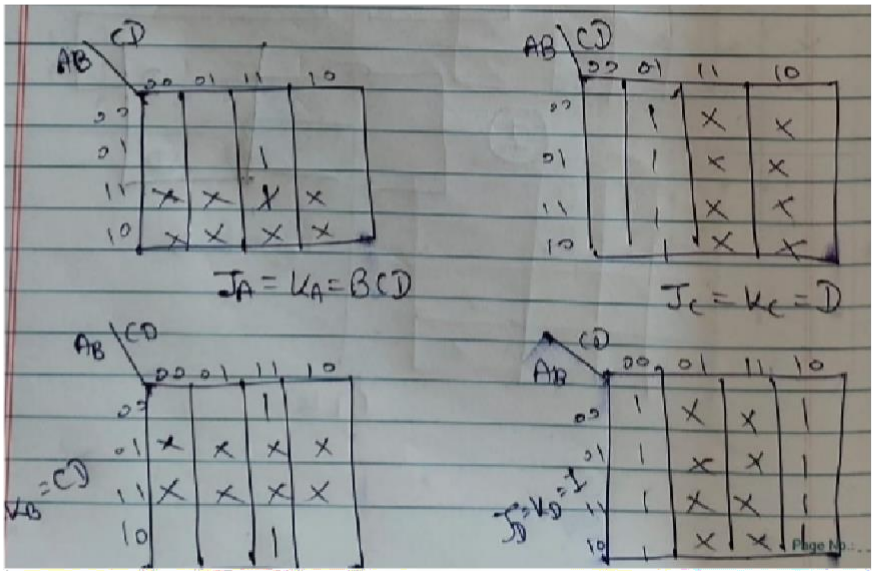
Down counter:

State Table:

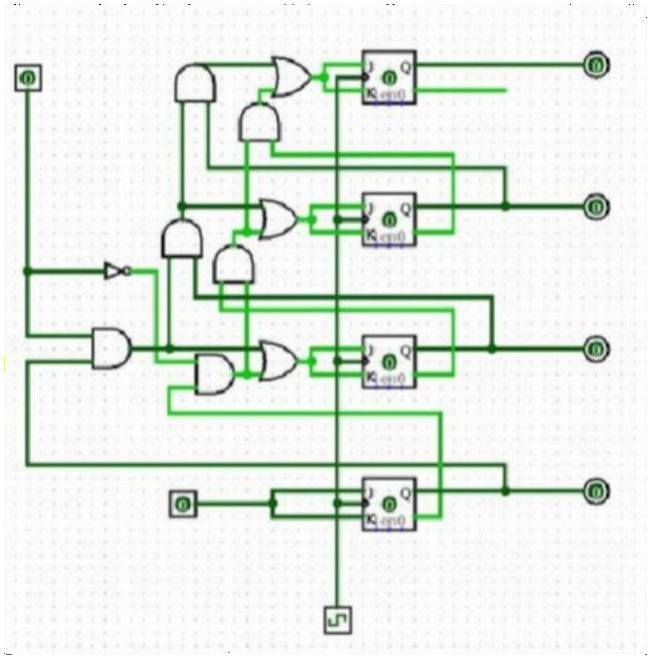
STATE	A	B	C	D	A+	B+	C+	D+	JA	KA	JB	KB	JC	KC	JD	KD
0	0	0	0	0	1	1	1	1	1	X	1	X	1	X	1	X
1	0	0	0	1	0	0	0	0	0	X	0	X	0	X	X	1
2	0	0	1	0	0	0	0	1	0	X	0	X	X	1	1	X
3	0	0	1	1	0	0	1	0	0	X	0	X	X	0	X	1
4	0	1	0	0	0	0	1	1	0	X	X	1	1	X	1	X
5	0	1	0	1	0	1	0	0	0	X	X	0	0	X	X	1

STATE	A	B	C	D	A+	B+	C+	D+	JA	KA	JB	KB	JC	KC	JD	KD
6	0	1	1	0	0	1	0	1	0	X	X	0	X	1	1	X
7	0	1	1	1	0	1	1	0	0	X	X	0	X	0	X	1
8	1	0	0	0	0	1	1	1	X	1	1	X	1	X	1	X
9	1	0	0	1	1	0	0	0	X	0	0	X	0	X	X	1
10	1	0	1	0	1	0	0	1	X	0	0	X	X	1	1	X
11	1	0	1	1	1	0	1	0	X	0	0	X	X	0	X	1
12	1	1	0	0	1	0	1	1	X	0	X	1	1	X	1	X
13	1	1	0	1	1	1	0	0	X	0	X	0	0	X	X	1
14	1	1	1	0	1	1	0	1	X	0	X	0	X	1	1	X
15	1	1	1	1	1	1	1	0	X	0	X	0	X	0	X	1

K-map & Equation:

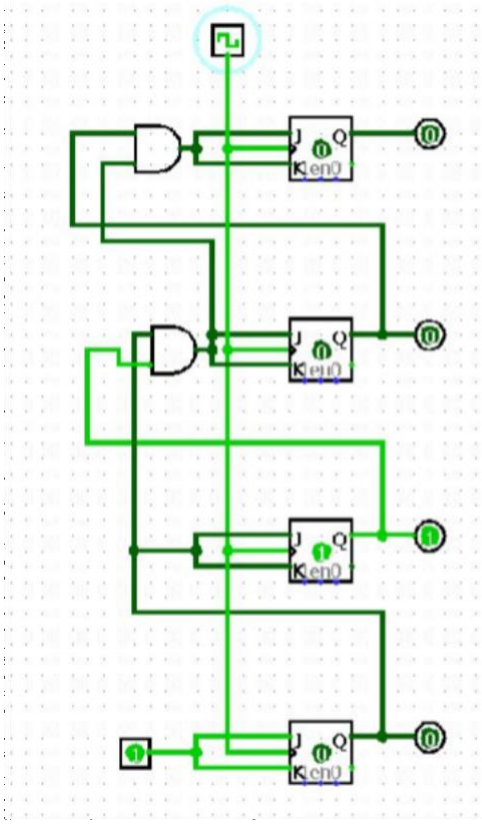


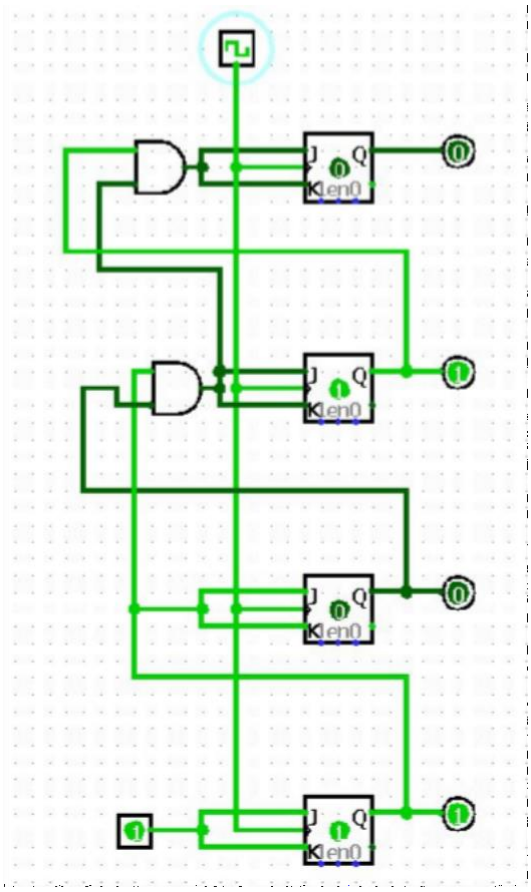
4 Bit Combine circuit:

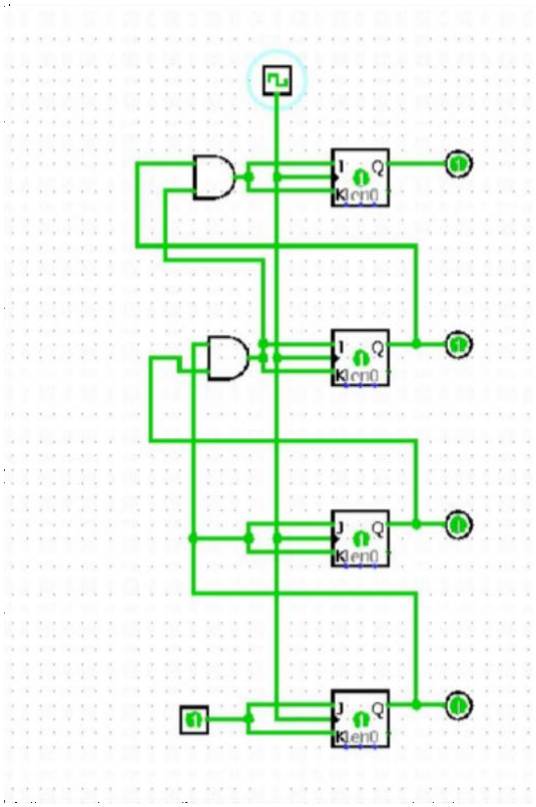


Screen shots:

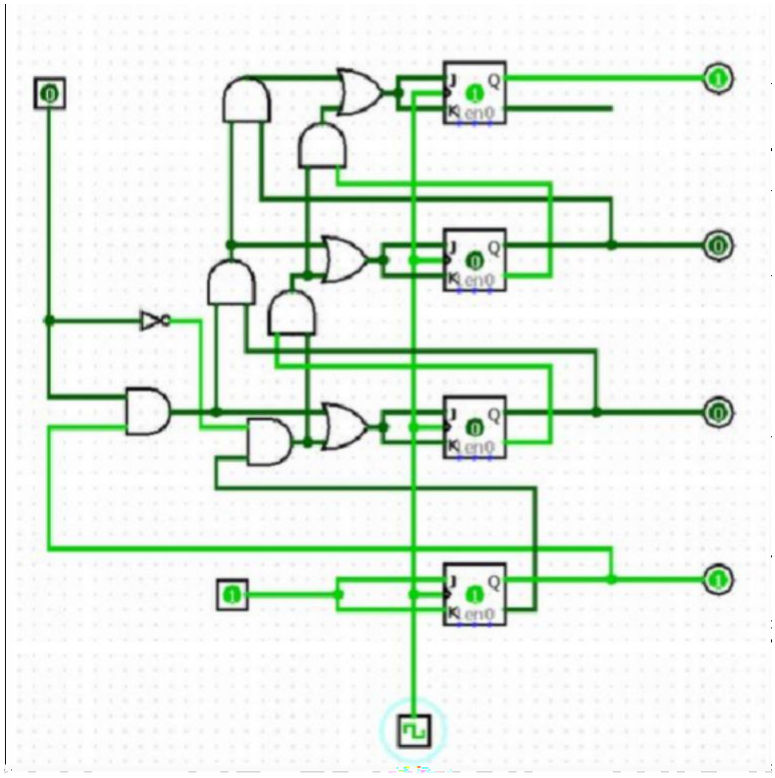
Up-counter:

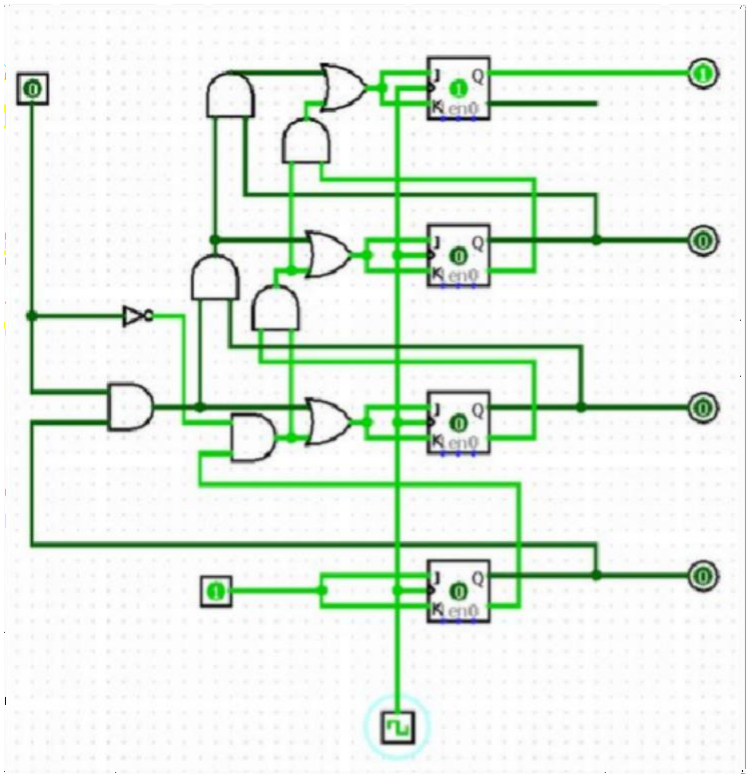


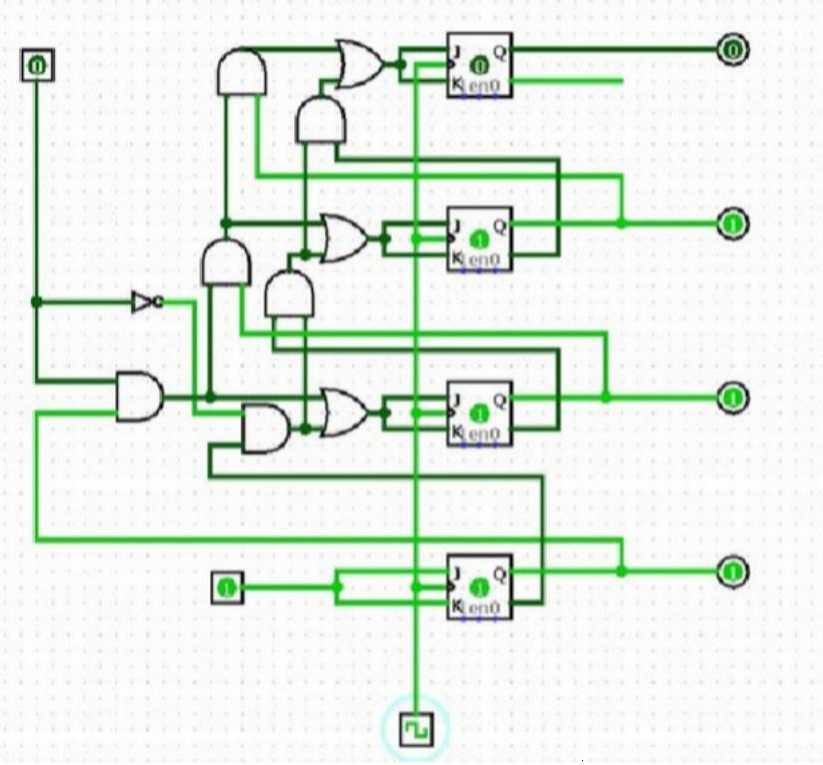

J
Q
K
en
0
1


J
Q
K
en
0
1

Combine (down counter):







Conclusion:

The 4-bit dual-mode counter using JK flip-flops was successfully designed and simulated. It correctly performs both up and down counting based on the MODE input. The asynchronous CLEAR resets the counter to 0000, and the synchronous LOAD allows presetting any value. The excitation tables and Boolean expressions were simplified using K-maps, and simulation verified proper operation of the circuit.

Practical 6

Problem Definition

Design a 4-bit synchronous counter that counts from 0000 (0) to 1001 (9) and then resets automatically to 0000. Use T Flip-Flops for implementation. The counter should operate in a synchronous manner, meaning all flip-flops are triggered by the same clock pulse. The design must include:

1. State diagram representing the 10 valid states (0–9).

2. Excitation table showing required T inputs for each flip-flop transition.

3. K-map minimization to derive simplified Boolean expressions for each T input.

4. Implementation and verification through simulation, ensuring the counter resets after 9 (MOD-10 operation).

Requirements / Specifications:

- Use four T Flip-Flops (T_3, T_2, T_1, T_0).
- The counter must follow the sequence 0000 → 0001 → 0010 → … → 1001 → 0000.
- Include an automatic reset logic for invalid states (1010–1111).
- Derive excitation table and minimized Boolean equations for all T inputs.
- Verify operation via simulation and truth table. Tools/Technology To Be Used
- Simulation Software: Logisim Evolution / Proteus / Multisim / Tinkercad Circuits
- Components: 4 × T Flip-Flops, AND/OR gates for reset logic
- Verification Method: Truth table and waveform observation

Aim :

To design and implement a 4-bit synchronous MOD-10 counter using T Flip-Flops that counts from binary 0000 (0) to 1001 (9) and automatically resets to 0000, ensuring correct synchronous operation and verification through simulation.

T flip flop:

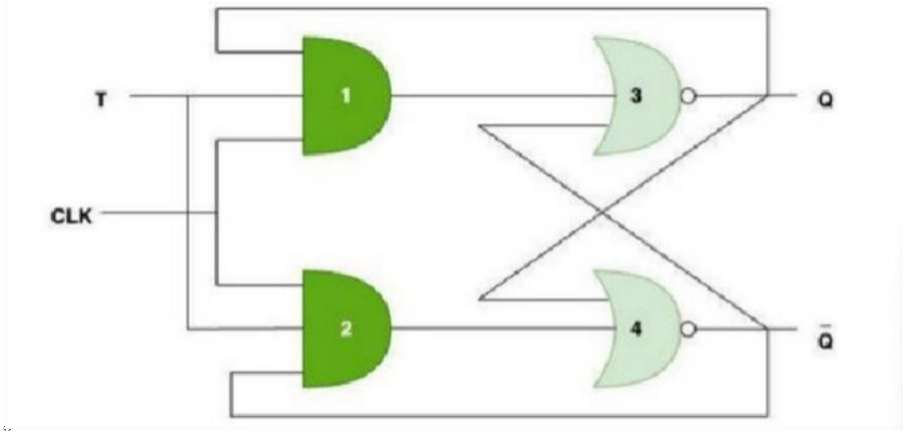
Truth table:

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1

Excitation Table:

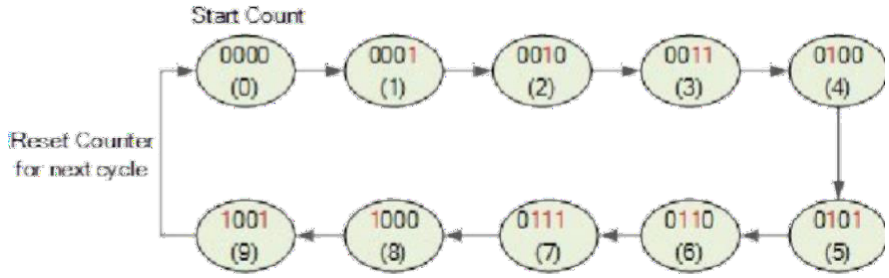
Q_t	Q(t+1)	T
0	0	0
1	0	1
0	1	1

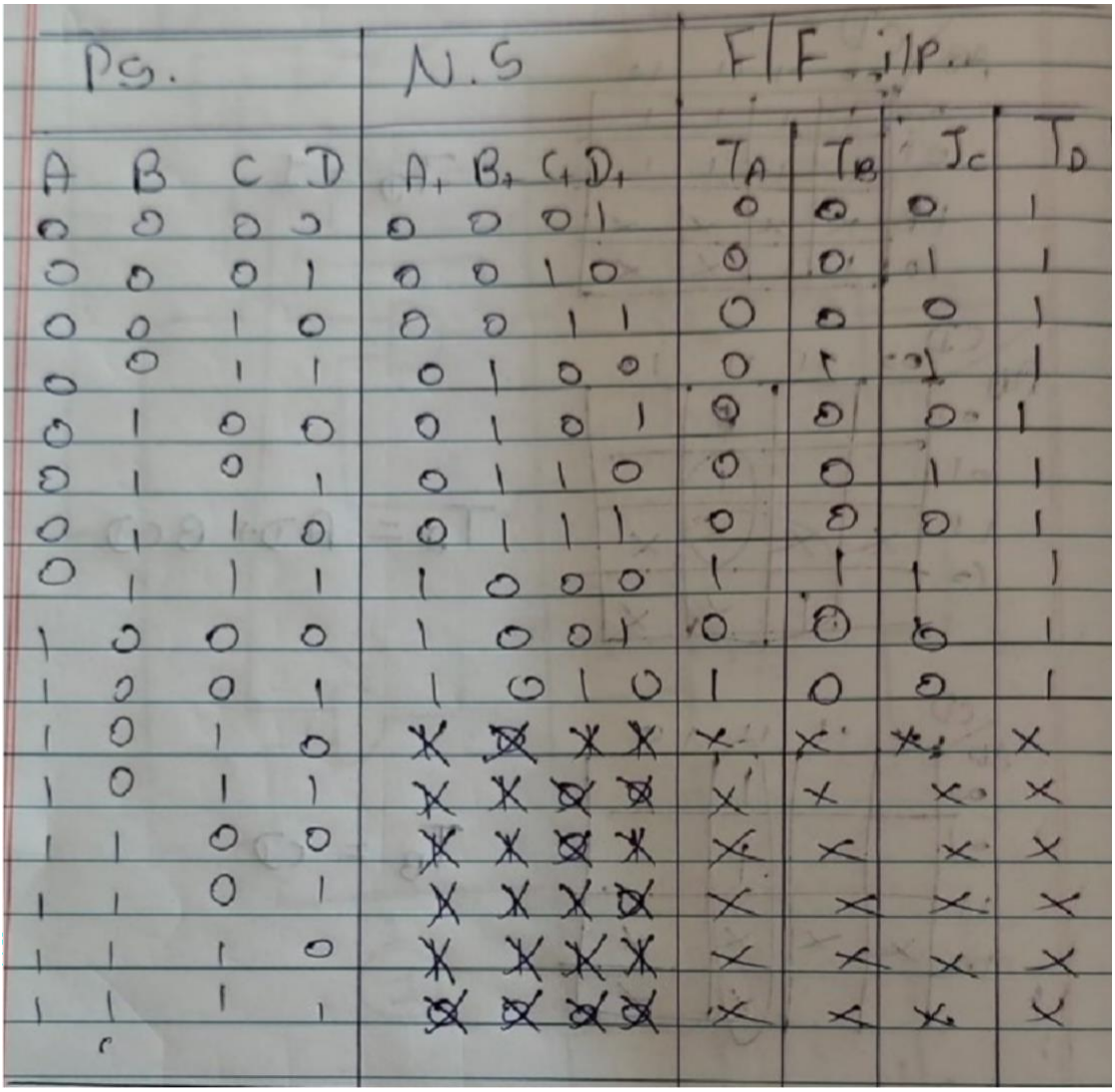
Circuit Diagram:



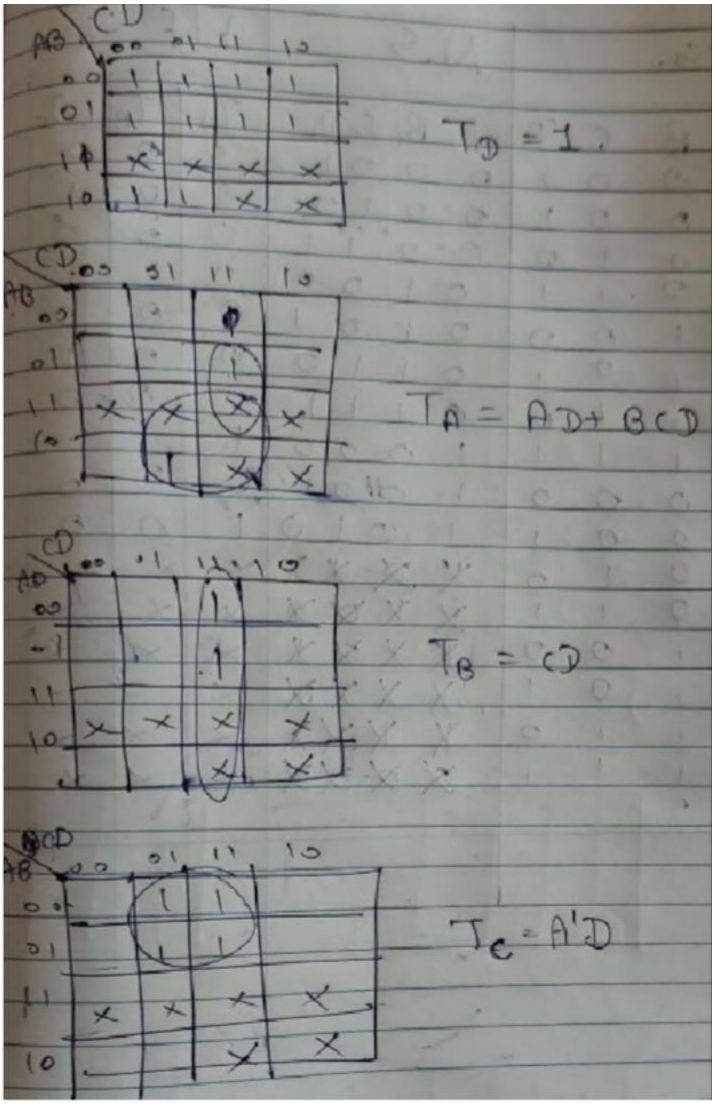
4-bit binary 0-9 counter:

State diagram & state table:

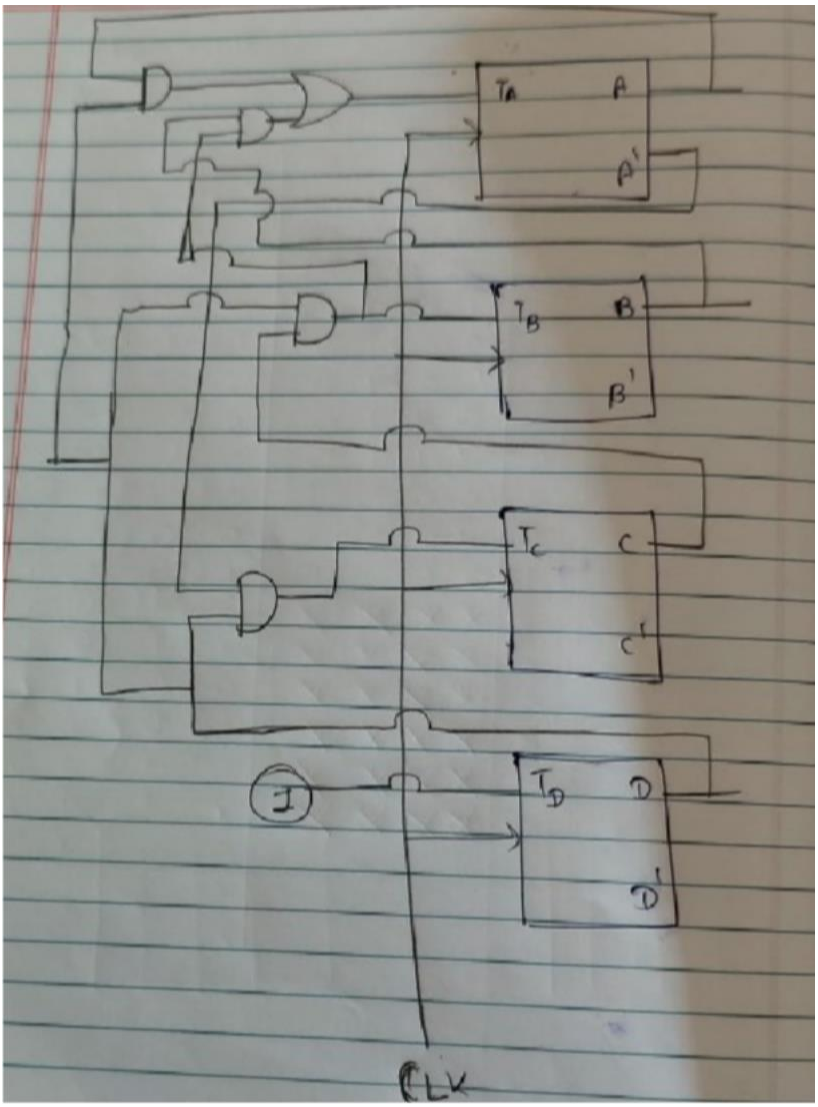




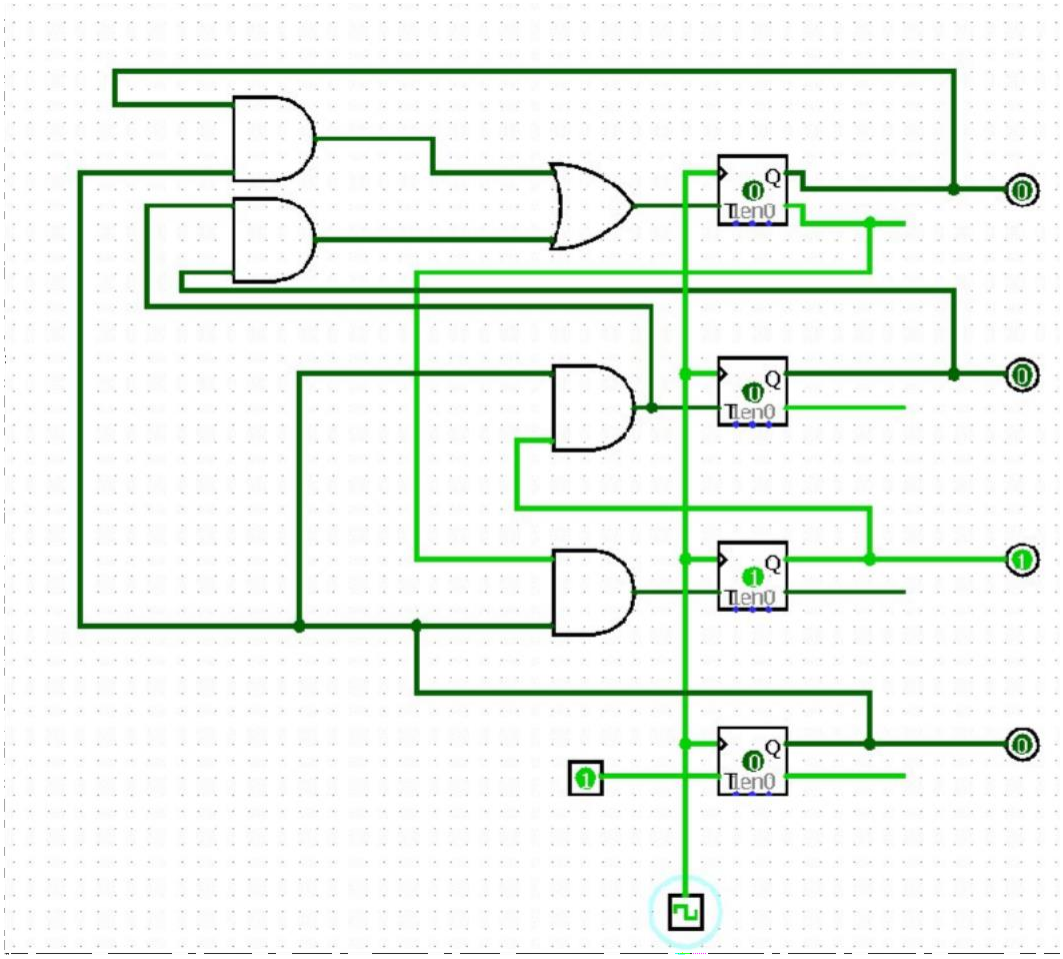
K-Map:



Circuit Diagram:



Screen shot:



Conclusion: This practical successfully demonstrates the design of a 4-bit bidirectional synchronous counter utilizing T flip-flops to achieve a toggle-based counting sequence. By applying K-map simplification to the excitation tables, the design proves that a single MODE control signal can effectively manipulate the toggle logic to switch between up and down counting modes. The inclusion of synchronous LOAD and asynchronous CLEAR functions highlights the precision of clock-driven data entry versus the immediate priority of hardware resets. Ultimately, the simulation confirms that using T flip-flops provides a streamlined and hardware-efficient approach to building versatile sequential circuits with synchronized state transitions.

Practical-7

Problem Definition:

Design and implement a 4-bit synchronous counter that follows the non-linear custom sequence:

2 → 5 → 7 → 11 → 3 → 2 → … (repeating)

Requirements:

1. Draw the state diagram and state transition table.

2. Use any one type of flip-flop (JK, T, or D) for the design.

3. Derive flip-flop input equations using the respective excitation table.

4. Implement the complete synchronous circuit and verify the repeating sequence using simulation software or hardware.

Tools/Technology To Be Used:

- Flip-flops (D/JK/T)
- Logic gates
- Simulation software: Logisim / Multisim / Proteus / TinkerCad / Digital
- Breadboard & ICs (optional for hardware testing)

Aim:

To design and implement a 4-bit synchronous counter that follows a non-linear custom sequence (2 → 5 → 7 → 11 → 3 → 2 → …) using flip-flops and logic gates. The objective includes deriving the required flip-flop input equations using excitation tables and verifying the correct sequence through simulation or hardware implementation.

T flip flop:

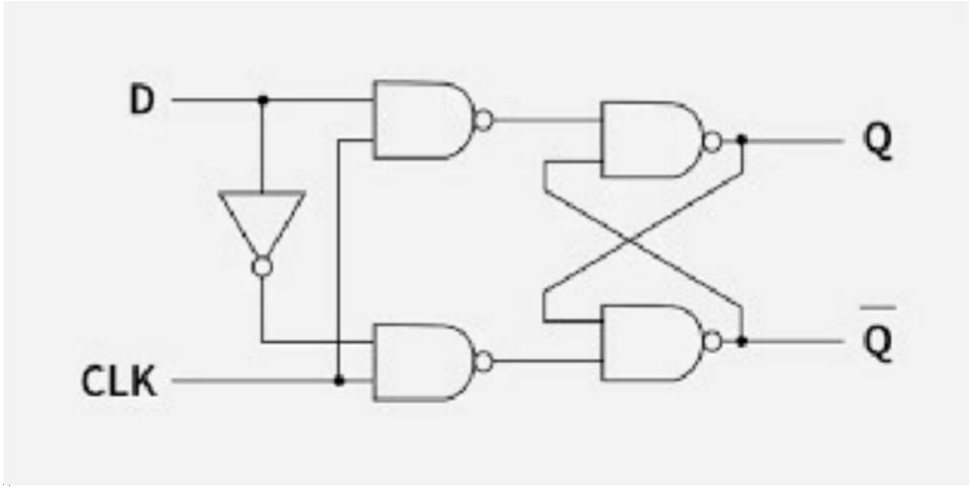
Truth table:

Q(n)	D(n)	Q(n+1) (Next)
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Table:

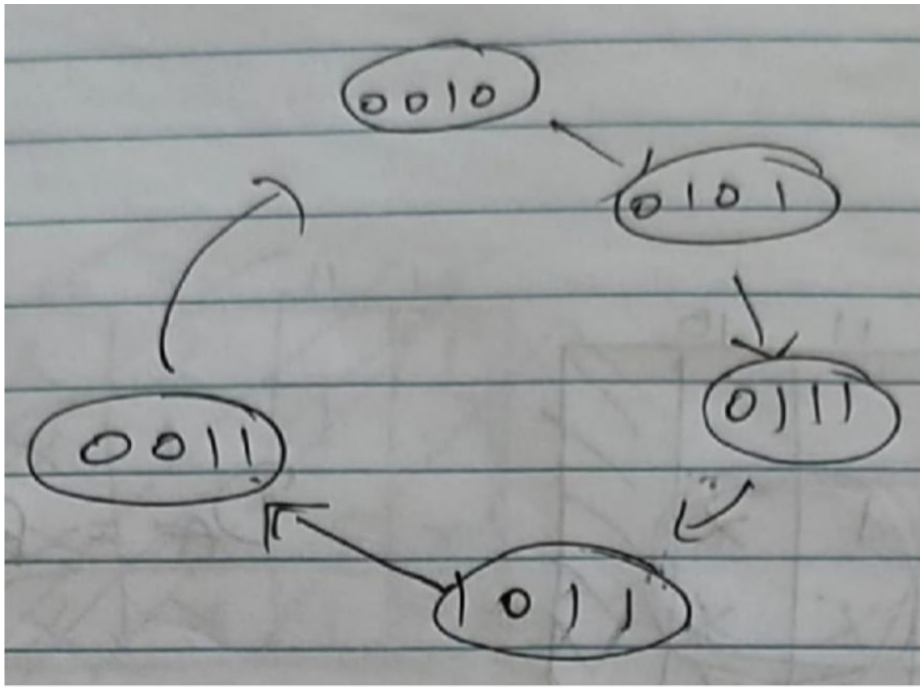
Q Output		Input
$Q_{(n)}$	$Q_{(n+1)}$	D_n
0	0	0
0	1	1
1	0	0
1	1	1

Circuit Diagram:



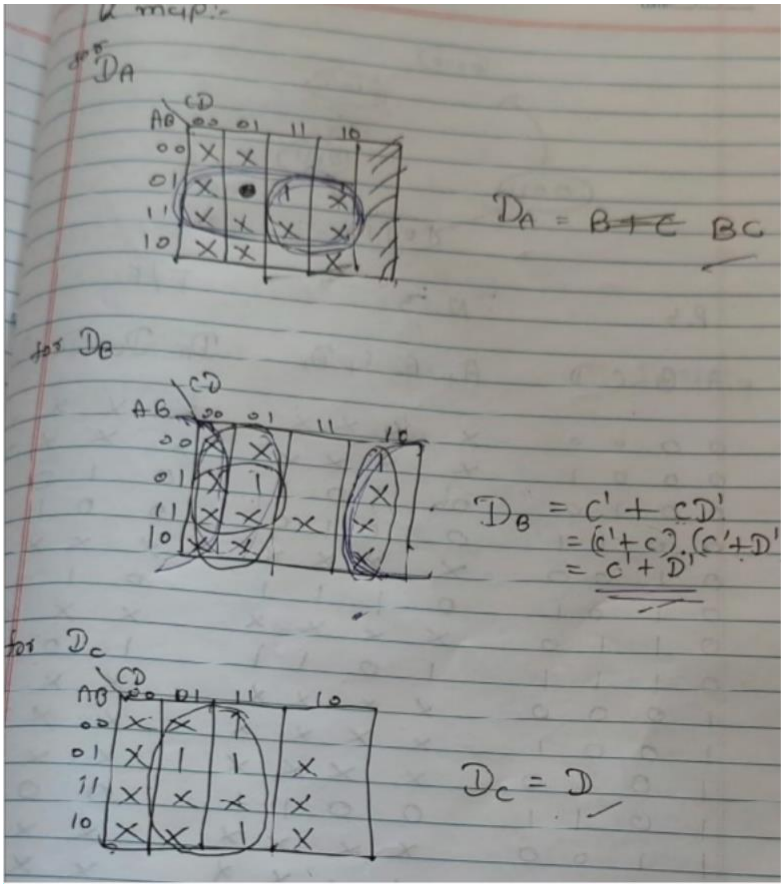
4-bit binary Random counter:

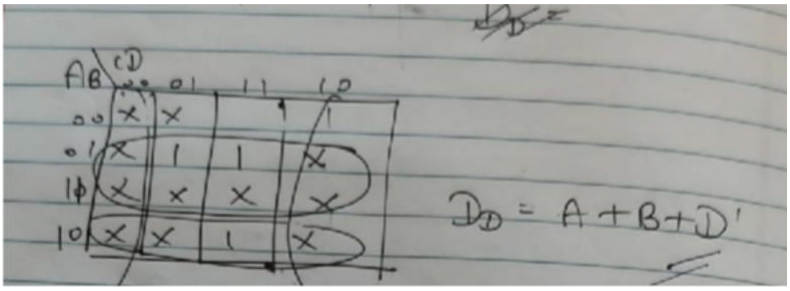
State diagram & State Table:



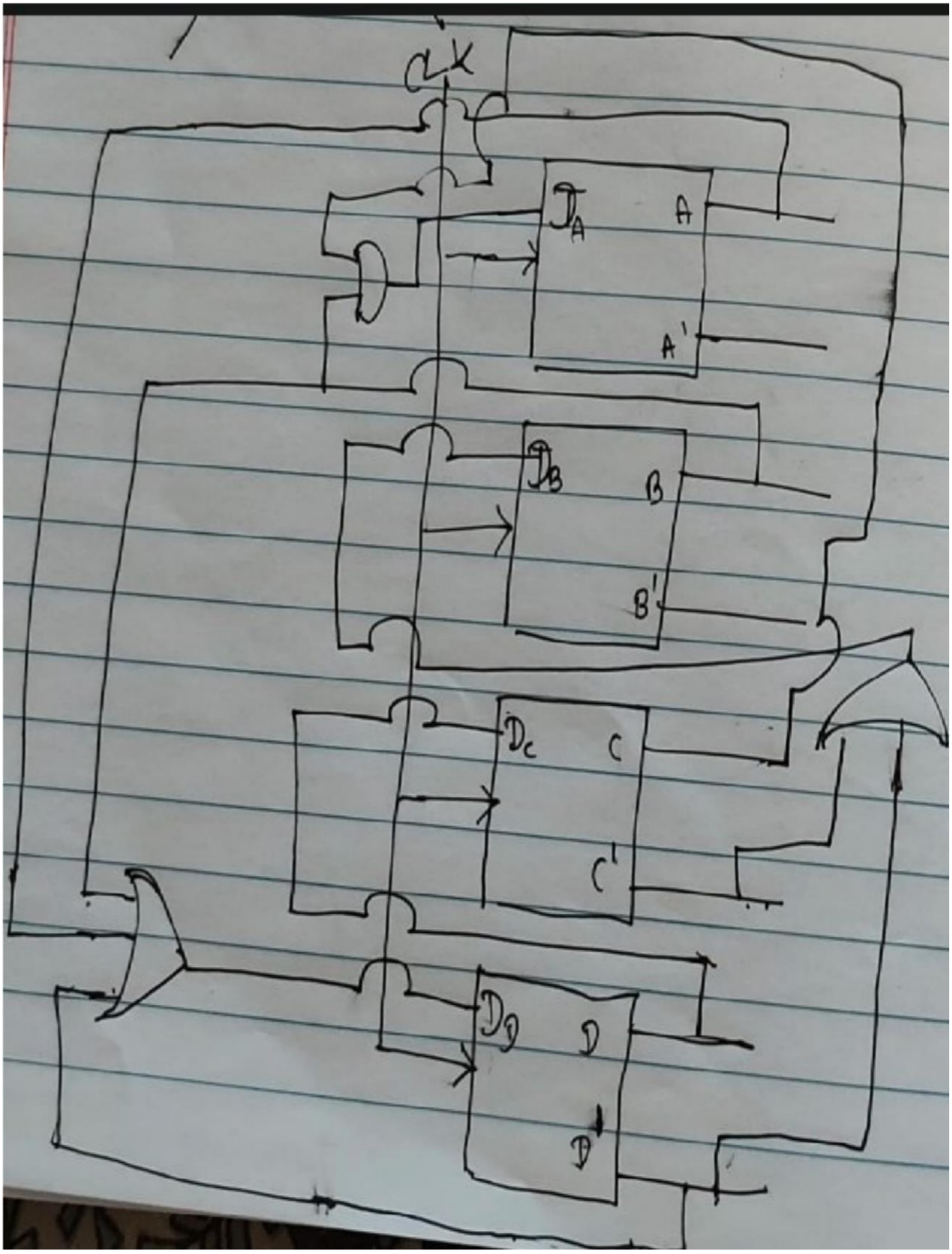
P.S				N. S				F/F			
A	B	C	D	A_+	B_+	C_+	D_+	D_A	D_B	D_C	D_D
0	0	0	0	x	x	x	x	x	x	x	x
0	0	0	1	x	x	x	x	x	x	x	x
0	0	1	0	0	1	0	1	0	1	0	1
0	0	1	1	0	0	1	0	0	0	1	0
0	1	0	0	x	x	x	x	x	x	x	x
0	1	0	1	0	1	1	1	0	1	1	1
0	1	1	0	x	x	x	x	x	x	x	x
0	1	1	1	1	0	1	1	1	0	1	1
1	0	0	0	x	x	x	x	x	x	x	x
1	0	0	1	x	x	x	x	x	x	x	x
1	0	1	0	x	x	x	x	x	x	x	x
1	0	1	1	0	0	1	1	0	0	1	1
1	1	0	0	x	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x	x

K-Map & circuit diagram:

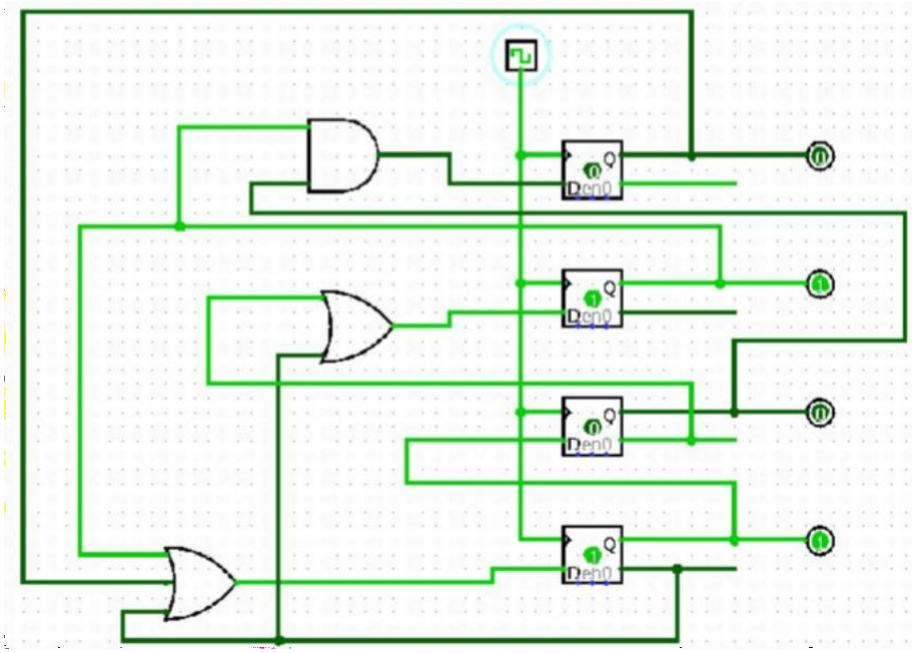


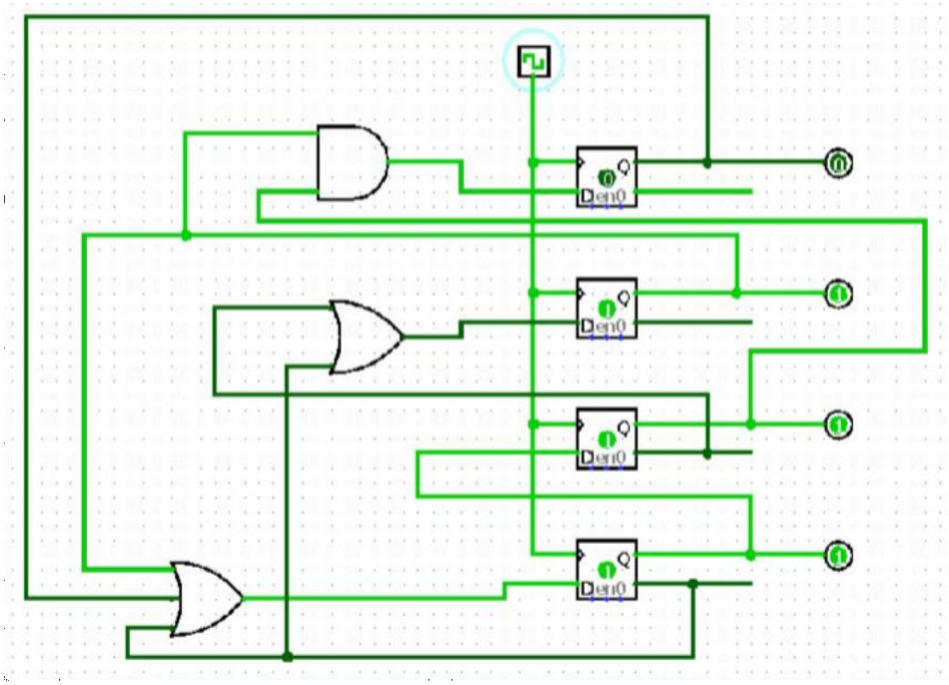


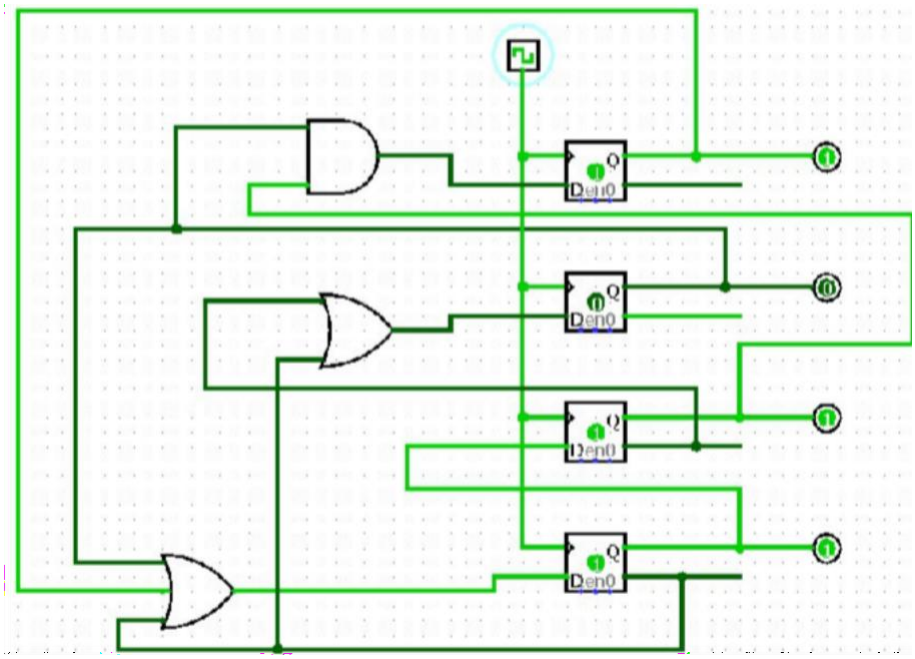
Circuit :

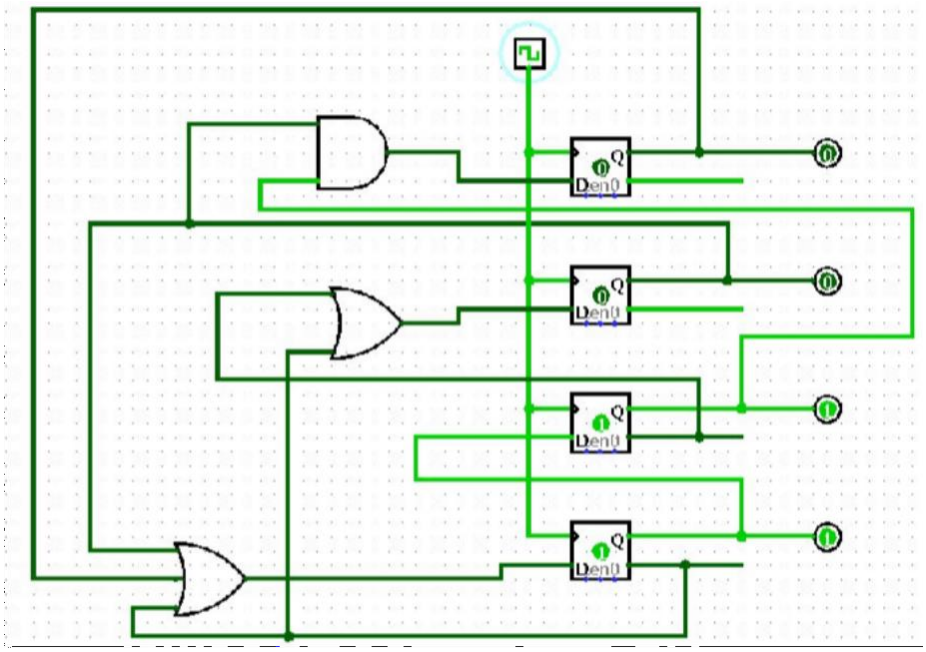


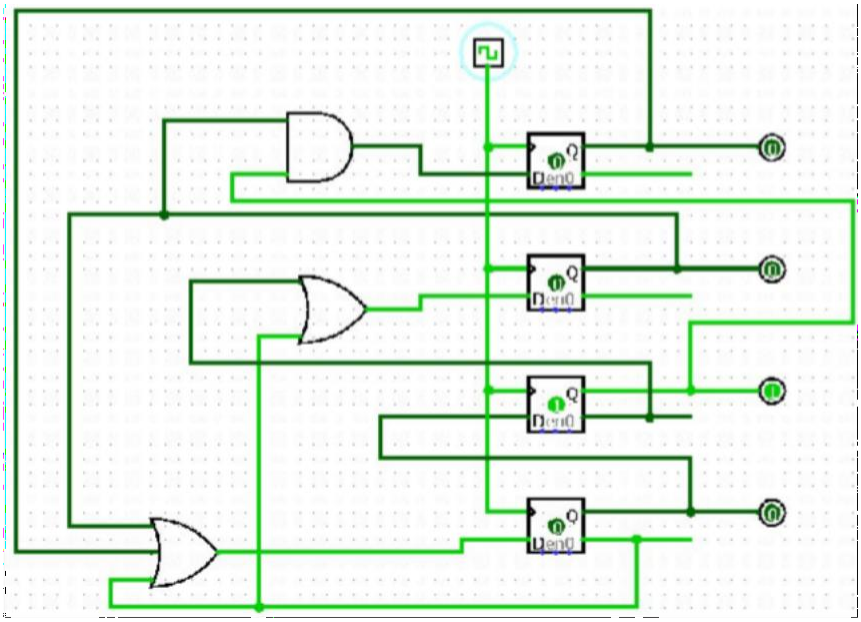
Screen shot:











Conclusion : This practical successfully demonstrates the design of a 4-bit synchronous counter tailored for a non-linear custom sequence. By utilizing state transition tables and flip-flop excitation logic, the circuit proves that synchronous systems can be programmed to skip traditional binary increments in favor of specific numerical patterns (2, 5, 7, 11, 3). The implementation confirms that by tying all flip-flop clock inputs to a common source, the system eliminates ripple delays and ensures all state changes occur simultaneously. Ultimately, the project validates the versatility of sequential logic design, showing how K-map minimization can transform a complex counting requirement into an optimized, hardware-efficient circuit.

Practical-8

Problem Definition:

The system is designed to generate a sequential lighting pattern using multiple LEDs to create a
"chasing" or running light effect.
Specifically, it performs the following operations:
-> It generates a binary count sequence using flip-flops (0 to 7)
->It uses this sequence to select one output at a time
->Only one LED is turned ON at any given moment
->The LEDs glow in a fixed order (LED0 → LED1 → LED2 → … → LED7)
->The sequence continuously repeats to produce a moving light effect
->The speed of LED transition is controlled using a clock signal
If the system is working correctly, it will create a smooth and continuous LED chasing pattern
where the light appears to move from one LED to the **next.**

Tools/Technology To Be Used:

The following tools and technologies are used to design and implement the LED Chaser Circuit:
Logisim Software – Used for designing and simulating the digital circuit
D Flip-Flops – Used to create a 3-bit counter (0 to 7 sequence)
Clock Generator – Provides timing pulses to control the speed of LED transitions
Multiplexer (8:1 MUX) – Selects one output line based on the counter input
Splitter / Bus – Used to combine and separate multi-bit signals
LEDs – Used to display the output visually
Basic Logic Gates (AND, OR, XOR, NOT) – Used to implement counter logic

Aim:

The system is designed to generate a sequential lighting pattern using multiple LEDs to create a
"chasing" or running light effect.
Specifically, it performs the following operations:
-> It generates a binary count sequence using flip-flops (0 to 7)
->It uses this sequence to select one output at a time
->Only one LED is turned ON at any given moment
->The LEDs glow in a fixed order (LED0 → LED1 → LED2 → … → LED7)
->The sequence continuously repeats to produce a moving light effect
->The speed of LED transition is controlled using a clock signal
If the system is working correctly, it will create a smooth and continuous LED chasing pattern
where the light appears to move from one LED to the **next.**

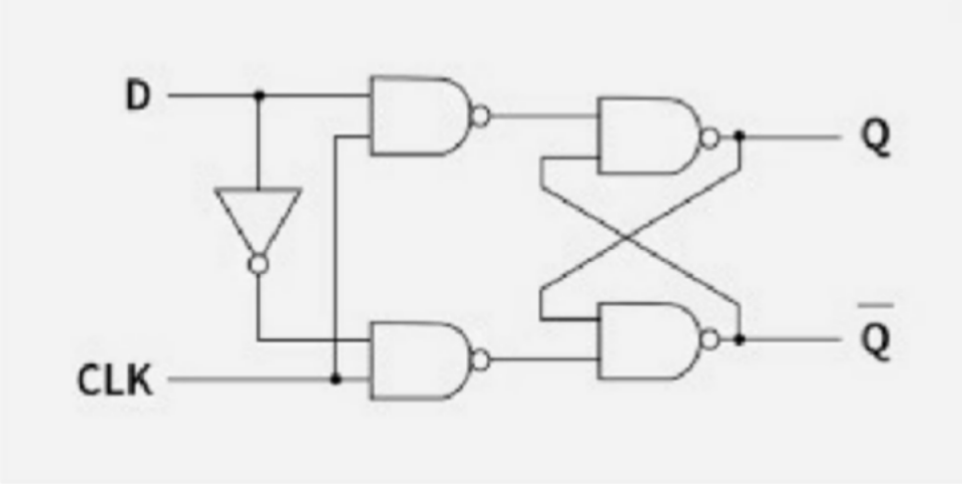
D flip flop:

Truth table:

Q(n)	D(n)	Q(n+1) (Next)
0	0	0
0	1	1
1	0	0
1	1	1

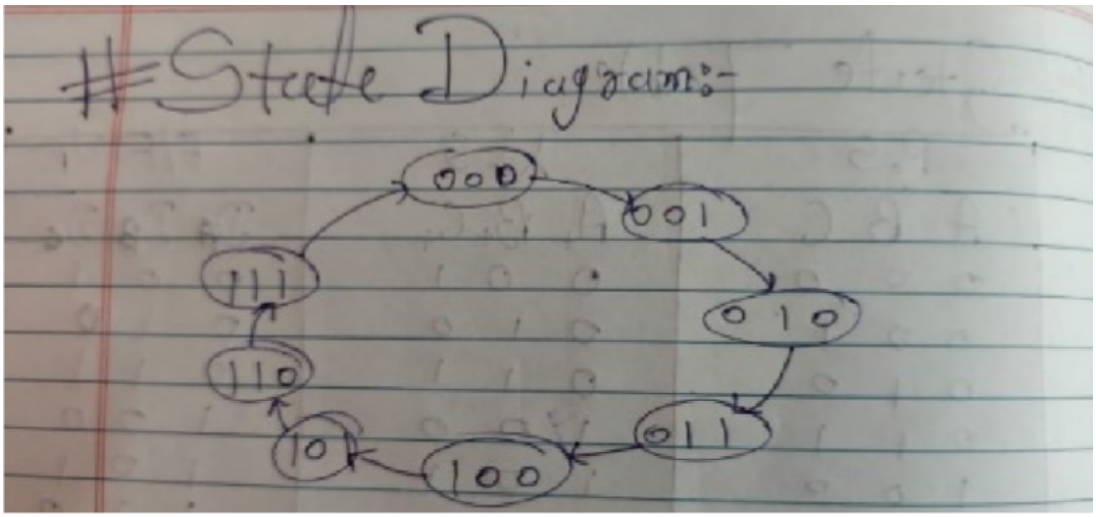
Excitation Table:

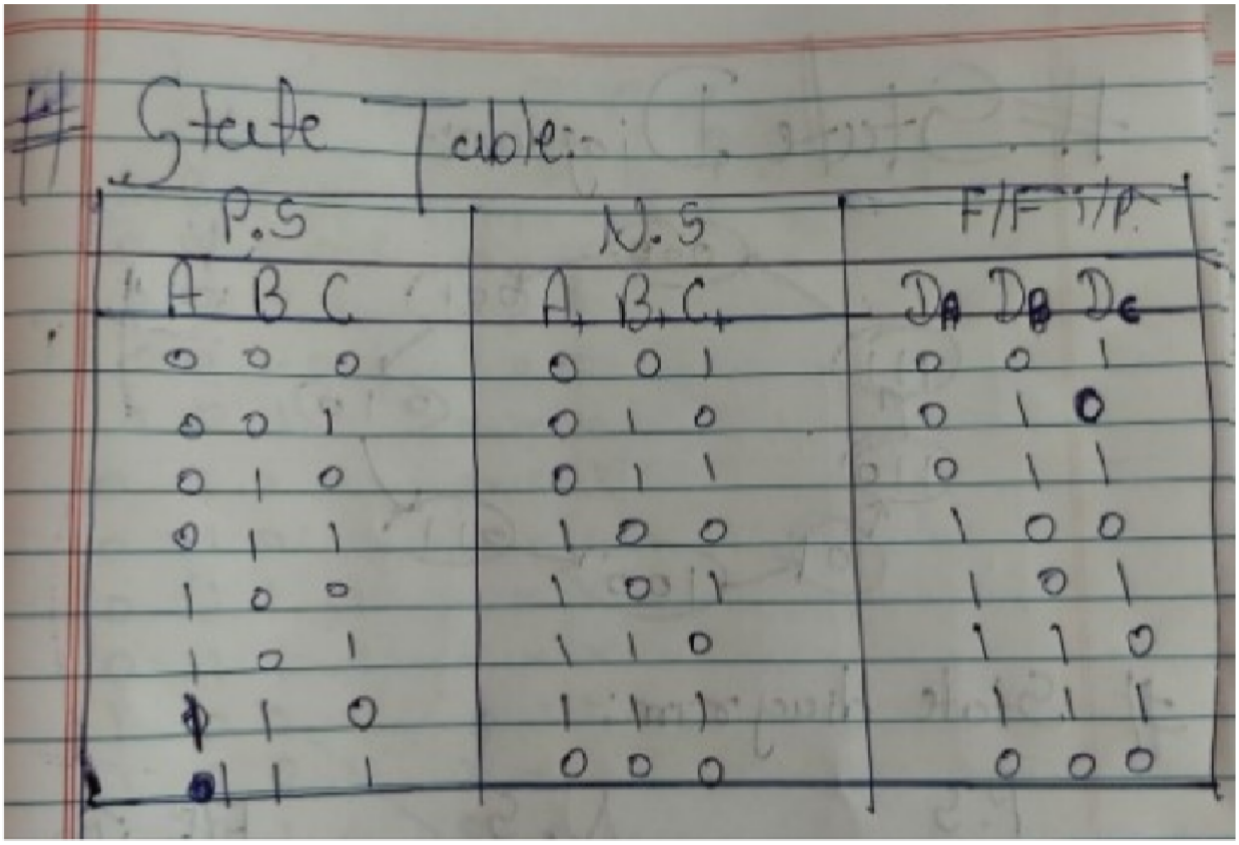
Q Output		Input
$Q_{(n)}$	$Q_{(n+1)}$	D_n
0	0	0
0	1	1
1	0	0
1	1	1



Circuit Diagram:

State diagram & State Table:

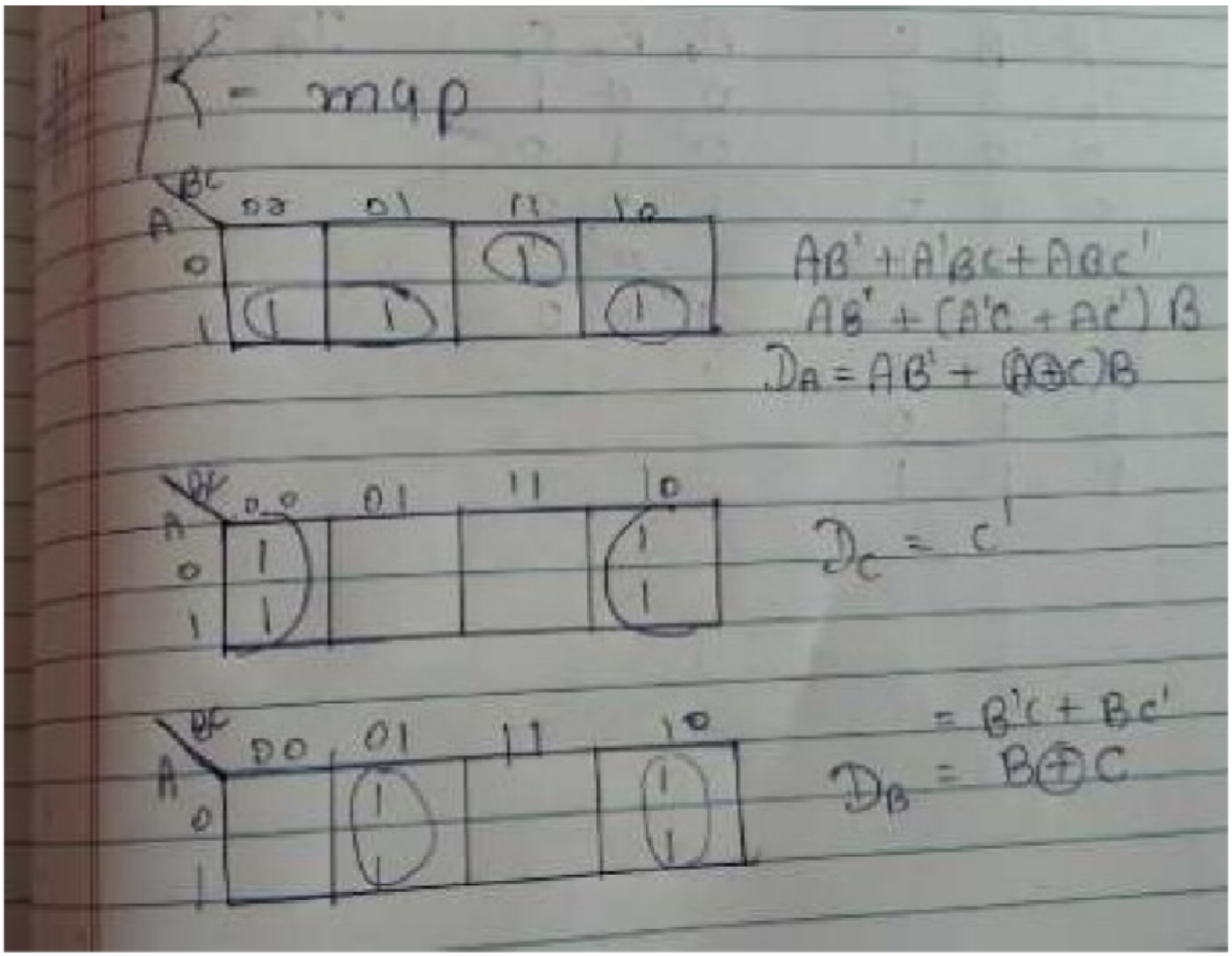




State Table:-

P.S			N.S			F/F I/P		
A	B	C	A_+	B_+	C_+	D_A	D_B	D_C
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0

K-Map & circuit diagram:



K - map

$AB' + A'Bc + ABc'$

$AB' + (A'C + Ac')B$

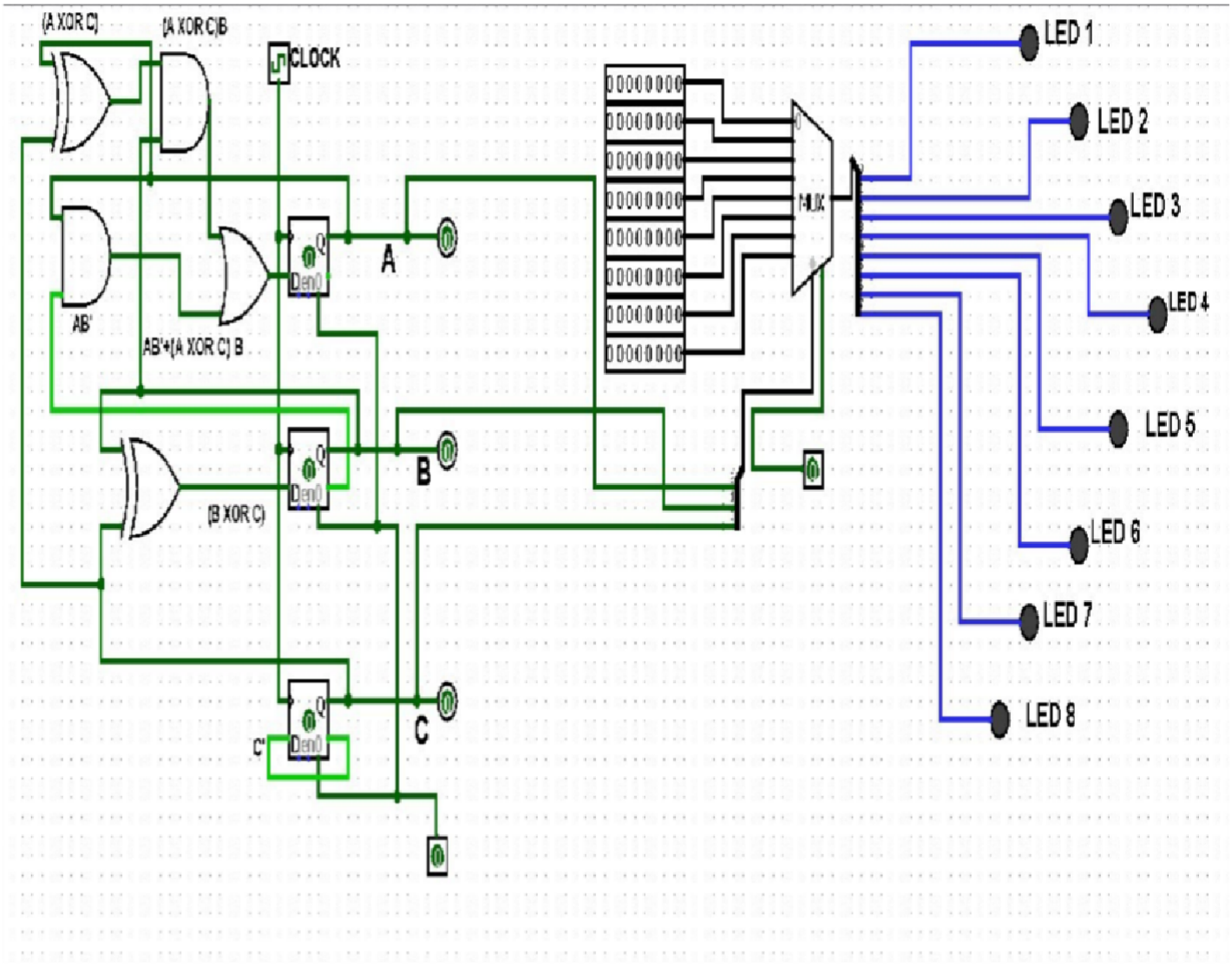
$D_A = AB' + (A \oplus C)B$

$D_C = C'$

$= B'C + Bc'$

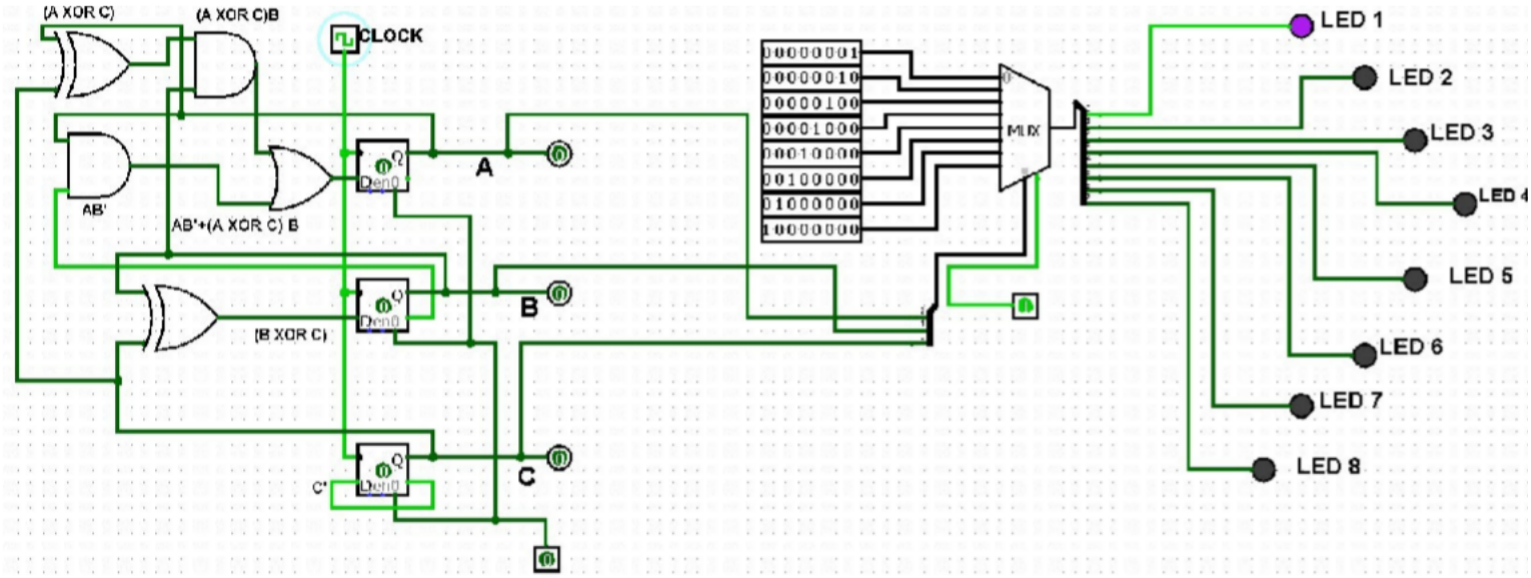
$D_B = B \oplus C$

Circuit of Logism :

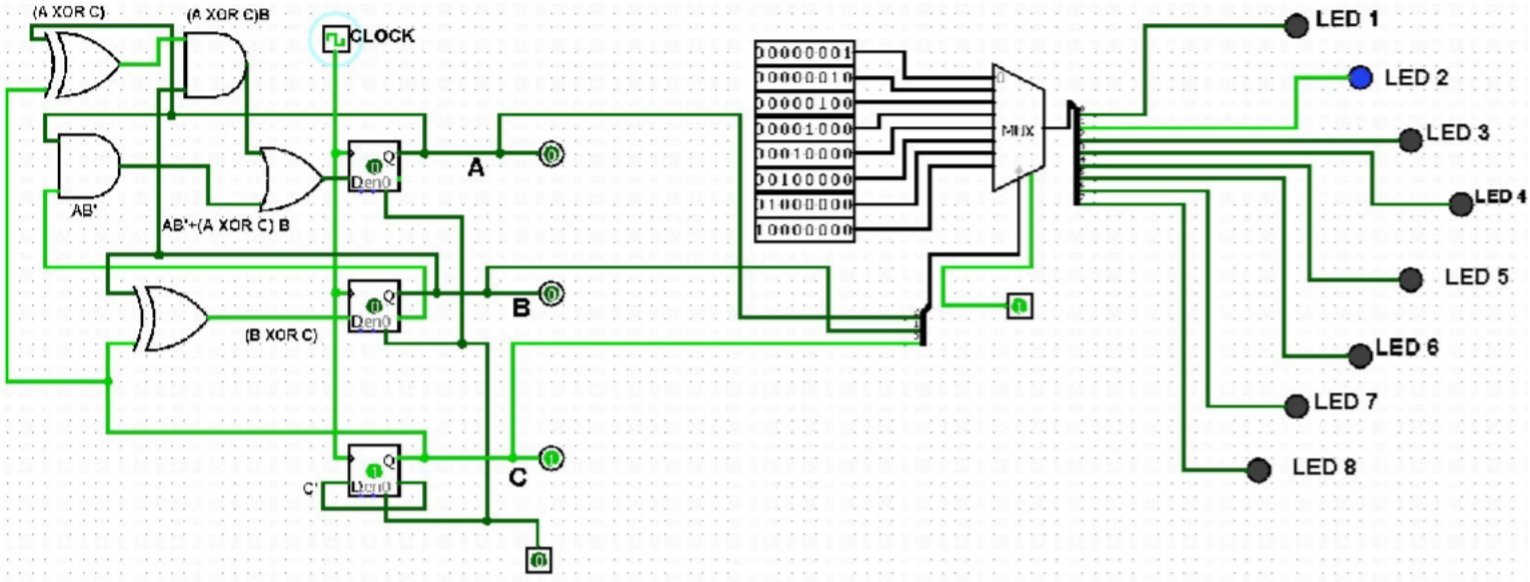


Screen shot:

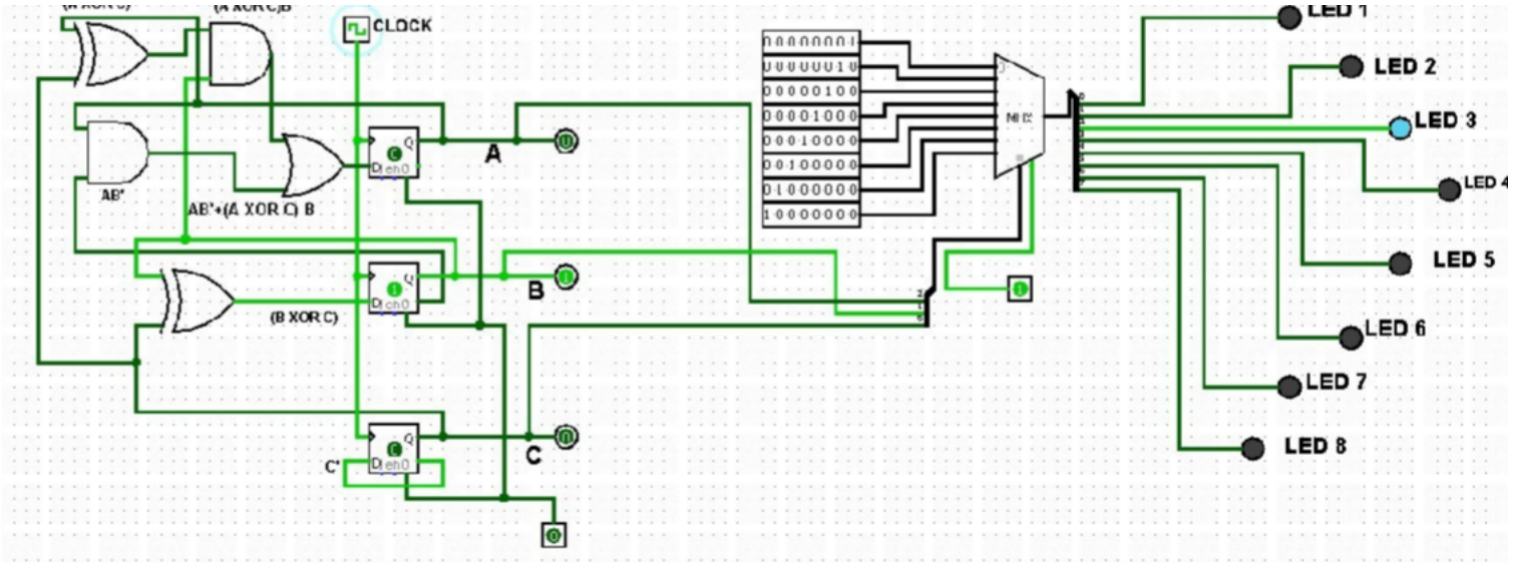
For LED 1:



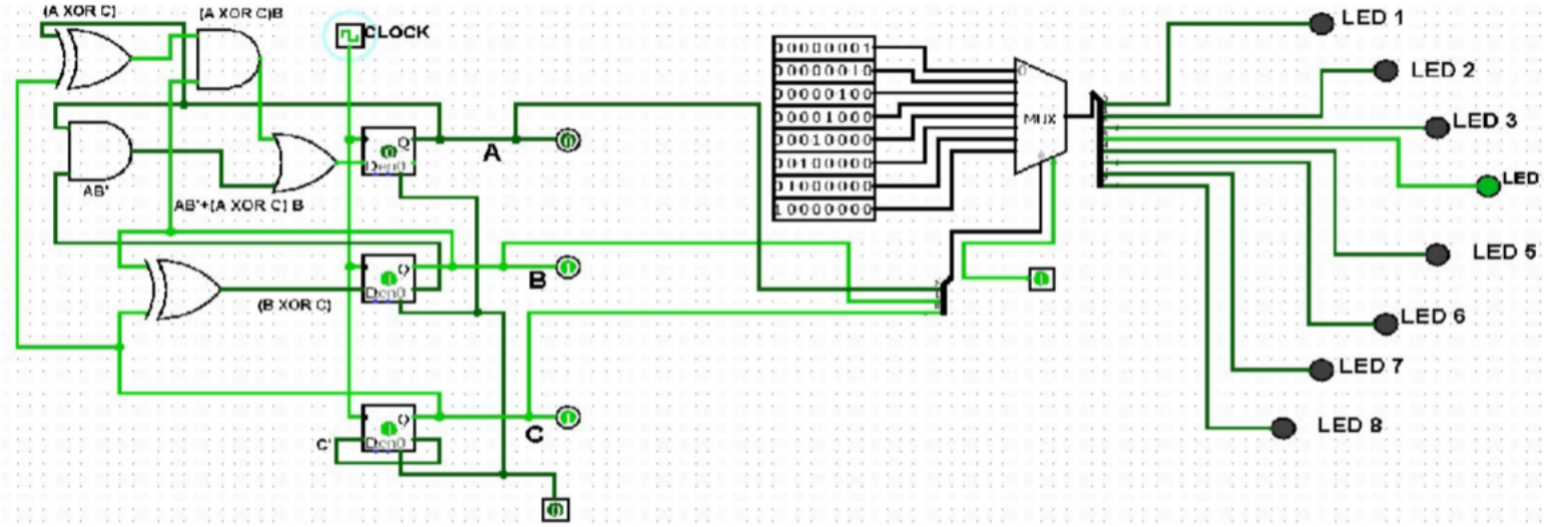
For LED 2:



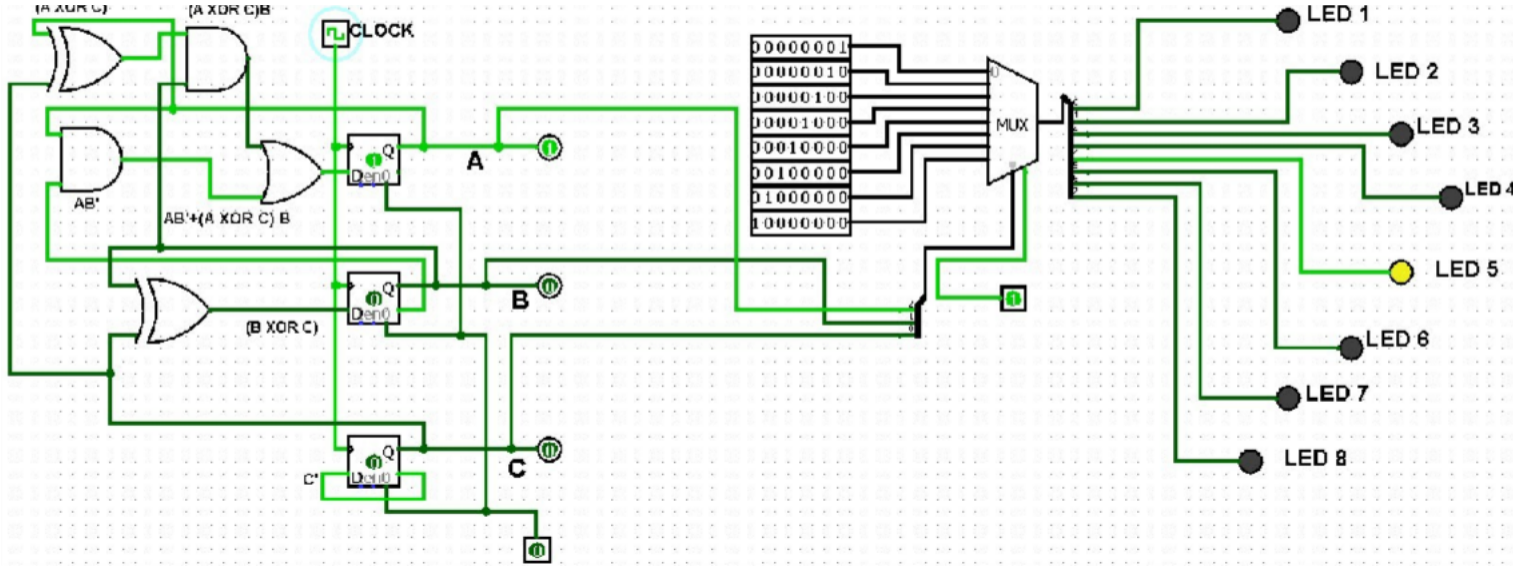
For LED 3:



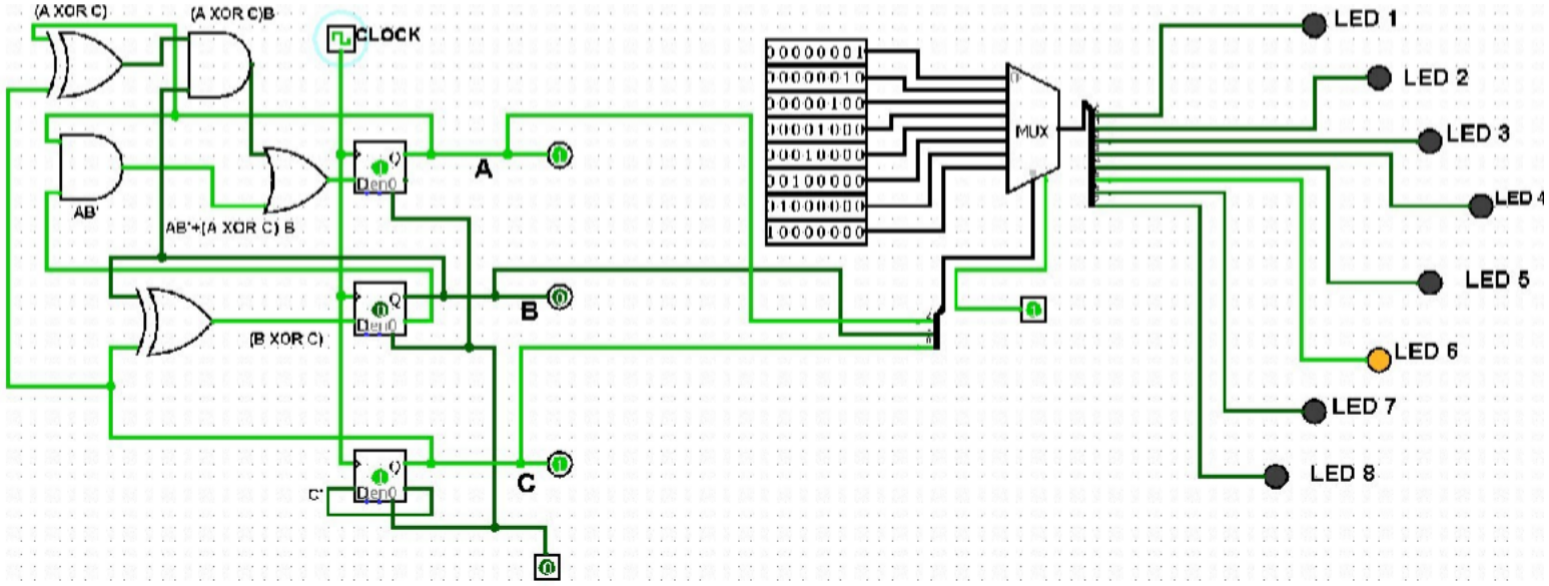
FOR LED 4:



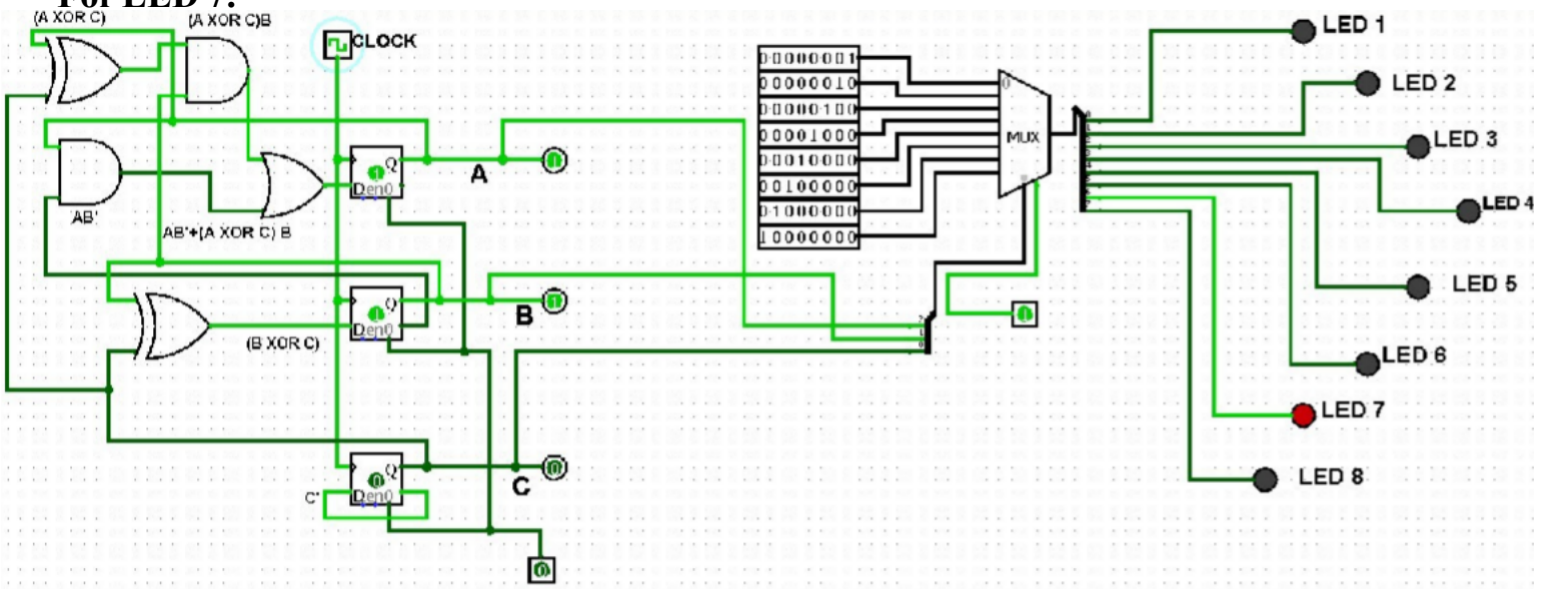
For LED 5:



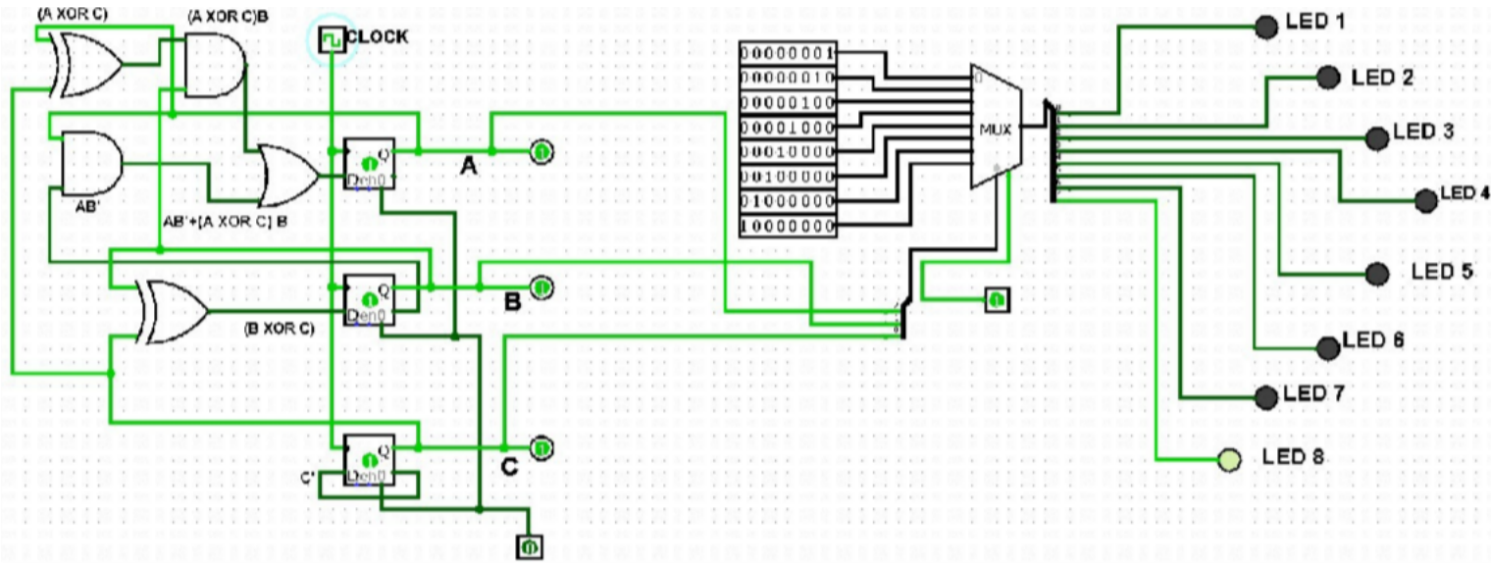
For LED 6:



For LED 7:



For LED 8:



Conclusion :

The LED Chaser Circuit was successfully designed and simulated using digital logic components in Logisim. The circuit produced a smooth sequential lighting pattern where LEDs turned ON one at a time using a counter and control logic. This project helped in understanding the practical implementation of sequential circuits, clock signals, and output control in digital systems.